

MarketHub-MVC

Multi-Vendor E-Commerce Marketplace

وثيقة المشروع الكاملة

تحليل، تصميم، تخطيط وتوثيق مشروع Full-Stack .NET لفريق من خمسة مطورين

ASP.NET Core MVC | .NET 10 (LTS) | SQL Server | Entity Framework Core | ASP.NET Core Identity | Stripe
(Test Mode)

Team: Member A | Member B | Member C | Member D | Member E

Version 1.0 | October 2026

Language: Arabic (Egyptian dialect) with English technical terms

جدول المحتويات (Table of Contents)

5	1. نظرة عامة على المشروع (Project Overview)
5	1.1 فكرة MarketHub-MVC
5	1.2 الـ Technology Stack والقرارات التقنية
5	1.3 Architecture بـ بسيطة؟
6	1.4 تصنيف المعلومات في الوثيقة
6	1.5 الافتراضات الأساسية (Assumptions)
7	2. التحليل التجاري ومتطلبات النظام (Business Analysis)
7	2.1 Executive Summary
7	2.2 Problem Statement
7	2.3 Proposed Solution
7	2.4 Project Objectives
7	2.5 Target Users
7	2.6 Business Value
8	2.7 MVP Scope و Project Scope
8	2.8 Future Enhancements و Out-of-Scope
8	2.9 Functional Requirements (FR)
9	2.10 Non-Functional Requirements (NFR)
9	2.11 Business Rules (BR)
9	2.12 Risks و Constraints
10	2.13 User Stories مع Acceptance Criteria
12	2.14 Use Cases
12	2.15 Definition of Done (على مستوى الـ Feature)
14	3. الأدوار والصلاحيات (User Roles and Authorization)
14	3.1 الفرق بين Authentication و Authorization
14	3.2 تحليل الـ Actors
14	3.3 تطبيق الـ Roles بـ ASP.NET Core Identity
16	3.4 عزل الـ Vendors (Vendor Isolation)
16	3.5 إيه إخفاء الزرار أو القائمة مش كفاية؟
17	4. الرسومات التوضيحية (Visual Diagrams)
18	Diagram 1: System Context Diagram
19	Diagram 2: Overall UML Use Case Diagram
21	Diagram 3: Customer Use Case Diagram
23	Diagram 4: Vendor Use Case Diagram
25	Diagram 5: Super Admin Use Case Diagram
27	Diagram 6A: ERD Overview (Relationships Only)
29	Diagram 6B: ERD: Identity, Vendors and Catalog
31	Diagram 6C: ERD: Orders, Payments and Commission
33	Diagram 7: High-Level System Architecture
35	Diagram 8: Application Component Diagram
37	Diagram 9: UML Class Diagram

39	Diagram 10: Deployment Architecture
41	Diagram 11: Vendor Registration and Approval (Activity)
42	Diagram 12: Product Creation (Activity)
43	Diagram 13: Shopping Cart (Activity)
45	Diagram 14: Checkout (Activity)
47	Diagram 15: Payment Processing (Sequence)
49	Diagram 16: Order Processing and Fulfillment (Sequence)
51	Diagram 17: Stripe Webhook Processing (Sequence)
53	Diagram 18: GitHub Collaboration Workflow
54	Diagram 19: Development Lifecycle (Sprint Loop)
55	Diagram 20A: Order Status State Machine
57	Diagram 20B: OrderItem Fulfillment State Machine
59	Diagram 21: Request Lifecycle (Browser to View)
60	Diagram 22: Vendor Isolation Authorization Flow
62	5. تصميم قاعدة البيانات (Database Design)
62	5.1 ازاى جداول ASP.NET Core Identity بتتوافق مع تصميمنا
62	5.2 قرار نمذجة الـ Multi-Vendor Order
62	5.3 قاموس البيانات (Data Dictionary)
67	5.4 تحليل الـ Normalization
68	5.5 إعداد EF Core (Fluent API)
69	5.6 استراتيجيات الـ Migrations
69	5.7 استراتيجيات الـ Seed Data
69	5.8 قرارات التصميم المهمة (ملخص)
70	6. معمارية النظام وهيكल المشروع (System Architecture)
70	6.1 مسؤوليات كل مكون
70	6.2 هيكل الفولدرات المقترح
71	6.3 ليه Areas/Admin و Areas/Vendor؟ (تقييم)
71	6.4 دورة حياة الـ Request كاملة
72	6.5 أمثلة كود
74	6.6 Error Handling و Logging و Validation
75	7. مواصفات الـ Features (Feature Specifications)
75	7.1 A. Public Storefront (Home, Listing, Details, Categories, Search, Filter, Sort, Pagination)
75	7.2 B. Authentication (Registration, Login, Logout, Roles)
76	7.3 C. Vendor Management (Application, Approval, Dashboard, Store Profile, Order Items, Summaries)
76	7.4 D. Product Management (Create, Edit, Images, Categories, Price/Stock Validation, Active/Inactive)
77	7.5 E. Shopping Cart (Add, Remove, Update, Stock Validation, Server-Side Price, Unavailable Items)
77	7.6 F. Orders and Checkout (Validation, Order Creation, OrderItems, Statuses, History, Vendor Views, Cancellation)
78	7.7 G. Payment Processing (Stripe Test Mode, Webhooks, Idempotency, Failed Payments)
78	7.8 H. Commission System (Settings, Calculation, Net Amount, Records, Snapshots, Reporting)
79	7.9 I. Super Admin Dashboard (Approvals, Categories, Orders, Commission, Analytics, Moderation)

80	8. معمارية Stripe والنظام المالي (Stripe and Financial Architecture)
80	8.1 أربع حاجات لازم نفرّق بينهم
80	8.2 ليه Stripe Checkout (Hosted) بدل بناء فورم كارت؟
80	8.3 ال Workflow الكامل (11 خطوة)
81	8.4 الكود الأساسي
83	8.5 حالات الفشل ومخاطر الاتساق (Consistency Risks)
83	8.6 قواعد الأمان في الدفع
83	8.7 تجربة ال Webhooks محلياً
84	8.8 Stripe Connect كمرحلة مستقبلية
85	9. خارطة الطريق التفصيلية (Development Roadmap)
85	9.1 ترتيب التنفيذ ولماذا نبني بشكل تدريجي (Incremental)
85	9.2 ربط ال Phases بال Sprints
85	9.3 تفاصيل كل Phase
96	10. خطة التعاون للفريق (Five-Member Collaboration Plan)
96	10.1 الفلسفة: خمسة أشخاص، مشروع واحد مفهوم عند الكل
96	10.2 الأدوار الدوّارة (Rotating Leadership Roles)
96	10.3 جدول المسؤوليات الدوّارة (Responsibility Matrix)
97	10.4 خطة كل Sprint بالتفصيل
103	10.5 مثال عملي: الفريق كله على Feature واحدة (Vendor Product Management, Sprint 3)
103	10.6 تنسيق التعديلات على الملفات المشتركة
103	10.7 ازاى نقل ال Merge Conflicts ونضمن إن الكل فاهم؟
105	11. سير العمل على Git و GitHub (Git and GitHub Workflow)
105	11.1 إعداد ال Repository
105	11.2 تسمية ال Branches وال Commits
105	11.3 مثال كامل: Feature صغيرة من الأول للآخر
106	11.4 حل ال Merge Conflicts
106	11.5 GitHub Issues و Project Board و Milestones
107	11.6 README و Environment Configuration و إدارة ال Secrets
107	11.7 تنسيق ال EF Core Migrations بين الأعضاء
107	11.8 مثال CI بسيط (GitHub Actions)
108	12. استراتيجية الاختبار وضمان الجودة (Testing and QA)
108	12.1 أنواع الاختبارات ومكانها في المشروع
108	12.2 هرم الاختبارات في مشروعنا
108	12.3 Test Cases الأساسية (ال 15 مجال المطلوبين)
110	12.4 مثال Authorization Test (xUnit + WebApplicationFactory)
110	12.5 مثال Unit Test للعمولة
110	12.6 ال Collective Testing و ال Regression
111	13. الأمان والموثوقية (Security and Reliability)
112	مثال عملي: لماذا نرفض السعر القادم من ال Browser؟
112	مثال عملي: ال Idempotency
113	14. النشر والتسليم النهائي (Deployment and Final Delivery)

113	14.1 ثلاث بيئات مختلفة لازم نفرّق بينها
113	14.2 خيارات الاستضافة [Team Decision]
113	14.3 خطوات النشر
114	14.4 Demo Accounts
114	14.5 README النهائي و Screenshots و Demonstration
115	15. خطة العرض النهائي (Final Presentation)
115	15.1 هيكل العرض (حوالي 20 إلى 25 دقيقة [Team Decision])
115	15.2 سيناريوهات الـ Live Demo المقترحة
115	15.3 نصائح التحضير
117	16. المراجع والروابط الرسمية (References)
117	Microsoft
117	Stripe
118	أدوات ومصادر أخرى
118	ملخص ما تمّ التحقق منه وما لم يتمّ

1. نظرة عامة على المشروع (Project Overview)

1.1 فكرة MarketHub-MVC

MarketHub-MVC هو موقع **Multi-Vendor E-Commerce Marketplace**. يعني بدل ما يبقى فيه متجر واحد بيبيع منتجاته، عندنا منصة بيسجل فيها أكثر من **Vendor**، وكل **Vendor** عنده متجره ومنتجاته ومخزونه. الـ **Customer** بيتصفح منتجات من متاجر مختلفة، يحطها في **Shopping Cart** واحد، ويدفع مرة واحدة بـ **Stripe (Test Mode)**. والـ **Super Admin** هو اللي بيدير المنصة: يوافق على الـ **Vendors**، يدبر الـ **Categories**، ويحدد نسبة العمولة (**Commission**). الفرق الجوهرى بين المتجر العادى والـ **Marketplace** إن كل صف في الـ **Order** ممكن يتبع **Vendor** مختلف، وده بيأثر على: الـ **Database Design**، الـ **Authorization**، الـ **Inventory**، حساب العمولة، وحتى شكل الـ **Order Status**. فهم النقطة دي هو مفتاح المشروع كله.

1.2 الـ Technology Stack والقرارات التقنية

المكون	القرار	ليه اخترناه؟
Language	#C	اللغة الرسمية لـ .NET. وكل الكورس بتاعنا عليها.
.NET Version	.NET 10 (LTS)	Microsoft Support Policy: .NET 10 اتصدر في November 11, 2025 وهو LTS ودعاه لحد November 14, 2028. أما .NET 8 فدعاه بينتهي November 10, 2026. يعني قبل ما نخلص المشروع بشهور.
Web Framework	ASP.NET Core MVC	بيفصل الـ Controllers عن الـ Views، وبيدينا Server-Side Rendering سهل للمبتدئين، ومناسب لـ Razor Views.
Database	SQL Server	Relational وده الأنسب لبيانات مالية (Orders, Payments) محتاجة Transactions و Constraints.
ORM	Entity Framework Core	بيحول الـ LINQ لـ SQL، وبيدبر الـ Migrations. وبيحمينا من SQL Injection لأنه بيستخدم Parameters.
Auth	ASP.NET Core Identity	Authentication + Roles + Password Hashing جاهزين ومجربين.
Frontend	HTML, CSS, JavaScript, Bootstrap 5	سريع ومعروف، ومش محتاجين Single Page Application.
Payments	Stripe Checkout (Test Mode) + Stripe.net	الـ Hosted Checkout بيقلل مخاطر الـ PCI لأن بيانات الكارت مبتعديش على سيرفرنا.
Source Control	Git + GitHub	Pull Requests و Code Reviews و Issues.

معلومة مهمة: رقم الـ patch بتاع .NET بيتغير شهرياً. استخدموا آخر SDK متاح من الموقع الرسمي، ودوروا على **global.json** لتثبيت الـ SDK عند كل الفريق.

1.3 الـ Architecture بسيطة؟

فريقنا 5 طلاب بيتعلموا. عشان كده اخترنا **Single Web Project (Modular Monolith بسيط)** بدل Microservices أو Clean Architecture بـ 5 projects:

- Project واحد اسمه **MarketHub.Web** فيه الـ Controllers والـ Services والـ Data.
- Project تاني للـ Tests اسمه **MarketHub.Tests**.
- مفيش **Repository Pattern**: الـ **DbContext** في الـ EF Core هو بالفعل Unit of Work وبيشتغل كـ Repository. لو لقيناه في Repository تاني هنزود Code من غير فائدة.
- الـ Services عندها Interfaces لسببين واضحين: الـ Dependency Injection والـ Unit Testing (نقدر نعمل Mock).

1.4 تصنيف المعلومات في الوثيقة

علشان نفرّق بين الحقائق والتوصيات، هنستخدم الوسوم دي:

الوسم	المعنى
[Verified]	اتأكدنا منها من Official Documentation وقت كتابة الوثيقة (October 2026).
[Recommendation]	تصميم بنوصي بيه، والفريق ممكن يغيّره بسبب مقنع.
[Assumption]	افتراض احنا حطيناه، لازم الفريق يتأكد منه.
[Team Decision]	قرار لازم الفريق ياخده بنفسه (مفيش إجابة صح واحدة).

الحقائق اللي اتحققنا منها [Verified]

- NET 10 هو LTS وفي Active support وينتهي دعمه November 14, 2028. و NET 8 في Maintenance وينتهي November 10, 2026 (Microsoft .NET Support Policy).
- في Stripe Checkout، الحدث `checkout.session.completed` بيتبعث لما العميل يدفع، والطرق المتأخرة (Delayed payment methods) بتبعث `checkout.session.async_payment_succeeded` لما الدفع ينجح بعدين، وممكن نسمع كمان لـ `checkout.session.async_payment_failed` (Stripe docs: Fulfill orders).
- لازم نتحقق من توقيع الـ Webhook (Signature Verification) باستخدام الـ `raw request body` وهيدر `Stripe-Signature` وسر الـ `(whsec_...)` Endpoint. ولازم الـ Handler يتعامل صح لو اتنادى أكثر من مرة لنفس الـ Checkout Session.
- الـ Stripe CLI بيوفر الأمر `stripe listen --forward-to localhost:PORT/path` لتجربة الـ Webhooks محلياً.

أشياء لازم الفريق يتأكد منها قبل الاعتماد عليها [Team Decision]

- الاسم الدقيق لدوال الـ `Stripe.net` في النسخة اللي هتتثبت (مثلاً `EventUtility.ConstructEvent` و `Stripe.Checkout.SessionService`): راجعوا الـ API Reference الرسمي لأن الأسماء والـ Signatures ممكن تتغير بين الإصدارات.
- أقل وأقصى مدة لـ `expires_at` في Checkout Session (بنفترض إن الحد الأدنى 30 دقيقة، [Assumption] راجعوها في الـ Stripe API docs).
- الدول والعملات المدعومة لحساب Stripe بتاعكم، وهل Stripe Connect متاح في بلدكم (مهم للمرحلة المستقبلية فقط).
- مزود الاستضافة (Hosting) ونوع الـ SQL Server المستضاف.

1.5 الافتراضات الأساسية (Assumptions)

- العملة الوحيدة في الـ MVP هي الـ USD (Stripe Test Mode)، والعمود `Currency` موجود للتوسع المستقبلي.
- مفيش Shipping Fee ولا ضرائب في الـ MVP: `TotalAmount` = مجموع `LineTotal`.
- الـ Cart بيتخزن في الـ Database للـ Customer المسجل فقط، والـ Guest لازم يسجل دخول قبل الإضافة للـ Cart.
- كل Customer عنده Role واحدة (Customer)، والـ Vendor عنده `Vendor`، والـ Admin عنده `SuperAdmin`.
- Vendor Application بتتعمل مع إنشاء الحساب في نفس الخطوة؛ الـ `Vendor` Role بتتضاف بعد موافقة الـ Admin فقط.
- الصور بتتخزن على الـ File System داخل `wwwroot/uploads` في الـ MVP.
- الفريق عنده 5 أعضاء، كل Sprint أسبوعين [Team Decision]، ولو الوقت ضيق ممكن يتضغط لأسبوع مع تقليل الـ Scope.

2. التحليل التجاري ومتطلبات النظام (Business Analysis)

ال Business Analysis هو المرحلة التي بنجواب فيها: إحنا بنبنّي إيه؟ ولمين؟ وليه؟ قبل ما نكتب سطر Code واحد. لو اتخطينا المرحلة دي هنكتشف في نص المشروع إن كل واحد فينا فاهم المشروع بشكل مختلف. كل الفريق (الخمس) لازم يحضر جلسات التحليل دي مع بعض.

Executive Summary 2.1

MarketHub-MVC منصة ويب بتجمع أكثر من بائع (Vendor) في مكان واحد. العميل بيشتري من كذا Vendor في عملية دفع واحدة، والمنصة بتسجل عمولتها على كل عنصر في الطلب، وال Vendor بيحسب الطلبات الخاصة بيه بس. ال MVP بيركز على تجربة الشراء الكاملة (من التصفح لحد الدفع في Stripe Test Mode) ولوحات تحكم بسيطة لل Vendor وال Admin.

Problem Statement 2.2

البائعين الصغار مش دايماً عندهم موقع خاص بيهم، وبناء متجر مستقل مكلف ومحتاج مهارات. والعملاء بيفضلوا يلاقوا كذا بائع في مكان واحد بدل ما يدخلوا على مواقع كتير. وعلى مستوى المطور، مفيش مشروع تعليمي أفضل من ال Marketplace لأنه بيجمع Authentication, Authorization, Relational Design, Transactions, Payments.

Proposed Solution 2.3

منصة ASP.NET Core MVC فيها ثلاث بوابات: Storefront عام، Vendor Area، و Admin Area. كل المصاحيات بتتفرض على ال Server، وكل الحسابات المالية بتتخسب على ال Server من ال Database، ومش من اللي جاي من ال Browser.

Project Objectives 2.4

#	الهدف	مقياس النجاح
O1	تجربة شراء كاملة Multi-Vendor	عميل يشتري منتجات من Vendor مختلفين في Checkout واحد ويدفع بـ Stripe Test Mode.
O2	عزل بيانات ال Vendors	مفيش Test Case في Authorization يسمح لـ Vendor A بالوصول لبيانات Vendor B.
O3	دفع آمن وموثوق	ال Order بيتحول لـ Paid من ال Webhook الموقع فقط، ومفيش تكرار لو ال Webhook اتبعت مرتين.
O4	حساب عمولة صحيح وقابل للتتبع	كل OrderItem مدفوع له CommissionRecord فيه نسبة العمولة وقت الشراء.
O5	تعلم جماعي	كل عضو شارك في كل Sprint وعمل Code Review وفهم ال Flow كامل.

Target Users 2.5

- Guest/Customer**: مشتري عادي بيدور على منتجات ويجب الدفع بسرعة.
- Vendor**: بائع صغير يعرض منتجاته ويتابع طلباته ومبيعاته.
- Super Admin**: مدير المنصة المسؤول عن الجودة والعمولات.

Business Value 2.6

المنصة بتكسب من العمولة على كل عملية بيع (Commission)، ويتوفر لل Vendor قاعدة عملاء جاهزة. بالنسبة لينا كفريق، القيمة هي مشروع Portfolio حقيقي بيثبت إننا نعرف نبني Full-Stack .NET.

MVP Scope و Project Scope 2.7

MVP (Must Have): Registration/Login. Vendor Application + Approval. Categories. Product ال داخل ال Search/Filter/Sort/Pagination. Shopping Cart. Checkout. Order + OrderItems. Stripe صور، + CRUD .Checkout (Test Mode) + Webhook. Commission Records. Vendor Dashboard. Admin Dashboard Reviews. Customer Cancel Order. Product Moderation. Basic Analytics (لو الوقت سمح): Charts .

Future Enhancements و Out-of-Scope 2.8

Future Enhancement	Out-of-Scope (MVP)
Stripe Connect (Connected Accounts + Transfers)	تحويل فلول حقيقيه للـ Vendors
Shipping Providers API	Shipping Fees وحساب التوصيل
Refund Workflow + CommissionRecord.Status = Reversed	Refunds تلقائية
Promotions Engine	Coupons وخصومات
ProductVariant entity	Product Variants (مقاسات/ألوان)
SMTP/SendGrid + Background Jobs	Email Notifications
Currency conversion service	Multi-Currency
Web API + JWT	Mobile App / API

Functional Requirements (FR) 2.9

ID	المتطلب	الأولوية
FR-01	الـ Guest يقدر يتصفح المنتجات ويبحث ويغفلتر ويرتب مع Pagination.	Must
FR-02	المستخدم يقدر يسجل حساب Customer ويدخل ويخرج.	Must
FR-03	المستخدم يقدر يقدم Vendor Application بـ Store Name فريد.	Must
FR-04	الـ Super Admin يوافق أو يرفض الـ Vendor Application مع سبب للرفض.	Must
FR-05	الـ Vendor المعتمد يدير Product CRUD وصور وActive/Inactive وStock.	Must
FR-06	الـ Customer يدير الـ Cart (إضافة، حذف، تعديل كمية).	Must
FR-07	الـ Checkout يتحقق من الأسعار والمخزون من الـ Database وينشئ الـ Order.	Must
FR-08	الدفع بـ Stripe Checkout ومعالجة الـ Webhook بشكل Idempotent.	Must
FR-09	حساب CommissionRecord لكل OrderItem بعد نجاح الدفع.	Must
FR-10	الـ Vendor يشوف OrderItems بتاعته فقط ويحدّث Fulfillment Status.	Must
FR-11	الـ Customer يشوف تاريخ طلباته وحالاتها.	Must
FR-12	الـ Admin يدير Categories ويضبط نسبة العمولة ويراقب الـ Orders.	Must
FR-13	Vendor Dashboard: ملخص مبيعات وعمولات.	Must
FR-14	Admin Dashboard: تقارير عامة للمنصة.	Should
FR-15	الـ Customer يكتب Review على منتج استلمه (Delivered).	Should
FR-16	الـ Customer يلغي Order لسه PendingPayment .	Should
FR-17	الـ Admin يوقف (Suspend) Vendor أو Product مخالف.	Should

Non-Functional Requirements (NFR) 2.10

ID	الفئة	المتطلب
NFR-01	Security	كل ال Authorization بيتفرض على ال Server، وكل ال POST محمي بـ AntiForgery.
NFR-02	Security	ال (Stripe keys, connection strings) Secrets مبيتحطش في GitHub أبدأ.
NFR-03	Reliability	معالجة ال Webhook وال Inventory داخل Database Transaction.
NFR-04	Performance	صفحة Product Listing بترجع بـ Pagination (12-24 منتج) وبـ Index على الأعمدة المستخدمة في الفلترة.
NFR-05	Usability	الموقع Responsive على الموبايل (Bootstrap Grid).
NFR-06	Maintainability	كل Feature ليها Service واضحة و Tests أساسية.
NFR-07	Auditability	الأسعار والعمولات بتتسجل كـ Snapshot وقت الشراء.
NFR-08	Compatibility	ببشتغل على آخر نسخة من Chrome و Edge و Firefox.

Business Rules (BR) 2.11

ID	القاعدة
BR-01	ال Vendor لازم يكون Approved علشان يضيف منتجات أو تظهر منتجاته للعملاء.
BR-02	ال Vendor يشوف ويعدل بياناته هو فقط (Products, OrderItems, Commissions).
BR-03	السعر الأساسي هو اللي في Products.Price وقت ال Checkout، ومبتثقش في أي سعر جاي من ال Browser.
BR-04	الكمية في ال Cart وال Order لازم تكون من 1 لحد المخزون المتاح.
BR-05	المخزون بيتحجز (Reserve) لحظة إنشاء ال Order ويتحرر لو ال Order اتلغى أو ال Session انتهت.
BR-06	ال Order بيبقى Paid فقط بعد Webhook موقع، ومش بعد ال Redirect.
BR-07	العمولة = $\text{LineTotal} \times \text{RatePercent} / 100$ بتقريب لأقرب سنت، ونسبة العمولة بتتسجل في CommissionRecord وقت الدفع.
BR-08	VendorNetAmount = GrossAmount - CommissionAmount
BR-09	لا يمكن حذف Product له OrderItems: بنعمل Deactivate بدل الحذف.
BR-10	ال Fulfillment Status مسموح بس بالترتيب Pending → Shipped → Delivered ، وبعد ما ال Order بيبقى Paid .
BR-11	ال Order بيبقى Completed لما كل ال OrderItems غير الملغية تبقى Delivered .
BR-12	Review مسموح بس لـ Customer اشترى المنتج واستلمه، وبواقع Review واحد لكل (Customer, Product).

Risks و Constraints 2.12

Constraints: مدة المشروع محدودة، الفريق مبتدئ، Stripe في Test Mode فقط، ومغيش Budget للاستضافة.

الخطر (Risk)	الاحتمال	التأثير	التخفيف (Mitigation)
تعارض ال Migrations بين الأعضاء	عالي	متوسط	قواعد Migration في الفصل 11 (عضو واحد في كل مرة + Rebase).
Overselling (بيع مخزون أكثر من الموجود)	متوسط	عالي	Reserve بـ Conditional UPDATE داخل Transaction + RowVersion .
Webhook مكرر أو متأخر	عالي	عالي	جدول StripeWebhookEvent بـ Unique على StripeEventId .
تسريب Secrets	متوسط	عالي	Secret Scanning + .gitignore + dotnet user-secrets .
Scope Creep	عالي	متوسط	قائمة Out-of-Scope + مراجعة Backlog في كل Sprint Planning.

الخطر (Risk)	الاحتمال	التأثير	التخفيف (Mitigation)
عضو مش فاهم جزء من النظام	متوسط	عالي	Pair Programming + Rotating Leadership + Knowledge Sessions.

Acceptance Criteria مع User Stories 2.13

US-06: Add products from different vendors to cart

As a Customer, I want to add products from different vendors to my cart **so that** I can purchase them through the marketplace.

الشرح بالعربي: العميل لازم يقدر يحط منتجات من أكثر من متجر في نفس الـ Cart. بنخزن في CartItems مين العميل وأنهى منتج وكام قطعة، لكن مش بنخزن السعر، لأن السعر بيتجاب من Products كل مرة.

العنصر	التفاصيل
Acceptance Criteria	(1) المنتج لازم يكون IsActive ومتجره (2) Approved. الكمية الكلية للمنتج في الـ $Cart \leq StockQuantity$. (3) لو المنتج موجود بالفعل، الكمية تزيد ومفیش صف مكرر. (4) الـ Cart بيعرض السعر الحالي من الـ Database.
Main Flow	Customer يضغط <code>CartService.AddAsync</code> ← <code>CartController.Add</code> ← Add to Cart ← يحمل المنتج من الـ DB ويتحقق ← Insert/Update في <code>CartItems</code> ← Redirect لصفحة الـ Cart.
Alternative Flows	Redirect ← Guest لـ Login ثم الرجوع. المنتج موجود ← زيادة الكمية.
Error Scenarios	منتج غير موجود (404)، Inactive، متجره Suspended، كمية صفر أو سالبة، تخطي المخزون.
Entities	CartItem, Product, VendorProfile, ApplicationUser
Components	CartController, ICartService, ApplicationDbContext, CartViewModel

US-09: Pay for my order using Stripe

As a Customer, I want to pay for my order securely online **so that** my purchase is confirmed without sharing my card with the marketplace.

الشرح: احنا مبستقبلش بيانات الكارت. العميل بيتحول لصفحة Stripe المستضافة، وبعد الدفع Stripe هو اللي بيبيلغنا بالـ Webhook.

العنصر	التفاصيل
Acceptance Criteria	(1) Order بيتعمل PendingPayment قبل التحويل. (2) بعد الدفع الناجح والـ Webhook يتحول لـ Paid. (3) الـ Redirect لوحده لا يغير الحالة. (4) لو الـ Webhook اكرر مفیش تغيير تاني.
Main Flow	Payment ← Redirect ← Create Stripe Session ← Create Order + Reserve Stock ← Checkout ← Webhook ← Order Paid + CommissionRecords.
Alternative Flows	العميل يقفل صفحة Stripe: الـ Session بتننتهي وبيجي checkout.session.expired ← الـ Order بيتلغى والمخزون يتحرر.
Error Scenarios	كارت مرفوض (العميل يحاول تاني داخل نفس الـ Session)، Webhook توقيعه غلط (404)، Stripe مش متاح وقت إنشاء الـ Session (Rollback أو إلغاء الـ Order).
Entities	Order, OrderItem, Payment, StripeWebhookEvent, CommissionRecord
Components	CheckoutController, ICheckoutService, IPaymentService, StripeWebhookController, IStripeWebhookService

US-14: Vendor creates a product

As a Vendor, I want to create and manage my own products **so that** I can sell them in the marketplace.

العنصر	التفاصيل
Acceptance Criteria	(1) Approved Vendor فقط، (2) $Price > 0$ و (3) Category و (4) $StockQuantity \geq 0$ موجودة ونشطة. (5) $VendorProfileId$ يتحدد من المستخدم الحالي مش من الفورم. (6) الصور: امتدادات مسموحة وحجم محدود.
Main Flow	Vendor يفتح Create ← Validation ← POST ← $ProductViewModel$ ← ال Service يحفظ المنتج والصور ← Redirect لـ My Products.
Alternative Flows	حفظ كمسودة ($IsActive = false$).
Error Scenarios	Vendor غير معتمد (403)، سعر سالب، ملف صورة خطر، محاولة Overposting على $VendorProfileId$.
Entities	Product, ProductImage, Category, VendorProfile
Components	Areas/Vendor/ProductsController, IProductService, ICurrentVendorService, ProductViewModel

US-04: Super Admin approves vendor

As a Super Admin, I want to review vendor applications **so that** only legitimate stores can sell on the platform.

العنصر	التفاصيل
Acceptance Criteria	(1) قائمة Pending. عند الموافقة: $Status = Approved$ + إضافة $Role Vendor$ + تحديث $SecurityStamp$. (2) عند الرفض: $RejectionReason$ إجباري. (3) تسجيل $ReviewedByUserId$ و $ReviewedAt$.
Main Flow	Admin يفتح Pending ← تراجع ← $Role + Status$ ← $VendorService.ApproveAsync$ ← Approve.
Error Scenarios	تطبيق اترجع قبل كده، المستخدم مش Admin، Vendor حذف حسابه.
Entities	VendorProfile, ApplicationUser (+ Identity Roles)
Components	Areas/Admin/VendorApplicationsController, IVendorService, UserManager

قائمة باقي الـ User Stories

ID	As a	I want to	So that
US-01	Guest	browse and search products	.I can find what I want before registering
US-02	Guest	register and log in	.I can buy products
US-03	Guest	apply as a vendor	.I can open my own store
US-05	Customer	view product details with images	.I can decide to buy
US-07	Customer	update quantities and remove cart items	.my cart matches what I want
US-08	Customer	enter shipping details at checkout	.the order can be delivered
US-10	Customer	view my order history with statuses per vendor	.I can track my purchases
US-11	Customer	cancel an unpaid order	.stock is released if I change my mind
US-12	Customer	review a product I received	.other customers get honest feedback
US-13	Vendor	manage my store profile	.customers recognize my brand

So that	I want to	As a	ID
.the stock shown is accurate	adjust inventory	Vendor	US-15
.I can fulfill orders	see only my order items and mark them shipped/delivered	Vendor	US-16
.I understand my earnings	see my sales and commission summary	Vendor	US-17
.products are organized	manage categories	Super Admin	US-18
.the platform earns revenue	configure the commission rate	Super Admin	US-19
.I can supervise the marketplace	monitor all orders and see platform reports	Super Admin	US-20

Use Cases 2.14

الوصف المختصر	Actor	الاسم	UC
تصفح المنتجات النشطة فقط من متاجر .Approved	Guest	Browse / Search / View Details	UC-01..03
إنشاء حساب .Customer	Guest	Register Account	UC-04
.Authentication	Guest, Customer, Vendor, Admin	Login / Logout	UC-05
حساب + VendorProfile بحالة .Pending	Guest	Apply as Vendor	UC-06
إضافة/حذف/تعديل.	Customer	Manage Cart	UC-07
التحقق + إنشاء Order + حجز المخزون.	Customer	Checkout	UC-08
دفع .Hosted Checkout	Customer, Stripe	Pay with Stripe	UC-09
طلباتي فقط.	Customer	View Order History	UC-10
يحرر المخزون.	Customer	Cancel Pending Order	UC-11
بعد التسليم.	Customer	Write Review	UC-12
تعديل بيانات الحساب.	Customer	Manage Profile	UC-13
Store, Products, Images, Inventory, Order Items, Fulfillment, Sales, Commission, Dashboard	Vendor	Vendor operations	UC-14..22
Applications, Suspend, Categories, Commission, Orders, Reports, Moderation	Super Admin	Admin operations	UC-23..29
.Signature + Idempotency + Update Order/Payment	Stripe	Process Stripe Webhook	UC-31

Definition of Done 2.15 (على مستوى الـ Feature)

الـ Feature مش بتتخسب خلصانة إلا لو:

1. كل Acceptance Criteria اتحققت واتجريت يدوياً.
2. فيه Unit/Integration Tests ولا Service Logic ولا Authorization.
3. الـ Validation شغالة على الـ Server (وعلى الـ Client كتحسين فقط).
4. الـ Pull Request اتراجع من عضو غير الكاتب واتعمله Approve ولا CI أخضر.

5. مفيش Secrets ولا Warnings جديدة.
6. اتحدّثت ال Documentation (README / ال Diagrams لو التصميم اتغير).
7. اتعمل Demo في ال Sprint Review وكل الفريق فاهم ال Flow.

3. الأدوار والصلاحيات (User Roles and Authorization)

3.1 الفرق بين Authentication و Authorization

- Authentication** معناها: *مين انت؟* بتأكد من هوية المستخدم (Email + Password). بيتم عن طريق **ASP.NET Core Identity** وبيطلع Cookie بعد تسجيل الدخول.
- Authorization** معناها: *انت مسموحتك تعمل إيه؟* بنستخدم (Customer, Vendor, SuperAdmin) و **Policy-Based Authorization** (زي **ApprovedVendor**).
منستخدم الاثنين مع بعض: ال Role بتحدد النوع العام، وال Policy بتضيف شروط دقيقة زي "ال Vendor لازم يكون Approved".

3.2 تحليل الـ Actors

Use Cases	Accessible Pages	Restricted	Permissions / Actions	Authentication	الـ Actor
UC-01..06	Products, /, / /Products/Details/{i */d}, /Account	, Cart, Checkout, Orders Area أي	تصفح، بحث، تفاصيل Register. منتج، Login. Vendor Application	مش مطلوب	Guest
UC-07..13	Cart, /Checkout, / /Orders, /Account/Manage	Vendors. Admin بيانات Area. Orders غيره	كل اللي بيعمله + Cart, Checkout, Payment, Orders Reviews، بتاعته.	<code>Authorize(Roles="Customer")</code>	Customer
UC-14..22	*/Vendor/	منتجات//Orders Commissions Vendor تاني، Admin Area، تعديل نسبة العمولة	Store Profile. Products بتاعته، Inventory. OrderItems بتاعته، Sales/Commission n بتاعته	Role Vendor + Policy ApprovedVendor	Vendor
UC-23..29	*/Admin/	لا يشتري ولا يبيع (حساب إداري)	Approvals. Suspend. Categories. Commission Rate كل ال Orders. Reports. Moderation	Role SuperAdmin	Super Admin

الـ Super Admin بيتعمل من الـ DbSeeder عند أول تشغيل، وبياناته (Email/Password) بتيجي من `user-secrets` أو Environment Variables، مش مكتوبة في الـ Code.

3.3 تطبيق الـ Roles بـ ASP.NET Core Identity

الخطوة 1: Roles و ApplicationUser

```
public class ApplicationUser : IdentityUser
{
    public string FullName { get; set; } = string.Empty;
    public bool IsActive { get; set; } = true;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public VendorProfile? VendorProfile { get; set; }
}

public static class Roles
{
    public const string Customer = "Customer";
    public const string Vendor = "Vendor";
    public const string SuperAdmin = "SuperAdmin";
}
```

```
}
```

الخطوة 2: تسجيل Identity في Program.cs

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services
    .AddIdentity<ApplicationUser, IdentityRole>(options =>
    {
        options.Password.RequiredLength = 8;
        options.User.RequireUniqueEmail = true;
        options.SignIn.RequireConfirmedAccount = false; // MVP: no email provider yet
        options.Lockout.MaxFailedAccessAttempts = 5;
    })
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

builder.Services.AddAuthorizationBuilder()
    .AddPolicy("ApprovedVendor", p => p.AddRequirements(new ApprovedVendorRequirement()))
    .AddPolicy("SuperAdminOnly", p => p.RequireRole(Roles.SuperAdmin));

builder.Services.AddScoped<IAuthorizationHandler, ApprovedVendorHandler>();
```

[Team Decision]: اسم ال `AddIdentity` Methods (مقابل `AddIdentityCore`) والإعدادات الافتراضية بتتغير بين الإصدارات. راجعوا [Microsoft Learn: Introduction to Identity on ASP.NET Core](#) للنسخة اللي بتستخدموها.

الخطوة 3: Policy بتتأكد إن ال Vendor معتمد فعلاً

```
public class ApprovedVendorRequirement : IAuthorizationRequirement { }

public class ApprovedVendorHandler : AuthorizationHandler<ApprovedVendorRequirement>
{
    private readonly ApplicationDbContext _db;
    public ApprovedVendorHandler(ApplicationDbContext db) => _db = db;

    protected override async Task HandleRequirementAsync(
        AuthorizationHandlerContext context, ApprovedVendorRequirement requirement)
    {
        if (!context.User.IsInRole(Roles.Vendor)) return;
        var userId = context.User.FindFirstValue(ClaimTypes.NameIdentifier);
        var approved = await _db.VendorProfiles
            .AnyAsync(v => v.UserId == userId && v.Status == VendorStatus.Approved);
        if (approved) context.Succeed(requirement);
    }
}
```

ال Handler ببسأل ال Database في كل Request. ده أمان أعلى: لو ال Admin عمل **Suspend** لل Vendor، الصلاحية بتتقطع فوراً حتى لو ال Cookie لسه بتقول إنه Vendor.

الخطوة 4: تطبيق ال Policy على ال Area كلها

```
[Area("Vendor")]
[Authorize(Policy = "ApprovedVendor")]
public abstract class VendorBaseController : Controller
{
    protected readonly ICurrentVendorService CurrentVendor;
    protected VendorBaseController(ICurrentVendorService currentVendor) => CurrentVendor =
        currentVendor;
}
```

كل Controller في `Areas/Vendor` بيورث من `VendorBaseController`، فمستحيل ننسى ال `[Authorize]` في Controller جديد.

الخطوة 5: الـ Role بتتضاف عند الموافقة

عند موافقة الـ Admin على الـ Vendor Application، بنعمل `userManager.AddToRoleAsync(user, Roles.Vendor)` ثم `userManager.UpdateSecurityStampAsync(user)`. الـ `SecurityStamp` بيخلي الـ Cookie القديم يتعمل له `Revalidation`، فالـ Role الجديدة تظهر من غير ما المستخدم يعيد الدخول يدويًا [Assumption]: راجعوا إعدادات `[SecurityStampValidatorOptions.ValidationInterval]`.

3.4 عزل الـ Vendors (Vendor Isolation)

الـ Vendor A لازم ما يقدرش يشوف أو يعدل منتجات أو Inventory أو OrderItems أو Commissions بتاعة Vendor B. بنطبق ده بثلاث طبقات (Defense in Depth):

1. `Policy ApprovedVendor` على مستوى الـ `Area`.
2. `ICurrentVendorService` بيحدد `VendorProfileId` للمستخدم الحالي من الـ `Database/Claims`. مش من أي `Form` أو `Query String`.
3. كل `Query` في `Vendor Services` بتفلتر بـ `VendorProfileId`، وبترجع `NotFound` بدل `Forbidden` علشان منكشفش وجود بيانات الـ Vendor التاني.

```
public async Task<Product?> GetOwnedProductAsync(int productId, int vendorId) =>
    await _db.Products
        .Include(p => p.Images)
        .FirstOrDefaultAsync(p => p.Id == productId && p.VendorProfileId == vendorId);
```

وفي الـ Controller:

```
[HttpPost, ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, ProductEditViewModel model)
{
    if (!ModelState.IsValid) return View(model);
    var vendorId = await CurrentVendor.GetVendorProfileIdAsync();
    var ok = await _productService.UpdateAsync(id, vendorId, model); // checks ownership inside
    return ok ? RedirectToAction(nameof(Index)) : NotFound();
}
```

لاحظ إن `ProductEditViewModel` مفيهوش `VendorProfileId` خالص، ده بيمنع `Overposting`.

3.5 ليه إخفاء الزرار أو القائمة مش كفاية؟

لو اكتفينا بإننا نخفي زرار "Edit" بتاع منتج Vendor تاني في الـ View، ده بيحسن الـ UX بس مش أمان. أي شخص يقدر:

- يكتب الـ URL بإيده: `Vendor/Products/Edit/17/`.
 - يبعث POST Request بـ Postman أو `curl` أو من Developer Tools من غير ما يمر بالـ UI أصلاً.
 - يغير `id` المخفي في الـ Form (ده اسمه **IDOR: Insecure Direct Object Reference**).
- الـ Browser مكان مش موثوق. القاعدة: **Hide in UI for convenience, enforce on the Server for security**. وده اللي بنختبره في Test Cases الفصل 12 (Vendor A يحاول يعدل منتج Vendor B).

4. الرسوم التوضيحية (Visual Diagrams)

- كل Diagram في الفصل ده مرسوم فعلياً كصورة PNG/SVG، ومصدره القابل للتعديل موجود في فولدر `diagrams/src` (ملفات `mmd` لـ Mermaid و `dot` لـ Graphviz). لو غيّرتوا التصميم، عدّلوا الـ `Source` وأعيدوا الـ `Render` بالأمر `./diagrams/render.sh` علشان الرسومات والـ `Code` يفضلوا متطابقين.

قاعدة الفريق: أي Pull Request بيغيّر الـ ERD أو الـ Flow الأساسي لازم يحدّث الـ Diagram المرتبط بيه في نفس الـ PR.

Diagram 1: System Context Diagram

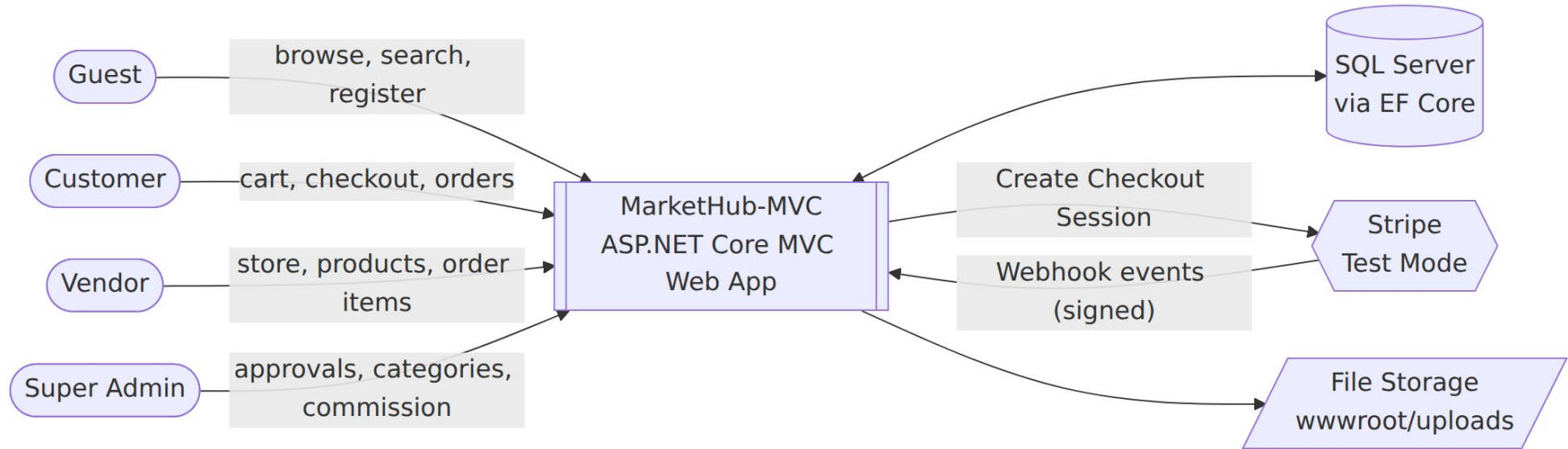
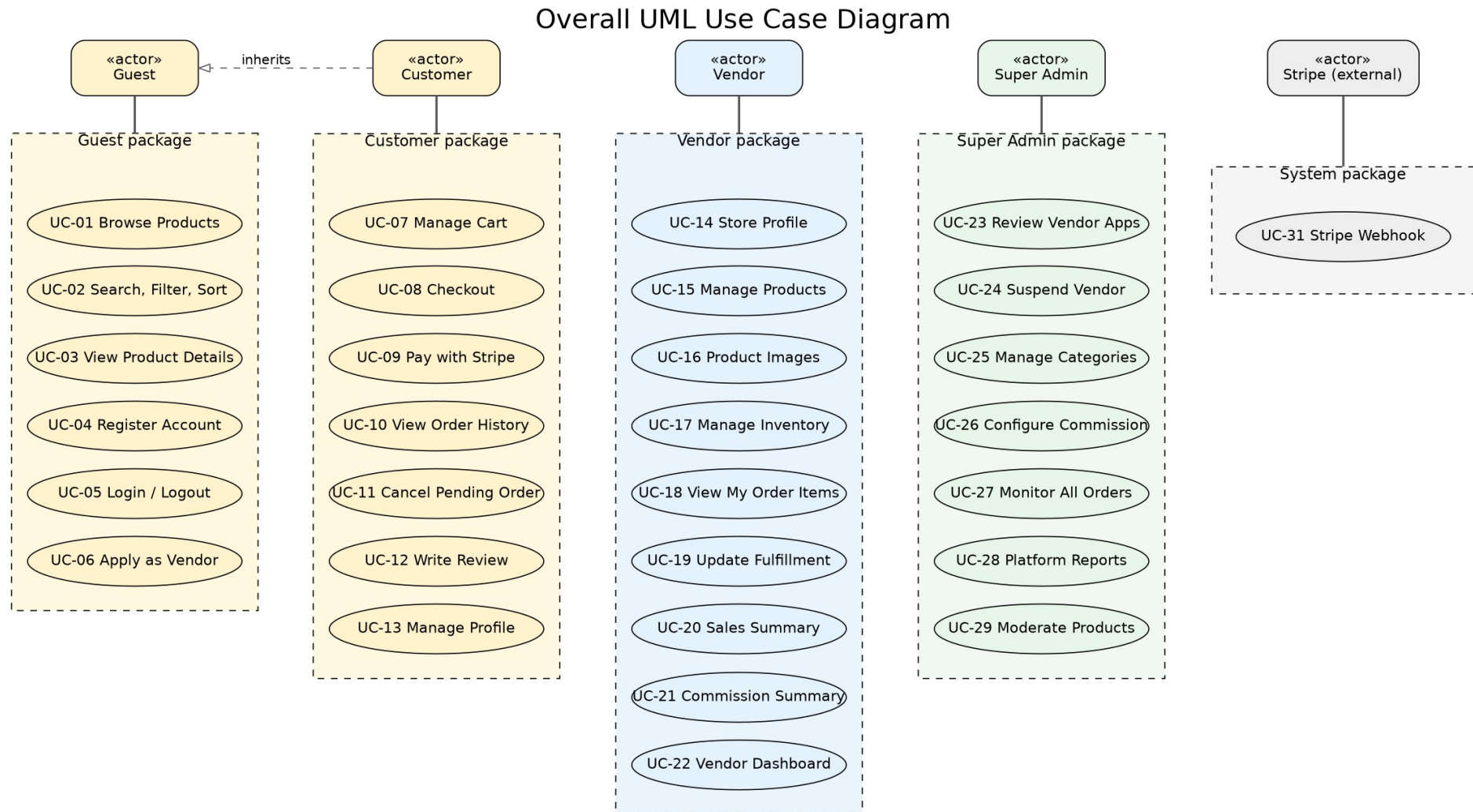


Diagram 1 | Editable source: diagrams/src/01_system_context.mmd (Mermaid) | Image: diagrams/png/01_system_context.png

الفكرة: ده أكبر صورة للنظام: مين بيستخدمه ومين بيتواصل معاه من بره. **MarketHub-MVC** في النص، حواليه الأربع Actors (Guest, Customer, Vendor, Super Admin)، وعلى الجانب الأنظمة الخارجية: **SQL Server** لتخزين البيانات، و **Stripe** لاستقبال المدفوعات، وال **File Storage** للصور.

ليه محتاجينه؟ علشان نحدد حدود المشروع (System Boundary). Stripe ببيعتلنا Webhook Events، يعني فيه سهم داخل من Stripe للتطبيق، ول لازم يكون عندنا Endpoint عام (Public HTTPS) يستقبله. ده بيأثر على ال Deployment.

Diagram 2: Overall UML Use Case Diagram



Each actor is linked to its package: it may perform every use case inside it (details in Diagrams 3, 4, 5)

Diagram 2 | Editable source: [diagrams/src/02_usecase_overall.dot](#) (Graphviz) | Image: [diagrams/png/02_usecase_overall.png](#)

الفكرة: يعرض كل ال Use Cases في صورة واحدة مقسمة ل Packages حسب ال Actor. ال **Customer** يرث كل اللي بيعمله ال **Guest** (سهم ال inherits). Stripe ك Actor خارجي مرتبط ب UC-31 (Process Stripe Webhook).
ازاي نقراه؟ كل Actor متصل بال Package بتاعه، يعني مسموح له ينفذ أي Use Case جواه. التفاصيل الدقيقة (include/extend) موجودة في الرسومات الثلاثة بعده.

Diagram 3: Customer Use Case Diagram

Customer Use Case Diagram

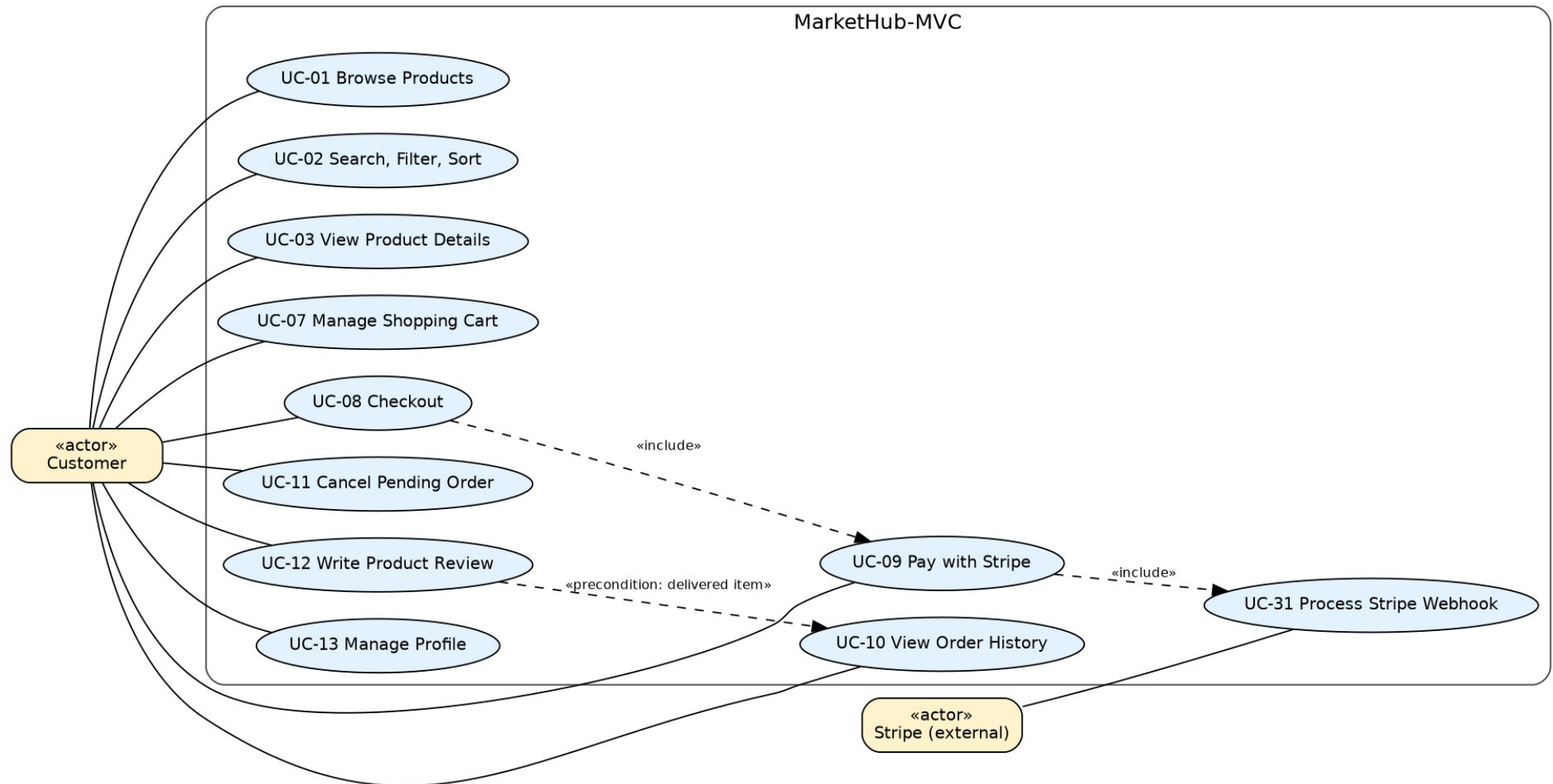


Diagram 3 | Editable source: diagrams/src/03_usecase_customer.dot (Graphviz) | Image: diagrams/png/03_usecase_customer.png

الشرح: ال Customer بيتصفح ويضيف للـ Cart ويعمل Checkout. العلاقة «include» بين UC-08 (Checkout) و UC-09 (Pay with Stripe) معناها إن الدفع جزء إجباري من ال Checkout. وبرضه UC-09 بيعتمد على UC-31 (ال Webhook) علشان التأكيد النهائي. UC-12 (Write Review) له شرط مسبق: لازم يكون فيه منتج اتسلّم.

Vendor Use Case Diagram (Approved Vendor)

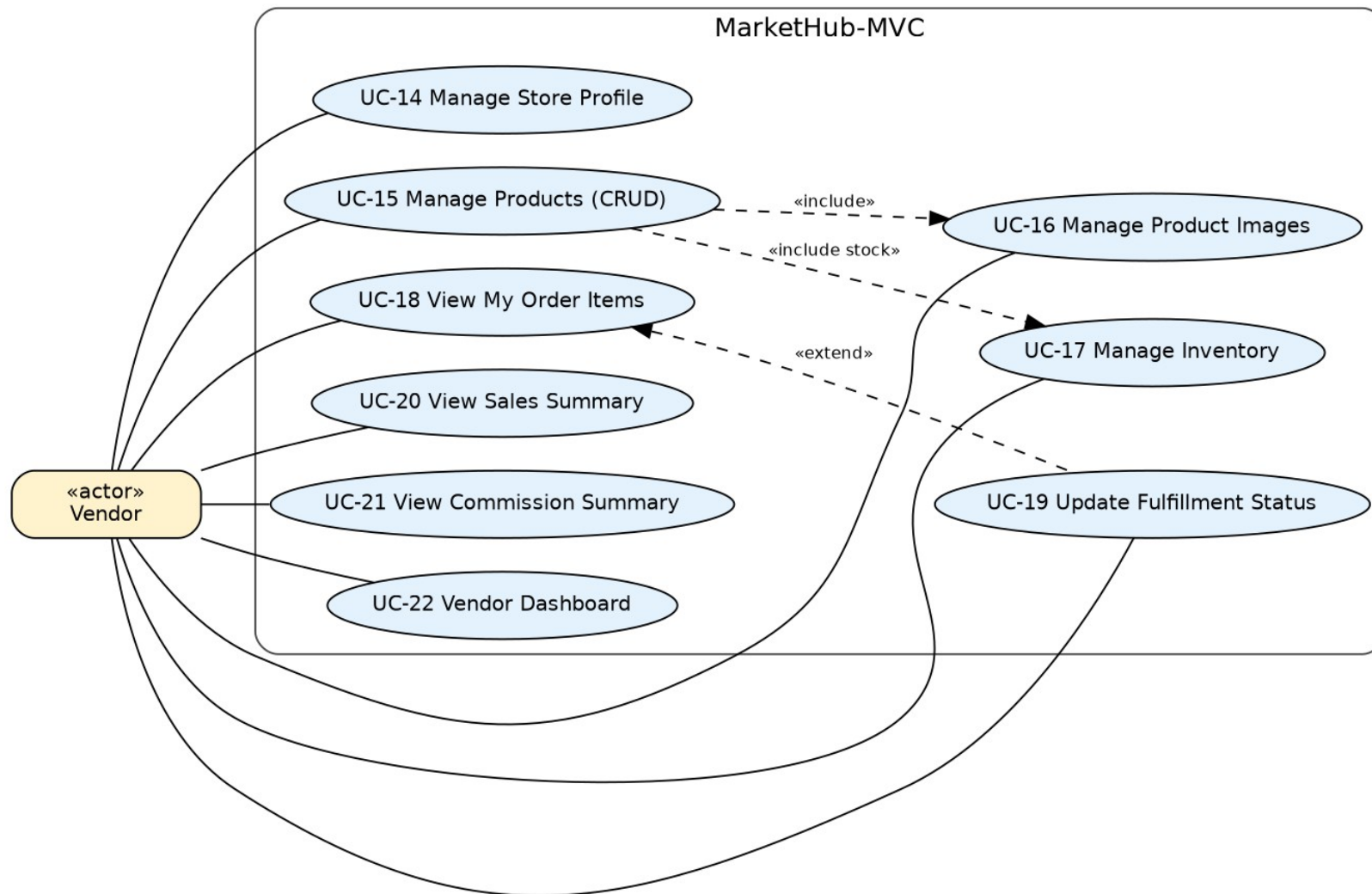


Diagram 4 | Editable source: diagrams/src/04_usecase_vendor.dot (Graphviz) | Image: diagrams/png/04_usecase_vendor.png

الشرح: كل ال Use Cases دي مشروطة بإن ال Vendor يكون UC-15 (Manage Products) **Approved**. بيشمل رفع الصور (UC-16) وإدارة المخزون (UC-17). (UC-19 (Update Fulfillment Status) بيتمد من UC-18 (View My Order Items) لأن ال Vendor مش هيعتدل حالة قبل ما يشوف ال Item.

Super Admin Use Case Diagram

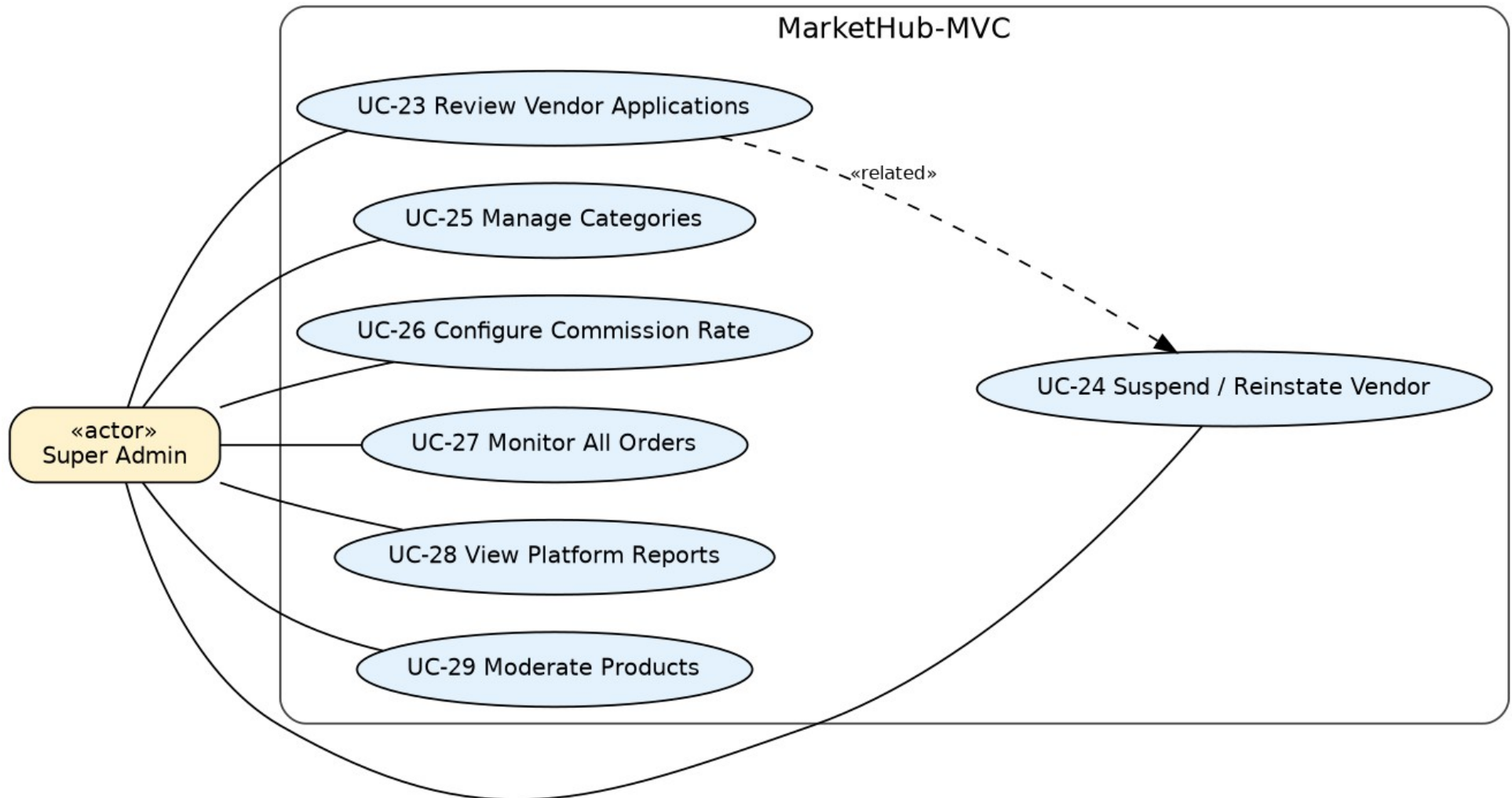


Diagram 5 | Editable source: diagrams/src/05_usecase_admin.dot (Graphviz) | Image: diagrams/png/05_usecase_admin.png

الشرح: ال Super Admin مسؤول عن الحوكمة: الموافقة على ال Vendors (UC-23) وإيقافهم (UC-24)، ال Categories (UC-25)، نسبة العمولة (UC-26)، ومراقبة الطلبات والتقارير (UC-27, UC-28). تعديل UC-26 بضيف صف جديد في **CommissionSettings** ومبيغيرش العمولات القديمة.

Diagram 6A: ERD Overview (Relationships Only)

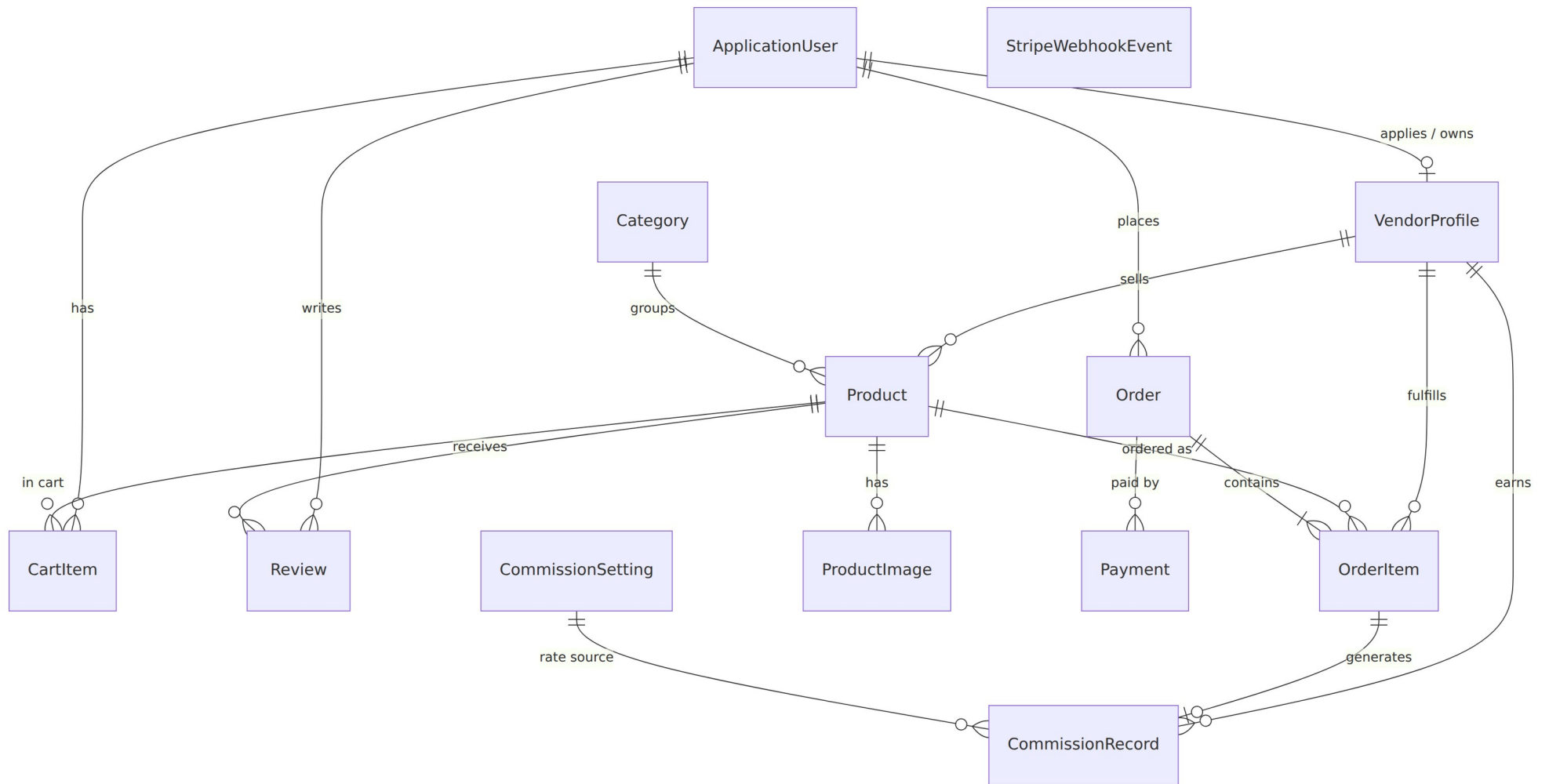


Diagram 6A | Editable source: diagrams/src/06a_erd_overview.mmd (Mermaid) | Image: diagrams/png/06a_erd_overview.png

الشرح: صورة مبسطة للعلاقات بين الجداول من غير الأعمدة. القراءة: `VendorProfile` `--o|` `ApplicationUser` معناها إن المستخدم الواحد عنده صفر أو `VendorProfile` واحد. `OrderItem` `--|{` `Order` معناها إن كل `Order` لازم يكون فيه `OrderItem` واحد على الأقل. `CommissionRecord` `--o|` `OrderItem` معناها إن كل `OrderItem` مدفوع له `CommissionRecord` واحد بالظبط. جدول `StripeWebhookEvent` مستقل علشان بيستخدم فقط لضمان الـ `Idempotency`.

Diagram 6B: ERD: Identity, Vendors and Catalog

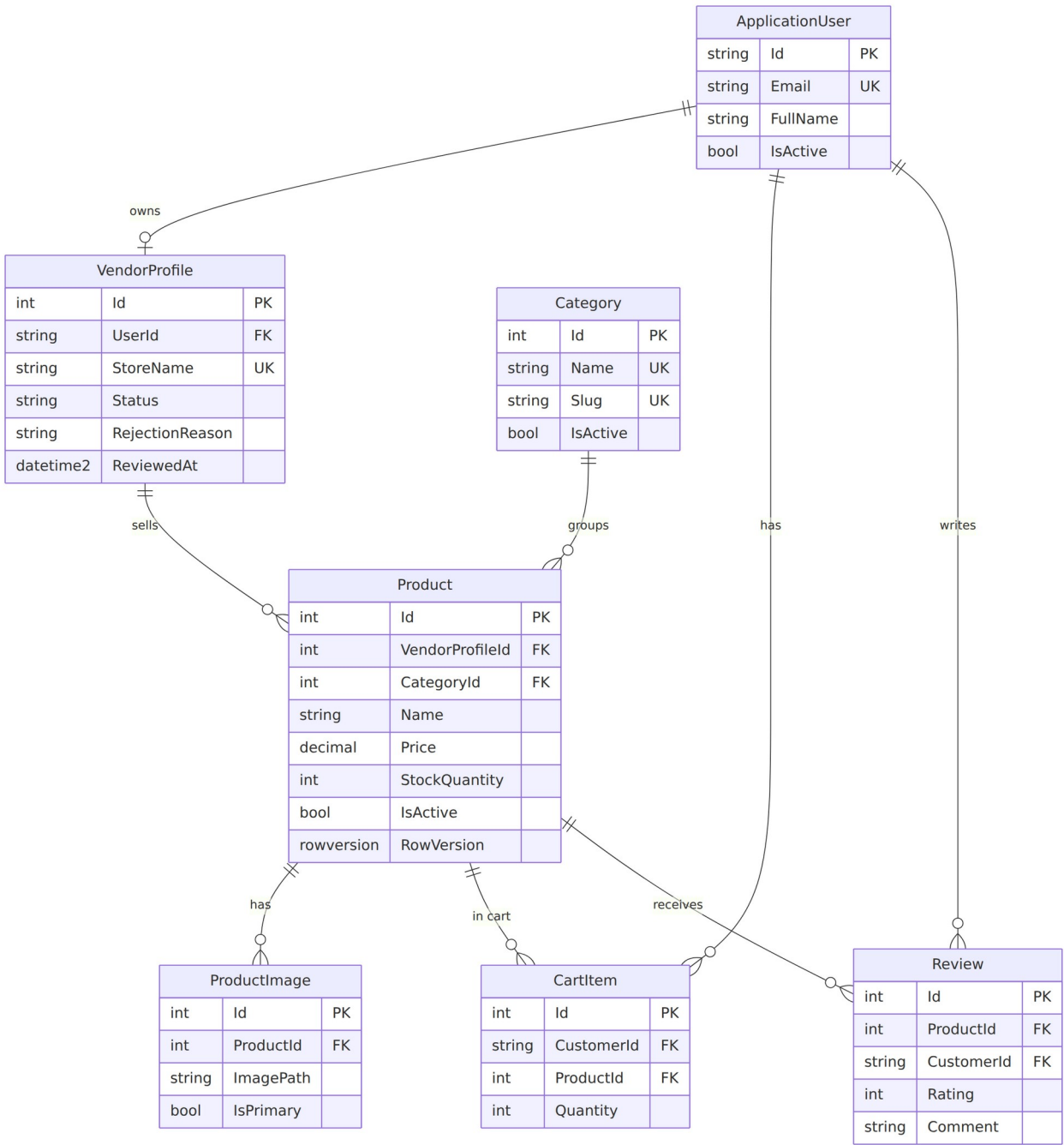


Diagram 6B | Editable source: diagrams/src/06b_erd_catalog.mmd (Mermaid) | Image: diagrams/png/06b_erd_catalog.png

الشرح: الجزء الخاص بالمستخدمين والمنتجات. `Product.VendorProfileId` هو اللي بيربط كل منتج بمتجره، ويعتمد عليه عزل البيانات. `CartItem` فيه Unique Index على `(CustomerId, ProductId)`. `Product.RowVersion` عمود `rowversion` يستخدم كـ Concurrency Token. تفاصيل الأنواع والقيود في الـ Data Dictionary (الفصل 5).

Diagram 6C: ERD: Orders, Payments and Commission

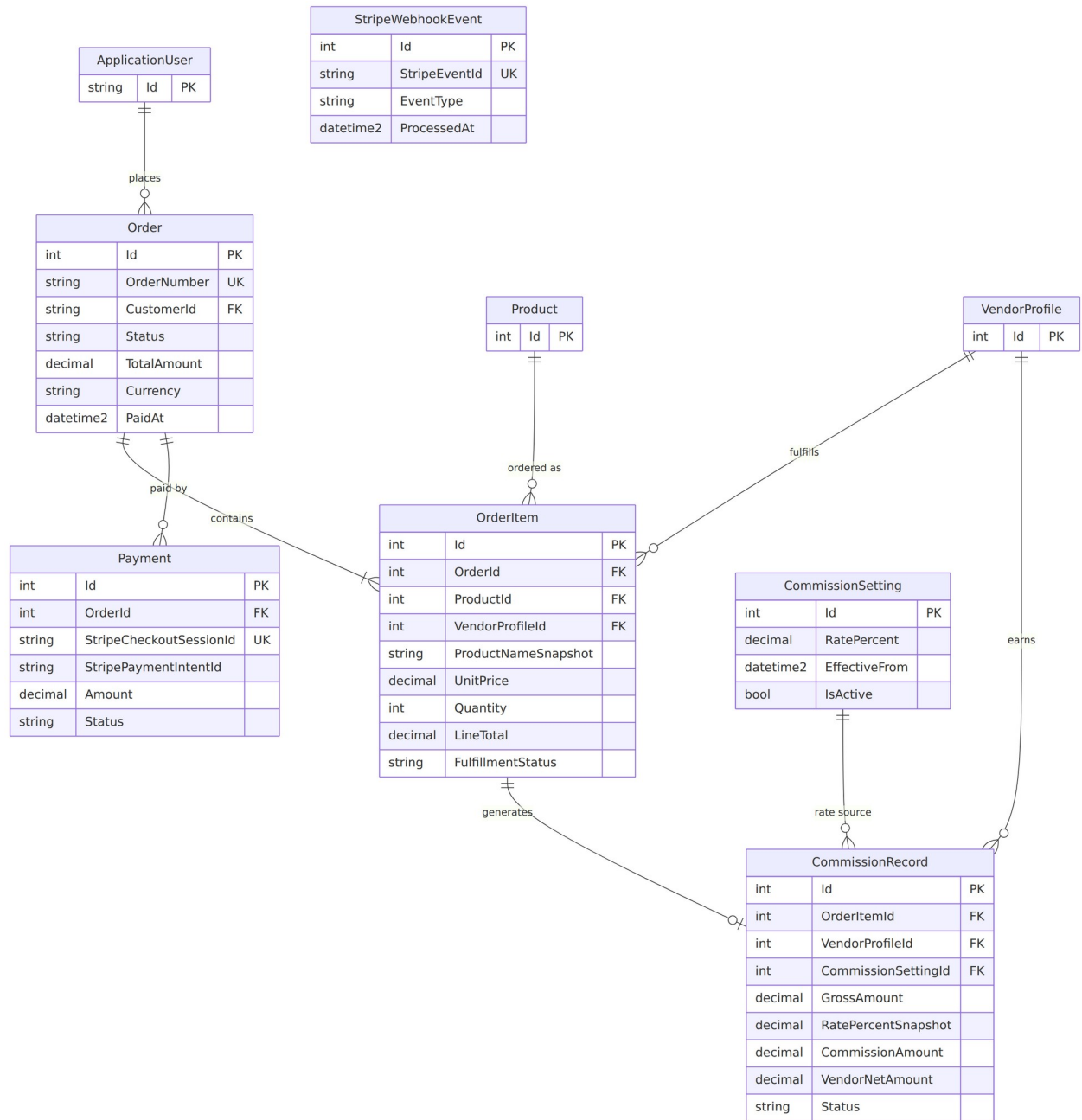


Diagram 6C | Editable source: [diagrams/src/06c_erd_orders_payments.mmd](#) (Mermaid) | Image: [diagrams/png/06c_erd_orders_payments.png](#)

الشرح: الجزء المالي. **OrderItem** يحتفظ بـ **VendorProfileId** و **UnitPrice** و **ProductNameSnapshot** وقت الشراء
علشان تغيير سعر المنتج بعدين ما يغيرش تاريخ الطلبات. **Payment** فيه **StripeCheckoutSessionId** بـ Unique Index.
CommissionRecord بيخزن **RatePercentSnapshot** علشان لو ال Admin غير النسبة، السجلات القديمة تفضل زي ما هي.

Diagram 7: High-Level System Architecture

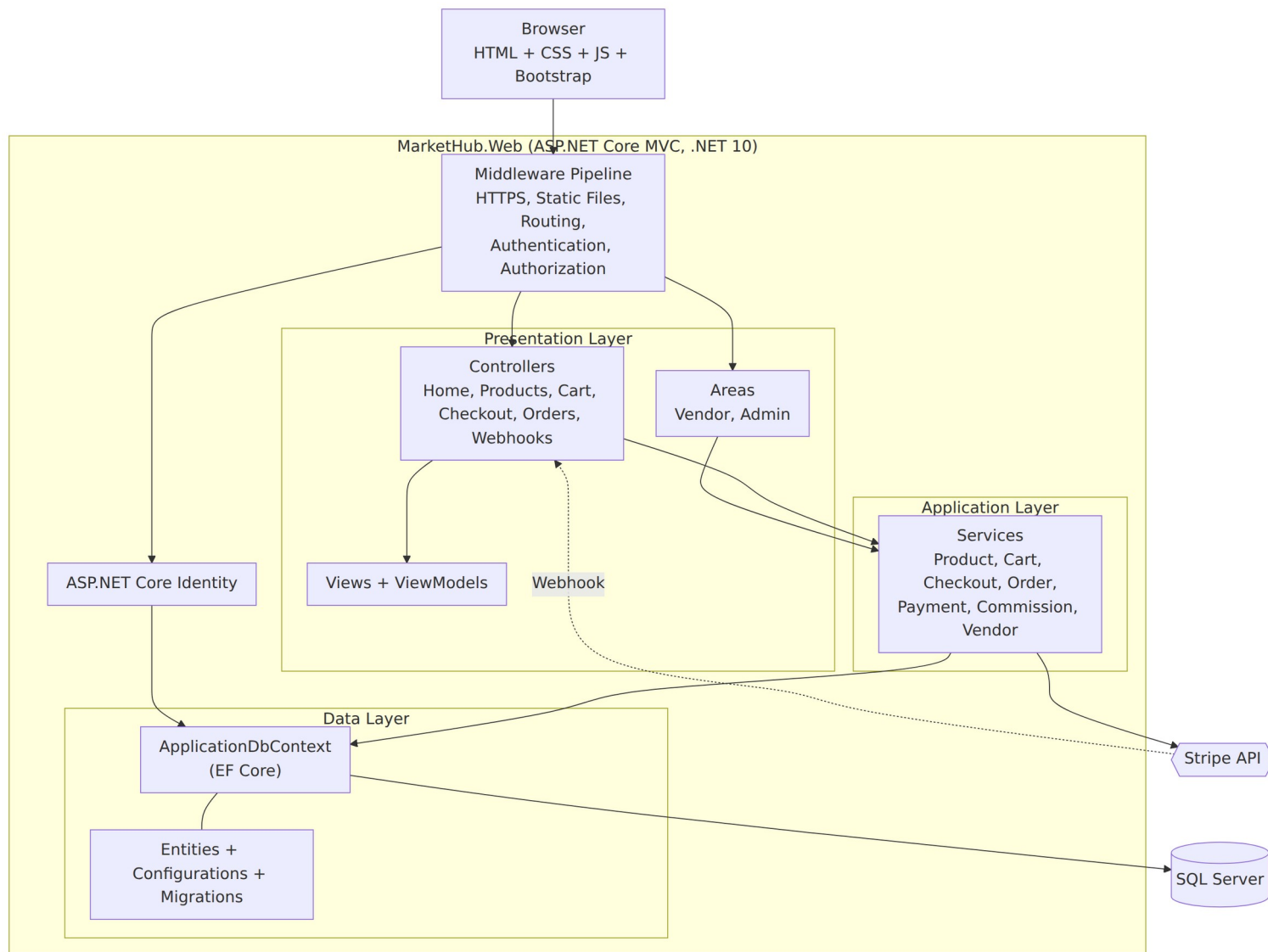


Diagram 7 | Editable source: diagrams/src/07_architecture.mmd (Mermaid) | Image: diagrams/png/07_architecture.png

الشرح: ال Request بيدخل من ال Browser على ال Middleware Pipeline (HTTPS, Static Files, Routing, Authentication, Authorization), بعدين ال Controller (أو Area Controller), اللى بينادي على ال Service. ال Service بتتعامل مع ال **ApplicationDbContext** اللى بيتكلم مع ال SQL Server, ومع Stripe عند الدفع. ال Webhook من Stripe بيدخل على ال **WebhookController** مباشرة. التقسيم ده مطابق تماماً لهيكل الفولدرات في الفصل 6.

Diagram 8: Application Component Diagram

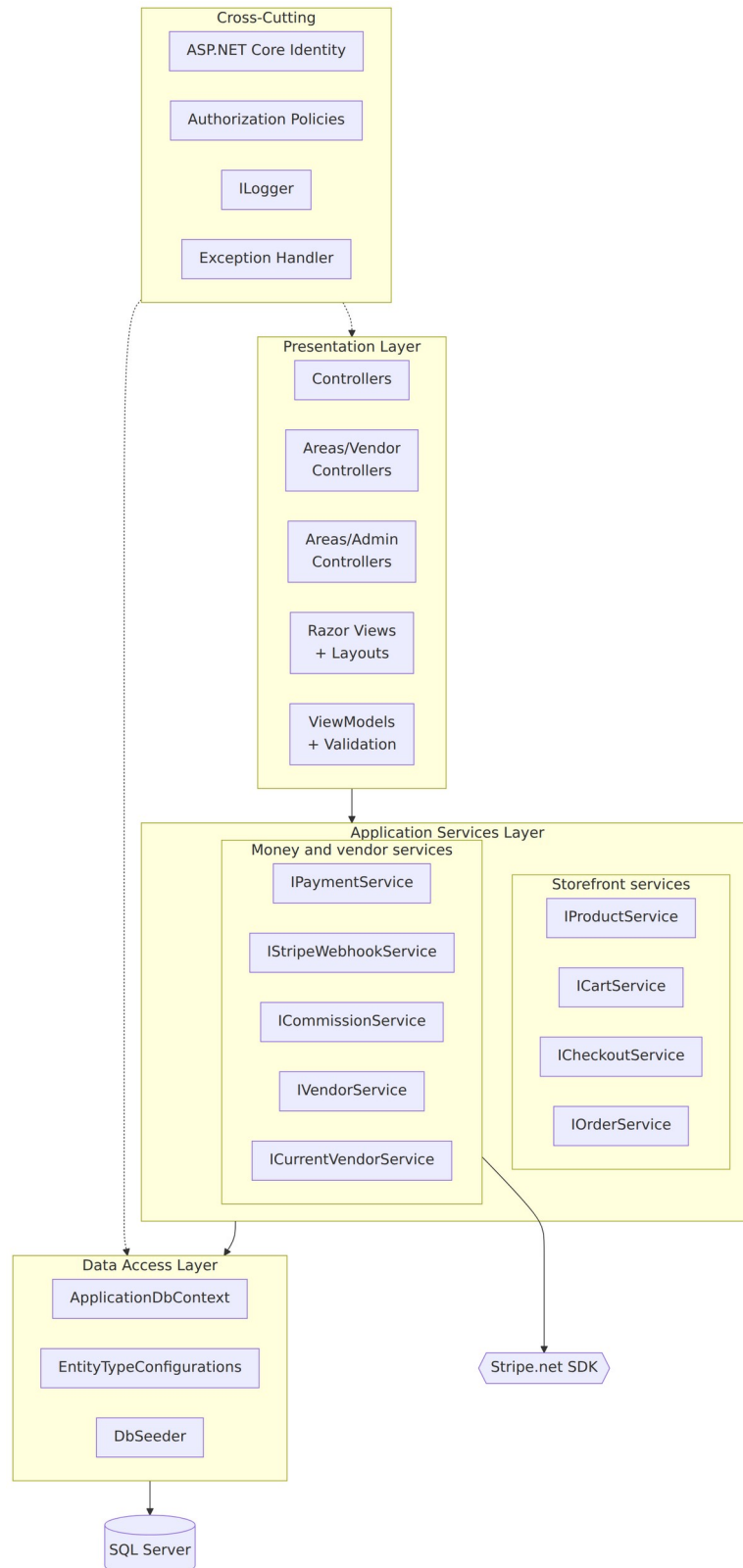


Diagram 8 | Editable source: diagrams/src/08_components.mmd (Mermaid) | Image: diagrams/png/08_components.png

الشرح: بيوضح مين بيعتمد على مين. ال Presentation بتعتمد على ال Services، وال Services بتعتمد على ال Data Access، ومفيش عكس ده (ال Service مبتعرفش حاجة عن ال Controller). مجموعة Money and vendor services هي اللي بتتعامل مع Stripe SDK. ال Cross-Cutting (Identity, Authorization, Logging, Exception Handler) بتخدم الطبقات كلها.

Diagram 9: UML Class Diagram

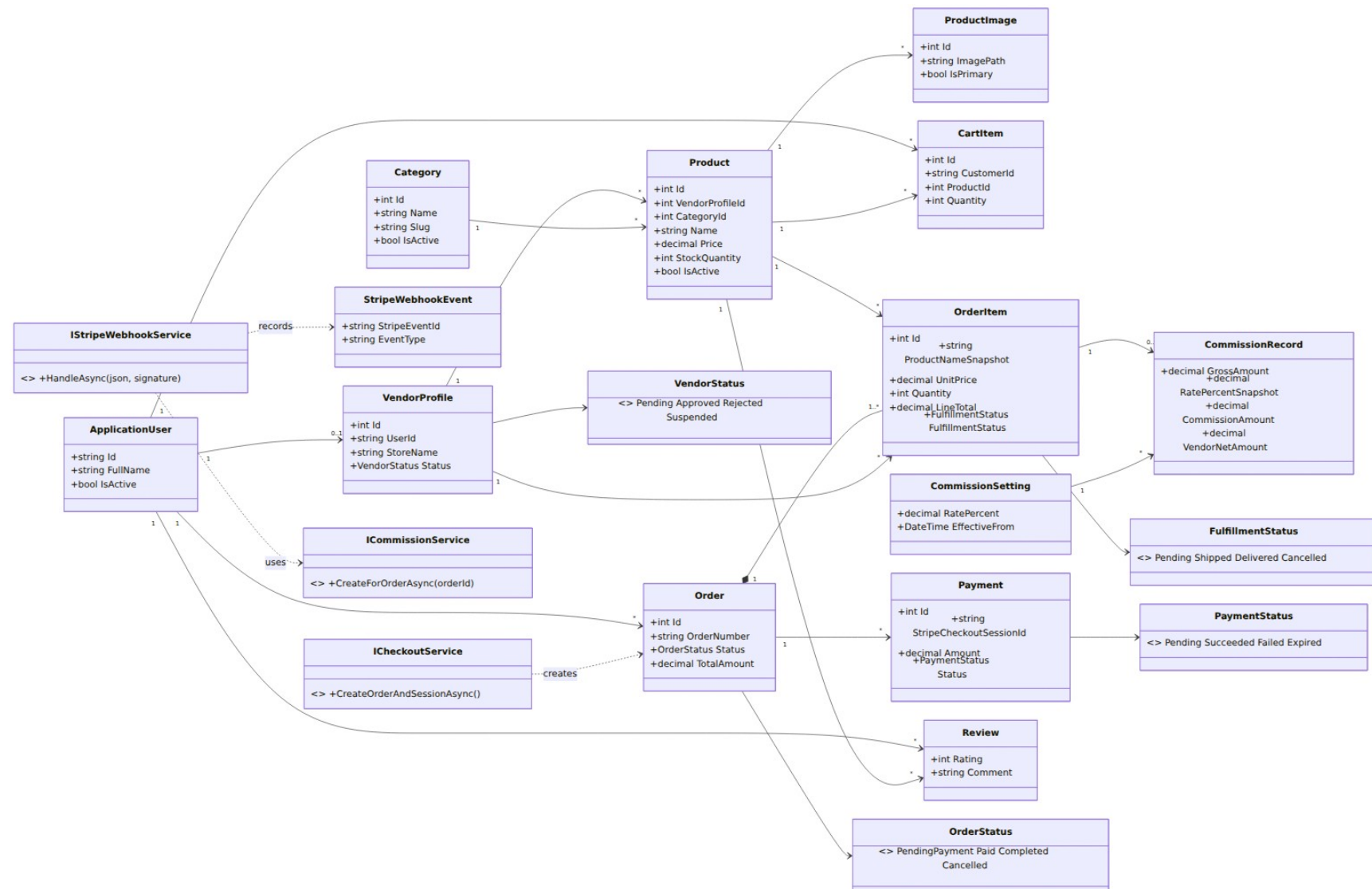


Diagram 9 | Editable source: [diagrams/src/09_class_diagram.mmd](#) (Mermaid) | Image: [diagrams/png/09_class_diagram.png](#)

الشرح: ال Entities وال Enums الأساسية مع علاقاتها: **Order** يحتوي (Composition) على **OrderItem** و **Product** مرتبط بـ **VendorProfile** و **Category**. ال Enums (**VendorStatus**, **OrderStatus**, **PaymentStatus**, **FulfillmentStatus**) بتخزن في ال Database كـ **string** باستخدام **<HasConversion>string** () علشان تكون مقروءة في ال SQL. ظهرت كمان Interfaces أساسية (**ICheckoutService**, **IStripeWebhookService**, **ICommissionService**) علشان تفهم مين بيخلق إيه.

Diagram 10: Deployment Architecture

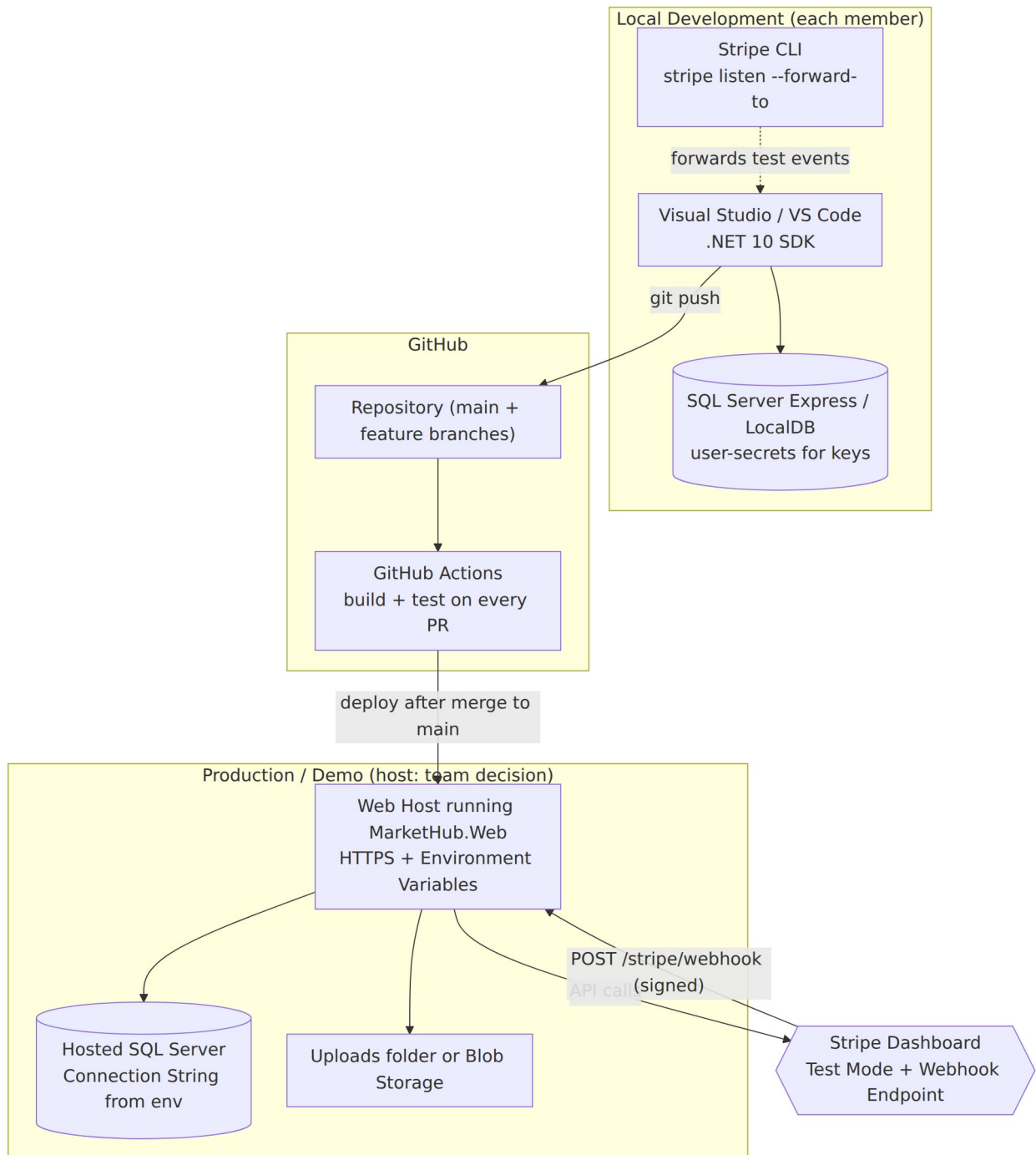


Diagram 10 | Editable source: diagrams/src/10_deployment.mmd (Mermaid) | Image: diagrams/png/10_deployment.png

الشرح: التمييز بين ثلاث بيئات: (1) Local Development: SQL Server محلي، `user-secrets`، و Stripe CLI يعمل Tests. (2) GitHub Actions Repository: Webhooks. (3) Production/Demo: Web Host + SQL Server مستضاف + Stripe Test Mode Webhook Endpoint. الاختيار مزود الاستضافة قرار الفريق [Team Decision]، والرسم مكتوب بشكل عام عمداً.

Diagram 11: Vendor Registration and Approval (Activity)

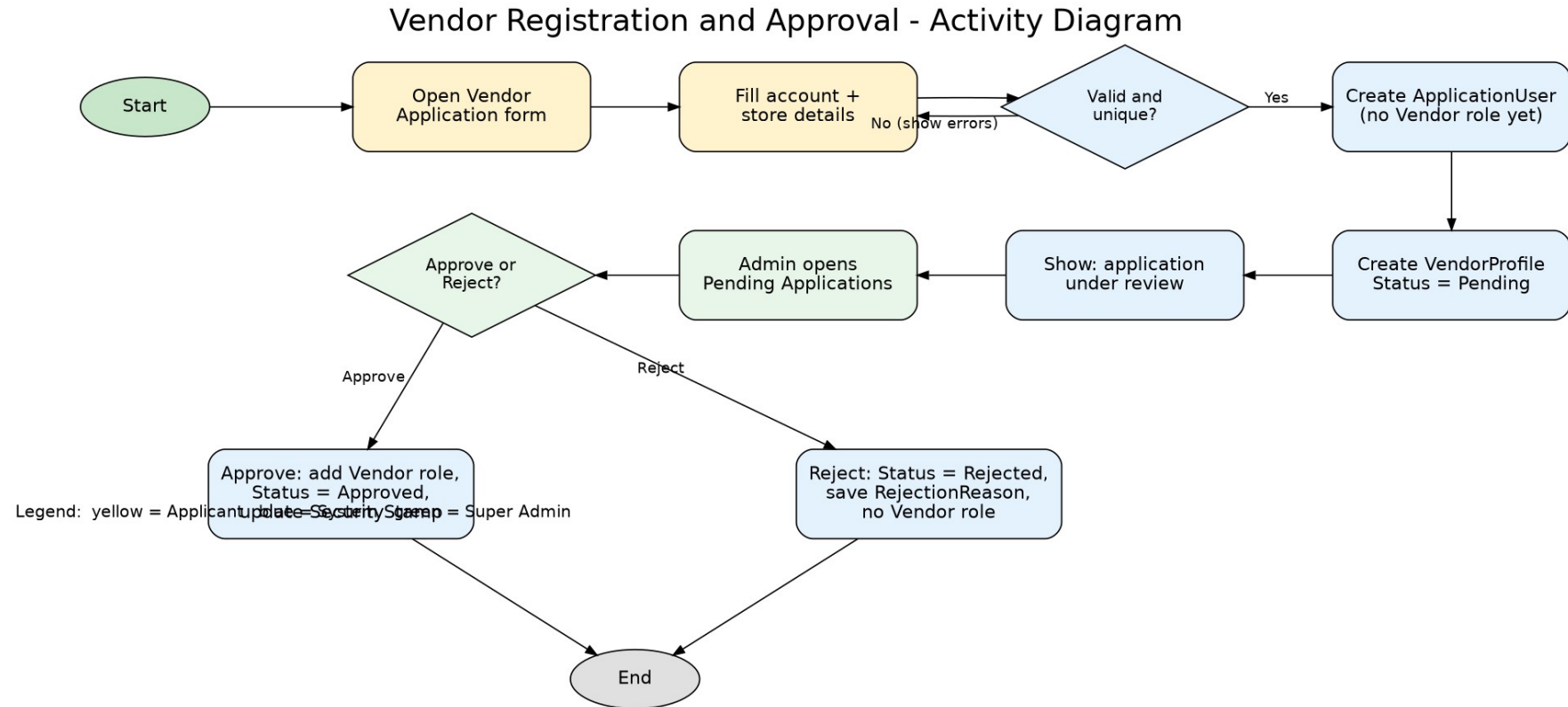


Diagram 11 | Editable source: diagrams/src/11_vendor_registration_activity.dot (Graphviz) | Image: diagrams/png/11_vendor_registration_activity.png

الشرح: الألوان بتبين مين المسؤول: أصفر للمتقدم، أزرق للـ System، أخضر للـ Admin. الـ System بيتحقق من صحة البيانات وتغرد الـ Email والـ StoreName، بعدين بينشئ ApplicationUser من غير Vendor Role و VendorProfile بحالة Pending. الـ Role بتتضاف فقط عند الموافقة، ومعها UpdateSecurityStampAsync. عند الرفض بنحفظ RejectionReason والمستخدم يفضل Customer عادي.

Diagram 12: Product Creation (Activity)

Product Creation - Activity Diagram

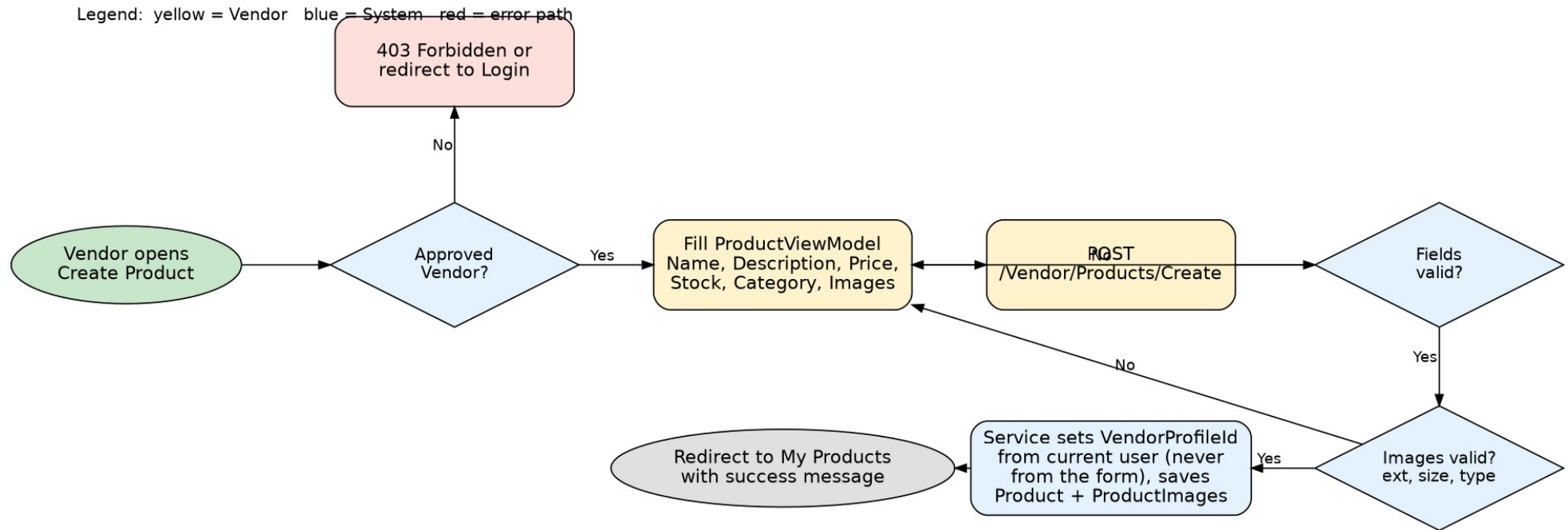


Diagram 12 | Editable source: diagrams/src/12_product_creation_activity.dot (Graphviz) | Image: diagrams/png/12_product_creation_activity.png

الشرح: أول بوابة: هل المستخدم Approved Vendor؟ لو لا، 403 أو Login. بعدها الفورم، ثم Validation على الـ Server (السعر أكبر من صفر، المخزون صفر أو أكثر، الـ Category موجودة). الصور بتتفحص (الامتداد، الحجم، نوع المحتوى). النقطة الأهم: الـ Service هي اللي بتحدد **VendorProfileId** من المستخدم الحالي، والـ Form مفيهوش الحقل ده.

Diagram 13: Shopping Cart (Activity)

Shopping Cart - Activity Diagram

Legend: yellow = Customer blue = System red = error path

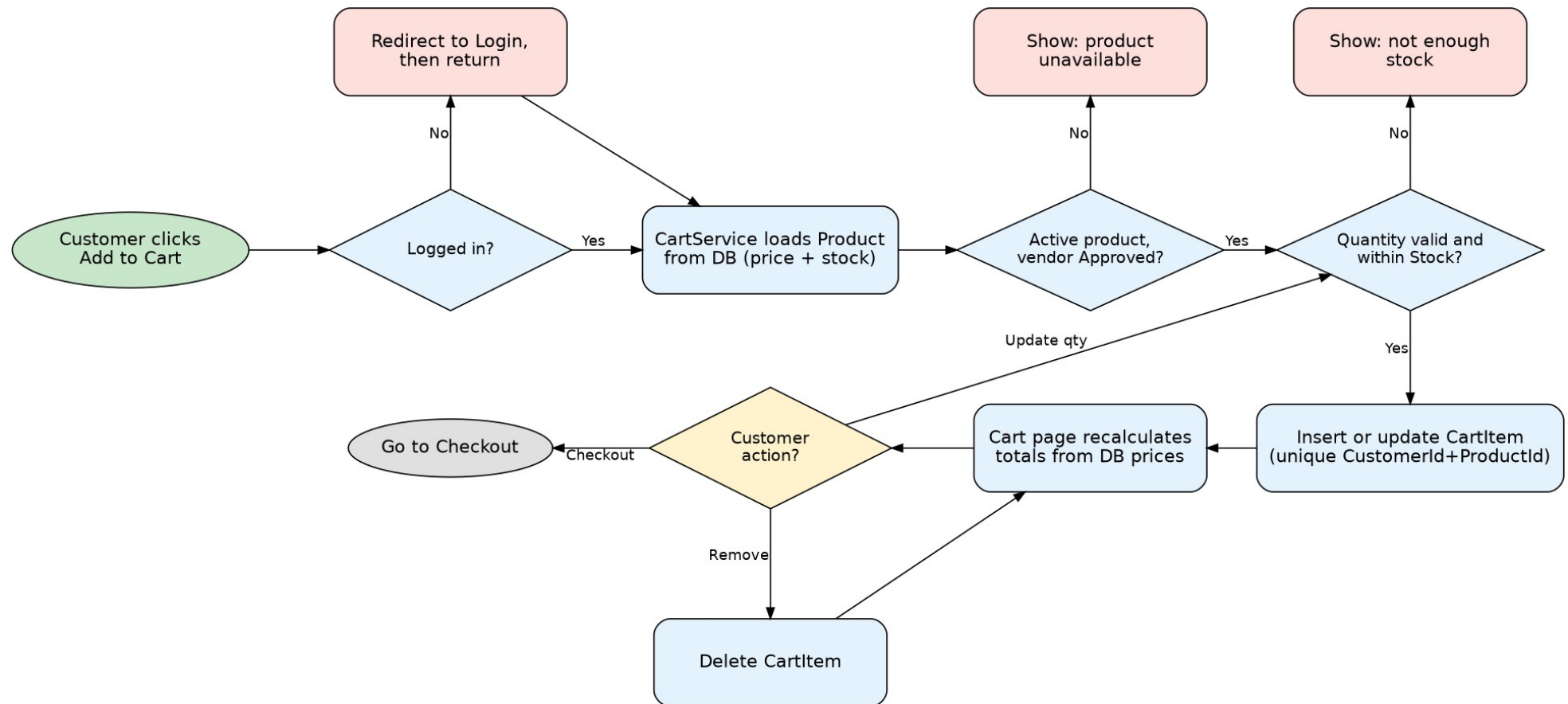


Diagram 13 | Editable source: diagrams/src/13_cart_activity.dot (Graphviz) | Image: diagrams/png/13_cart_activity.png

الشرح: ال Guest بيتحوّل لـ Login. بعدين ال **CartService** بيحمّل المنتج من ال Database (السعر والمخزون الحاليين) ويتأكد إنه Active وإن ال Vendor Approved وإن الكمية صحيحة. بنعمل Insert أو Update على **CartItem** (Unique) على العميل والمنتج). صفحة ال Cart بتعيد حساب المجاميع من أسعار ال Database في كل مرة، علشان لو السعر اتغير العميل يشوف الجديد.

Checkout - Activity Diagram

Legend: yellow = Customer blue = System red = error path

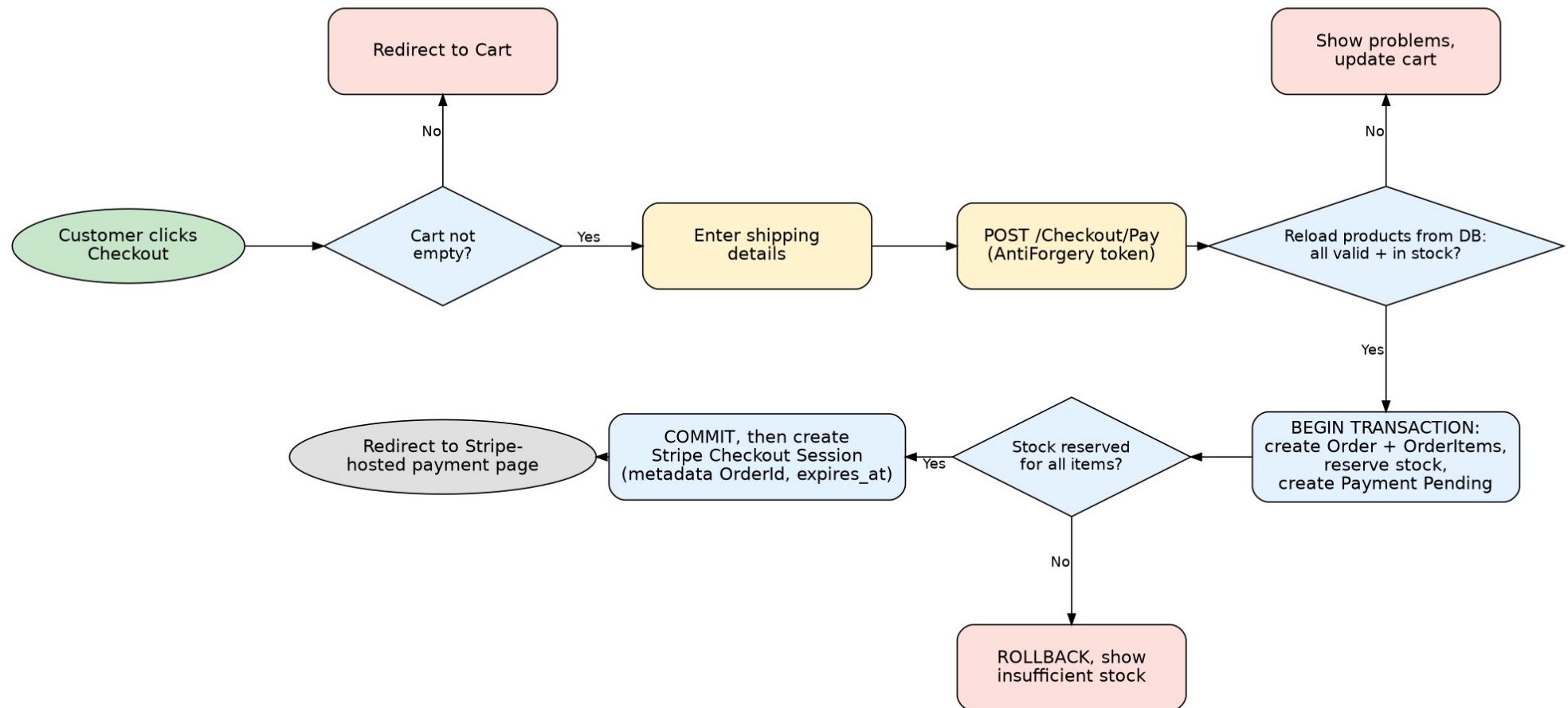


Diagram 14 | Editable source: diagrams/src/14_checkout_activity.dot (Graphviz) | Image: diagrams/png/14_checkout_activity.png

الشرح: عند الضغط على Pay، بنعيد تحميل كل منتج من الـ Database ونتأكد تاني (Active, Approved, Stock). لو كله تمام نبدأ Transaction: نعمل **Order** بحالة **PendingPayment** و **OrderItems** بـ Price Snapshot، ونحجز المخزون بـ Conditional UPDATE، وننشئ **Payment** بحالة **Pending**. لو الحجز فشل لأي منتج نعمل Rollback. بعد الـ COMMIT بنكلم Stripe لإنشاء الـ Checkout Session ونحول العميل لصفحة الدفع.

Diagram 15: Payment Processing (Sequence)

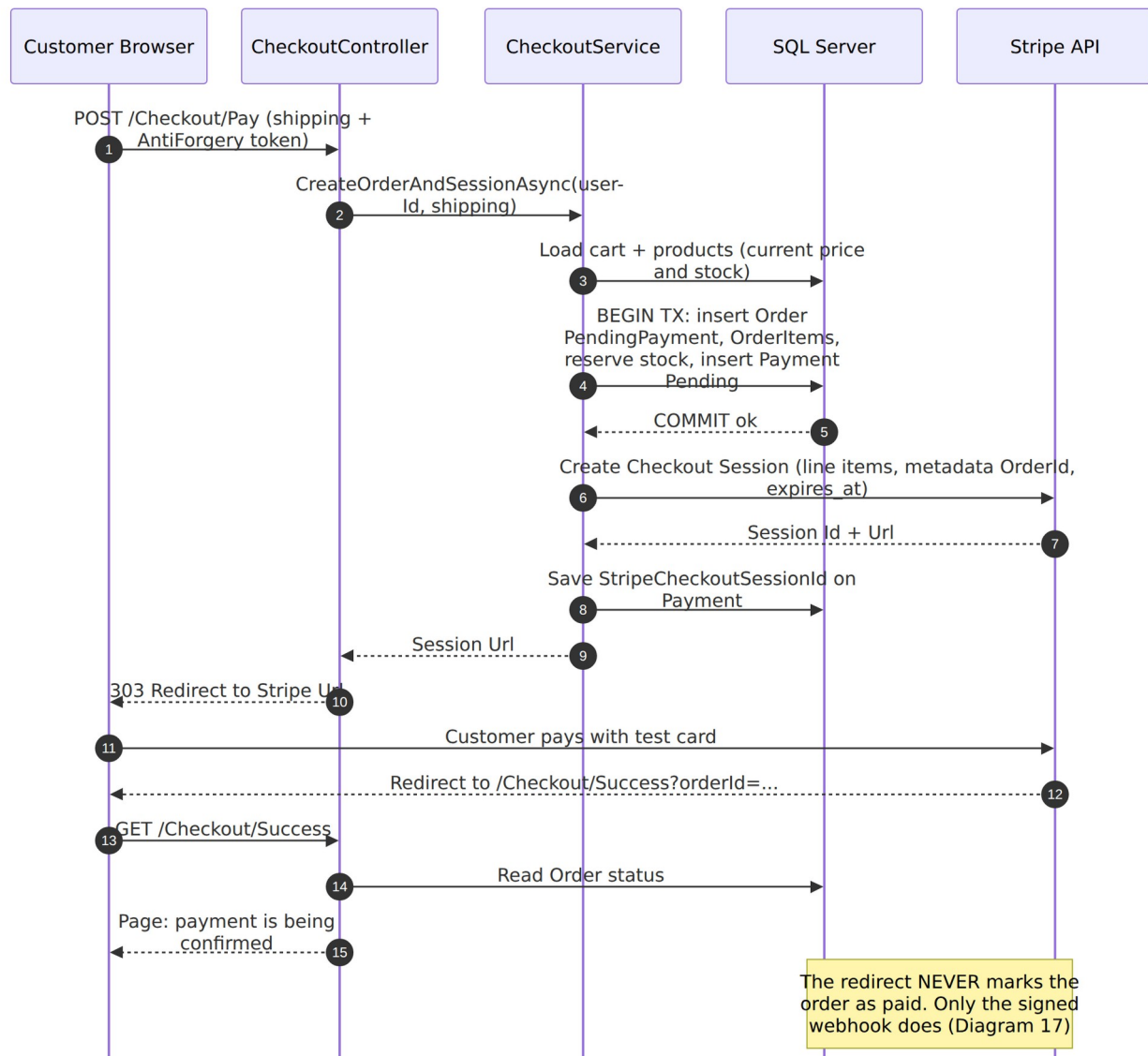


Diagram 15 | Editable source: diagrams/src/15_payment_sequence.mmd (Mermaid) | Image: diagrams/png/15_payment_sequence.png

الشرح خطوة بخطوة: (1) العميل يضغط (2-4). Service ال Pay (POST + AntiForgery). العميل ال Cart وتعمل ال Transaction. (6-7) إنشاء Checkout Session عند Stripe مع metadata فيها OrderId و(8). expires_at حفظ StripeCheckoutSessionId. العميل بيدفع بـ كارت تجريبي ويرجع لـ /Checkout/Success. (10-12) صفحة النجاح بتعرض الحالة فقط وبتقول "الدفع قيد التأكيد". ال Redirect عمره ما بيغير ال Order لـ Paid: لأن العميل ممكن يقفل المتصفح أو يعطل ال URL. التأكيد الحقيقي من ال Webhook (Diagram 17).

Diagram 16: Order Processing and Fulfillment (Sequence)

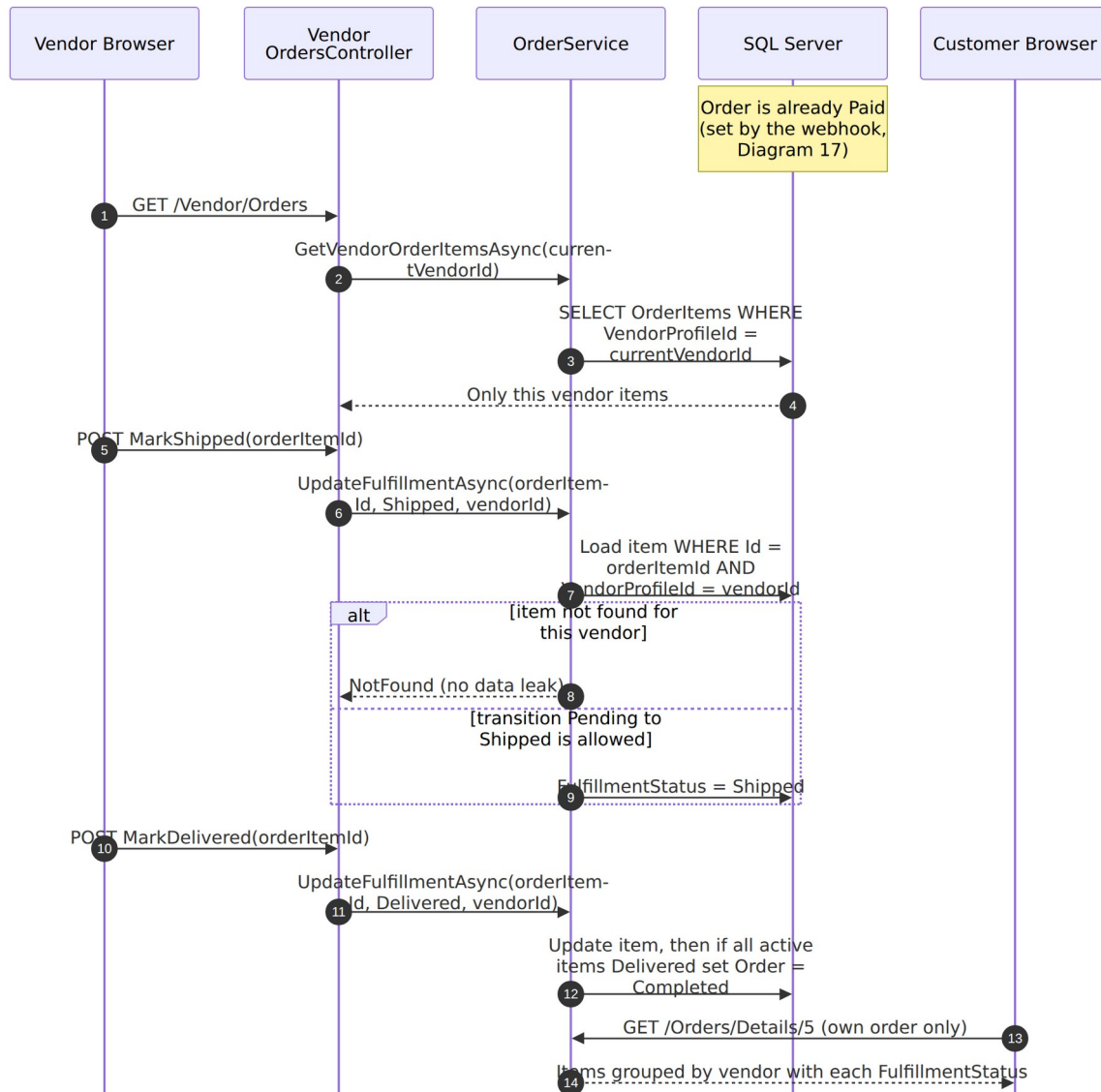


Diagram 16 | Editable source: [diagrams/src/16_order_processing_sequence.mmd \(Mermaid\)](#) | Image: [diagrams/png/16_order_processing_sequence.png](#)

الشرح: بعد ما ال Order يبقى **Paid**، ال Vendor بيفتح **Vendor/Orders/** وال Service بتجيب ال OrderItems اللي **VendorProfileId** بتاعها = ال Vendor الحالي بس. عند Mark Shipped بنحمل ال Item بشرط الملكية، ولو مش موجود بترجع NotFound (من غير ما نكشف وجوده). التحويلات المسموحة: **Pending → Shipped → Delivered**. لما كل ال Items غير الملغية تبقى **Delivered** ال Order بيتحول ل **Completed**. العميل بيشوف طلبه مقسم حسب ال Vendor.

Diagram 17: Stripe Webhook Processing (Sequence)

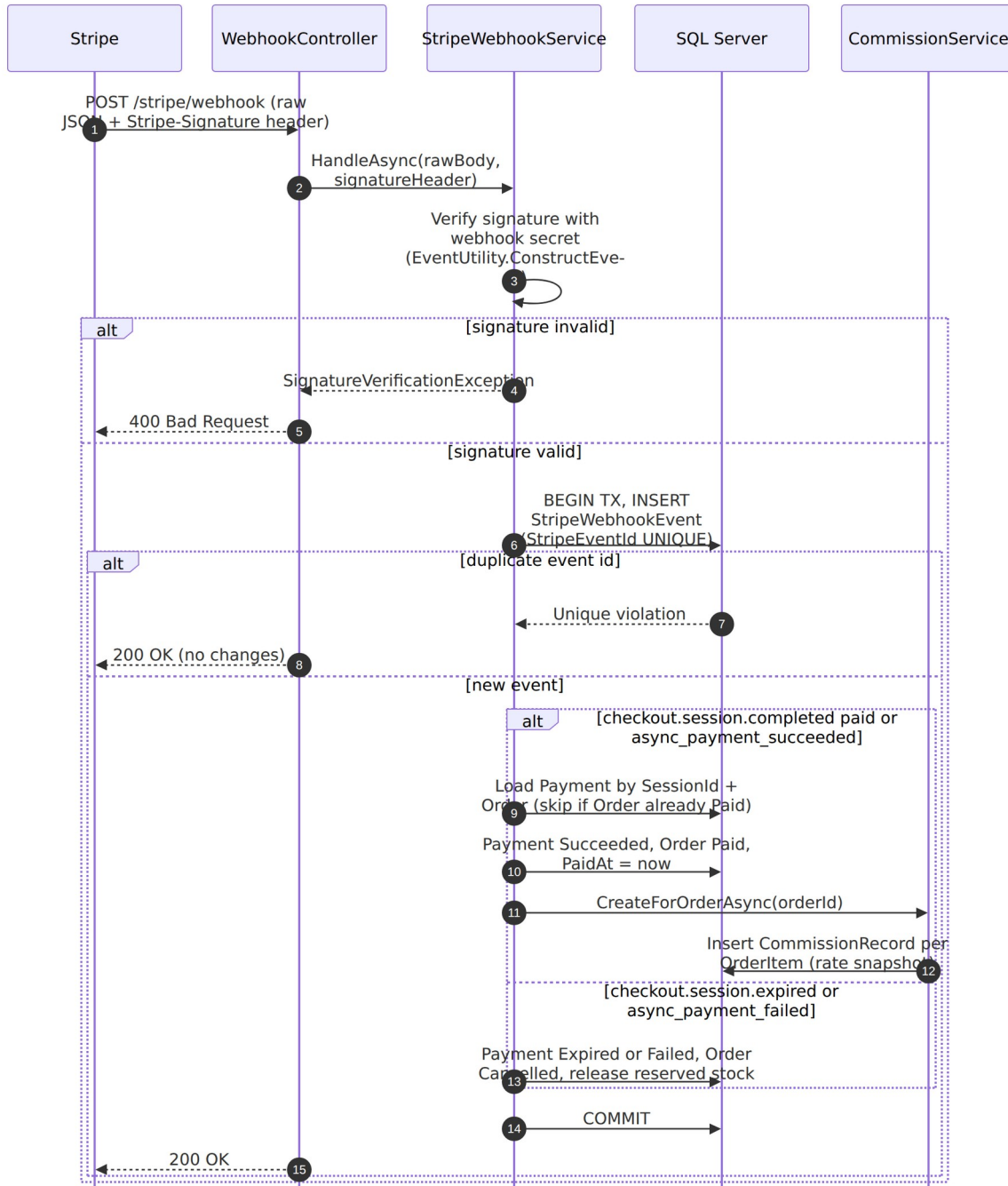


Diagram 17 | Editable source: diagrams/src/17_webhook_sequence.mmd (Mermaid) | Image: diagrams/png/17_webhook_sequence.png

الشرح: ده أهم Diagram في المشروع. (2-1) Stripe ببيعت POST على `stripe/webhook/` ب JSON خام وهيدر `Stripe-Signature`، وال Controller بيقرا ال `raw body` (مش Model Binding). (3) التحقق من التوقيع بسر ال Webhook. لو فاشل: 400. (4) بنبدأ Transaction وندخل صف في `StripeWebhookEvent` ب Unique على `StripeEventId`. لو الصف موجود بالفعل: يبقى ال Event اتعالج قبل كده، نرجع 200 من غير أي تغيير. (5) لو `checkout.session.completed` و `payment_status = paid` (أو `async_payment_succeeded`): نحدّث `Payment = Succeeded` و `Order = Paid` وننشئ (6) `CommissionRecords` لو `expired` أو `async_payment_failed: Order = Cancelled` ونرجع المخزون المحجوز. (7) COMMIT ثم 200. لأن تسجيل ال Event جوه نفس ال Transaction، لو حصل فشل في النص كل حاجة بتتراجع وال Stripe هيعيد المحاولة، ومغيش حالة نص-نص.

Diagram 18: GitHub Collaboration Workflow

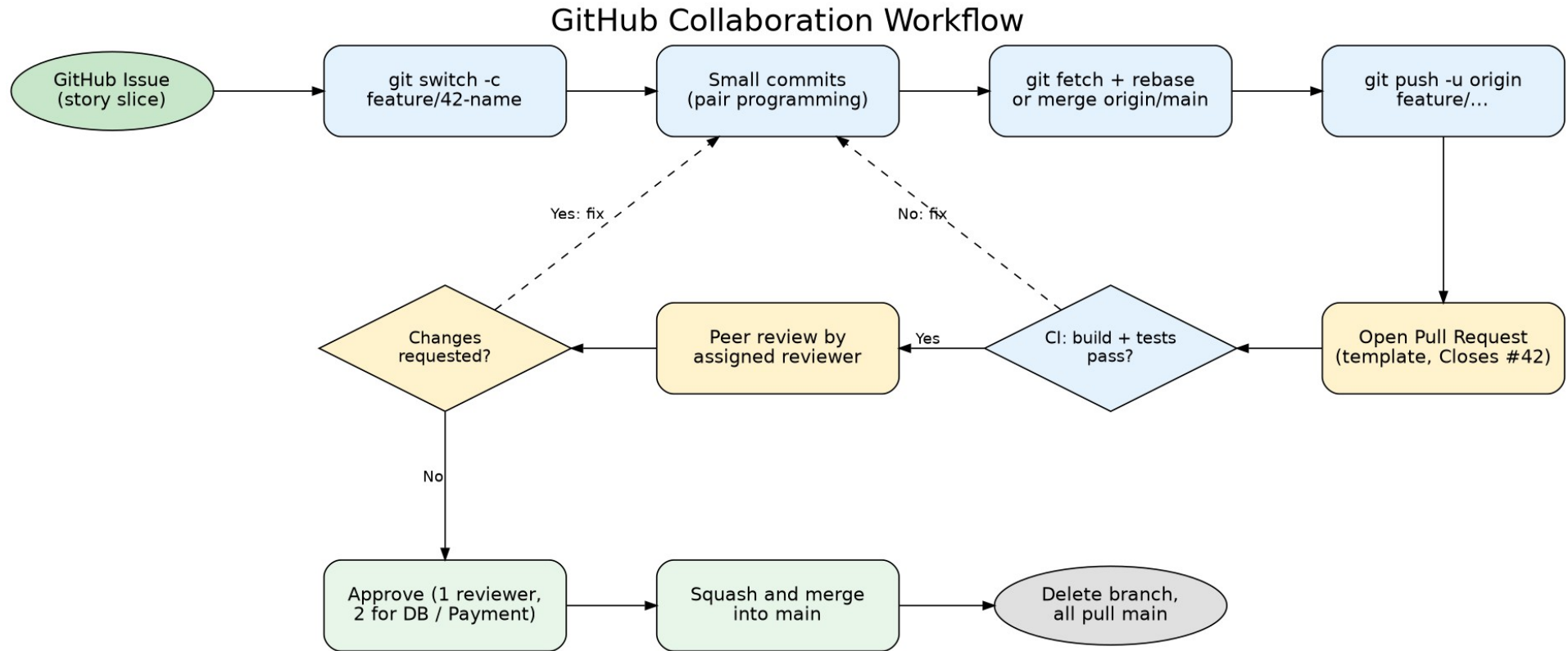


Diagram 18 | Editable source: diagrams/src/18_github_workflow.dot (Graphviz) | Image: diagrams/png/18_github_workflow.png

الشرح: كل شغل يبدأ من GitHub Issue، وبعدين Branch من **main**، و Commits صغيرة، ومزامنة مع **main** قبل ال Push، و Pull Request بقالب جاهز. ال GitHub Actions بتبني وتشغل ال Tests. المراجعة بتتم من عضو غير الكاتب. ال Merge بيكون Squash علشان تاريخ **main** يفضل نظيف. الفصل 11 فيه الأوامر العملية.

Diagram 19: Development Lifecycle (Sprint Loop)

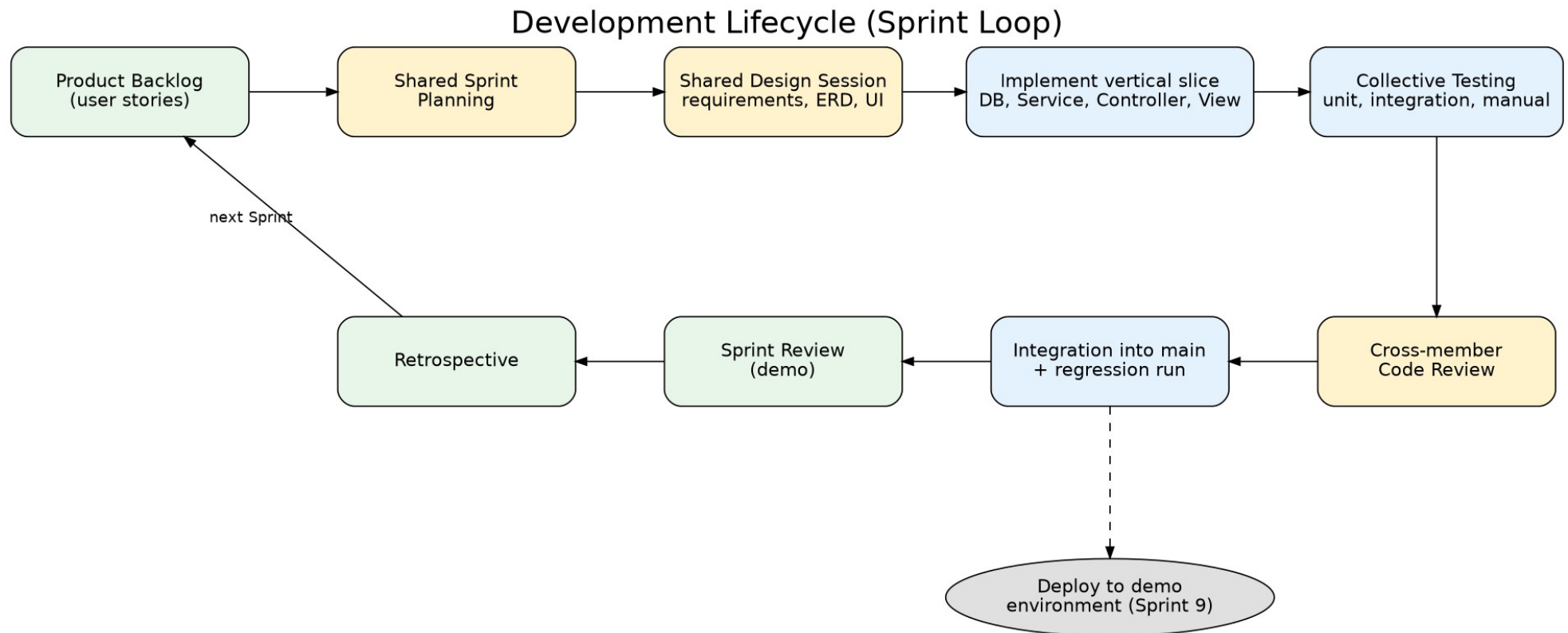


Diagram 19 | Editable source: diagrams/src/19_development_lifecycle.dot (Graphviz) | Image: diagrams/png/19_development_lifecycle.png

الشرح: الدورة بتكرر كل Sprint: Backlog, Planning مشترك، Design Session مشتركة، تنفيذ Vertical Slice كامل (Database + Service + Controller + View)، اختبار جماعي، Code Review بين الأعضاء، Integration مع Sprint Review (Demo)، ثم **main**، Retrospective. ال Deployment لل Demo بيبدأ من Sprint 9.

Diagram 20A: Order Status State Machine

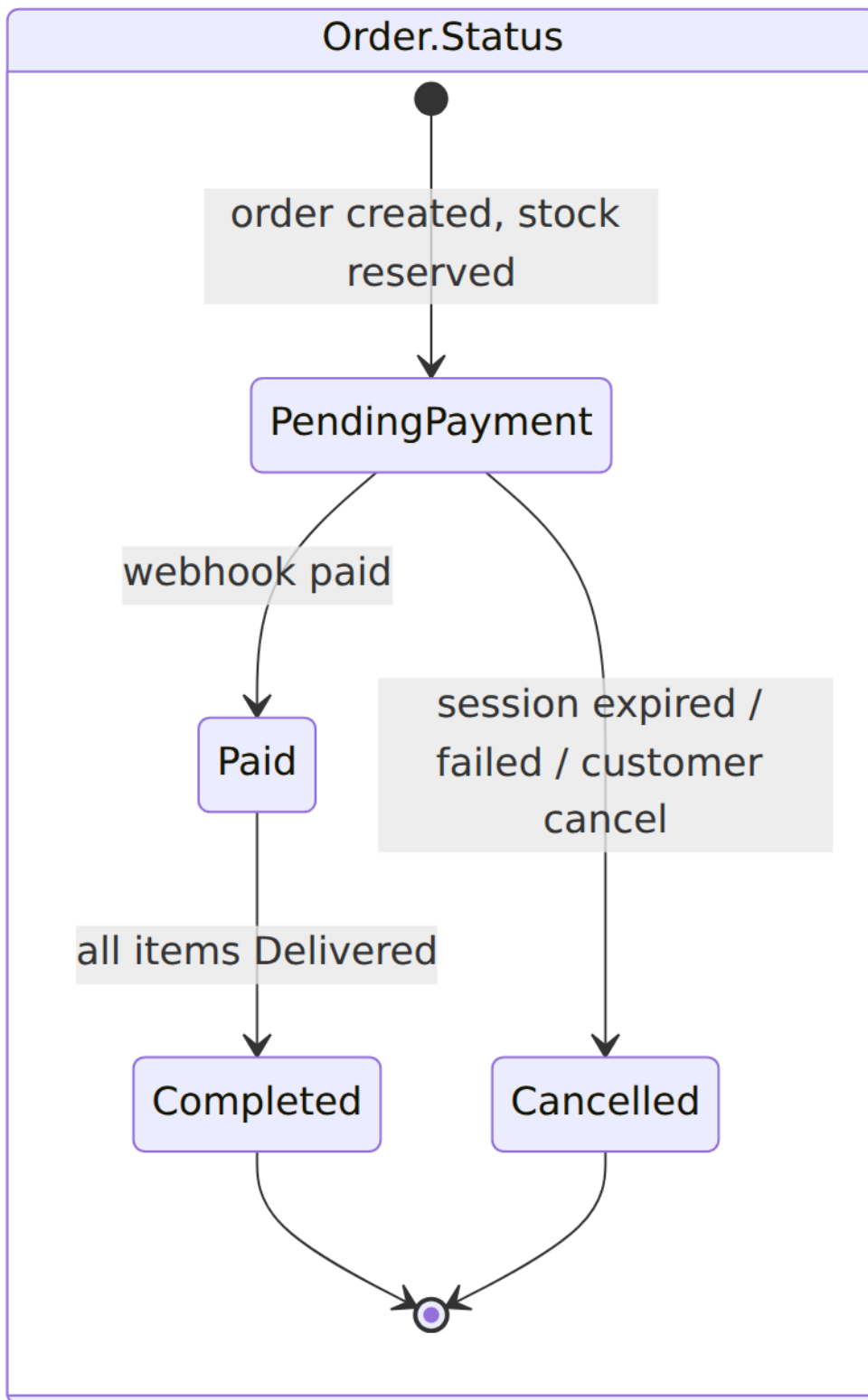


Diagram 20A | Editable source: [diagrams/src/20_state_machines.mmd](#) (Mermaid) | Image: [diagrams/png/20_state_machines.png](#)

الشرح: **Order.Status** له أربع حالات: **PendingPayment** (بعد ال Checkout)، **Paid** (بعد ال Webhook)، **Completed** (كل ال Items اتسلّمت)، **Cancelled** (انتهت ال Session أو فشلت أو العميل ألغى). مغيث انتقال من **Paid** إلى **Cancelled** في ال **MVP** (ده بيحتاج Refund وهو خارج ال Scope).

Diagram 20B: OrderItem Fulfillment State Machine

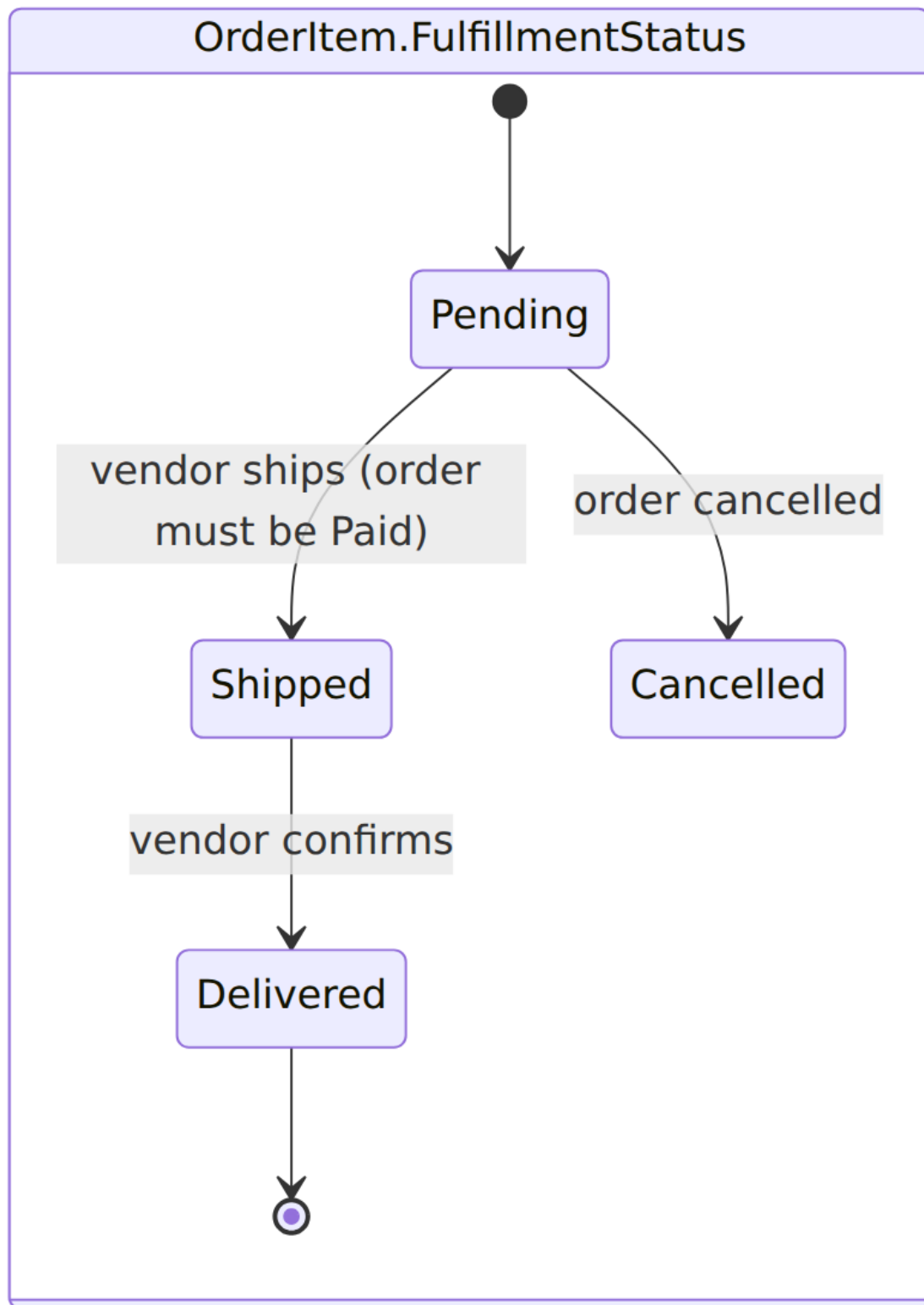


Diagram 20B | Editable source: [diagrams/src/20b_orderitem_payment_states.mmd](#) (Mermaid) | Image: [diagrams/png/20b_orderitem_payment_states.png](#)

الشرح: ال Vendor هو اللي بيحرك **FulfillmentStatus** من **Pending** لـ **Shipped** لـ **Delivered**، وبيس بعد ما ال Order يبقى **Paid**. حالة **Cancelled** بتتخط تلقائياً لما ال Order كله يتلغى.

Diagram 21: Request Lifecycle (Browser to View)

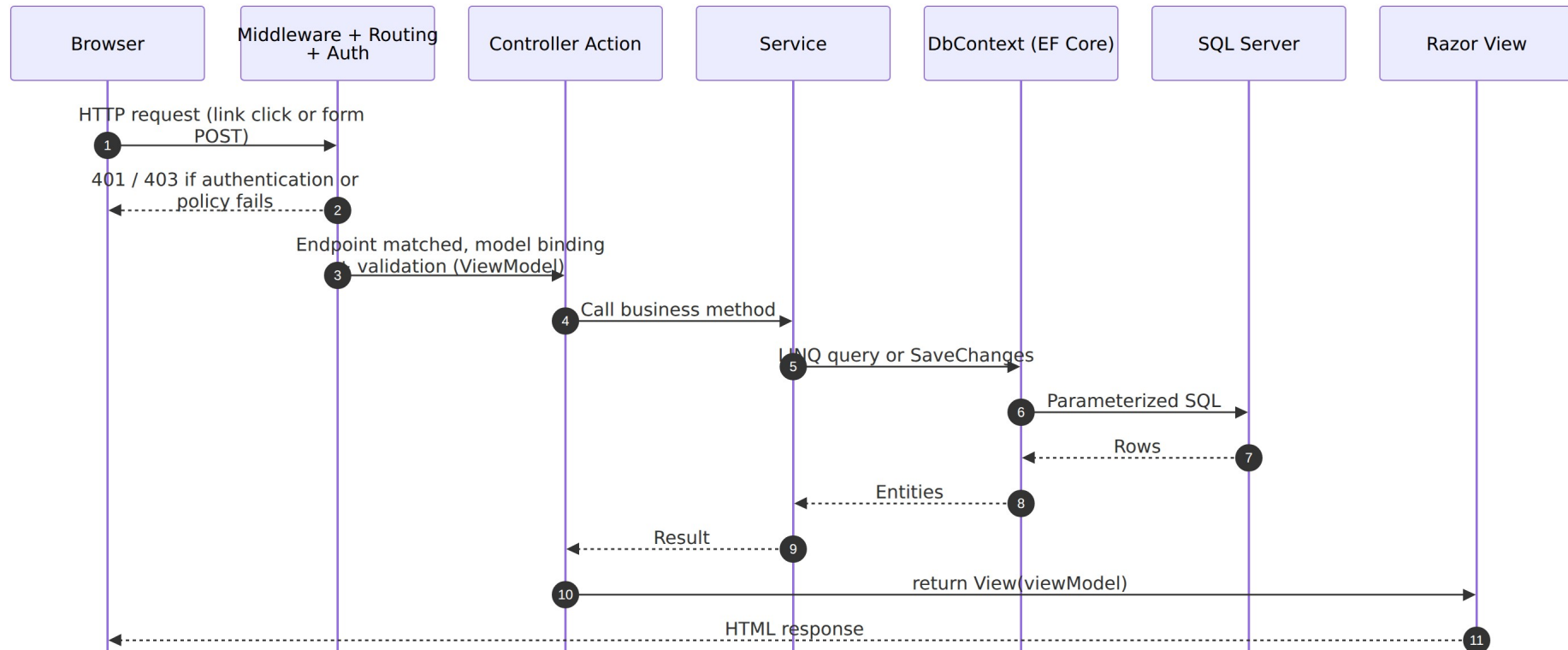


Diagram 21 | Editable source: diagrams/src/21_request_lifecycle.mmd (Mermaid) | Image: diagrams/png/21_request_lifecycle.png

الشرح: رحلة Request واحد: ال Browser يبعث Request، ال Middleware بيعمل HTTPS Redirect، Static Files، Routing، Authentication، Authorization (لو فشل: 401 أو 403). بعدها ال Controller Model Binding وال Validation على ال ViewModel، ثم ال Controller بينادي ال Service، وال Service تستخدم EF Core اللى بيولد SQL بـ Parameters ويبرجع Entities، وال Controller يمررها لـ View على شكل ViewModel وال Razor بيطلع HTML.

Diagram 22: Vendor Isolation Authorization Flow

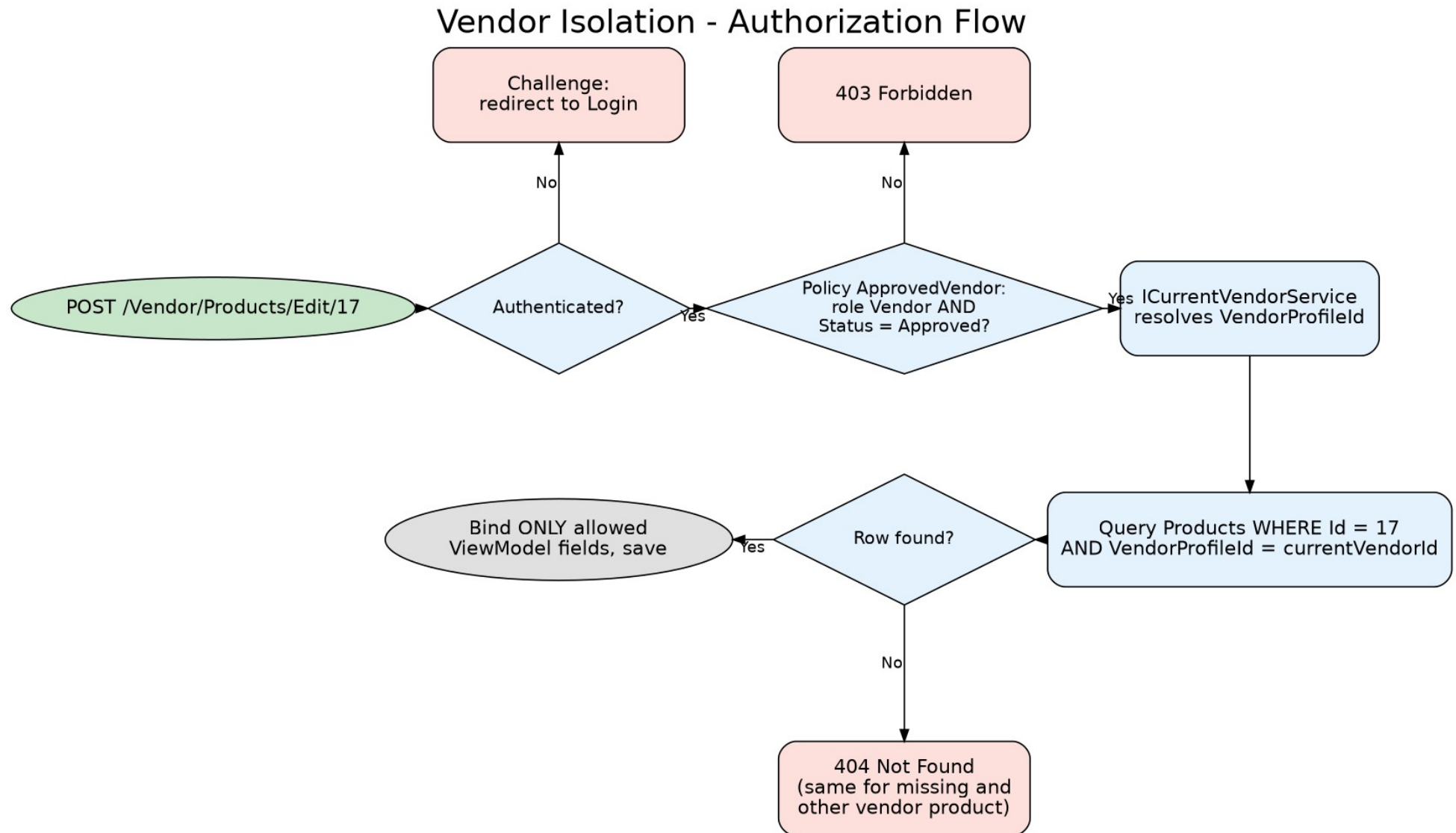


Diagram 22 | Editable source: diagrams/src/22_vendor_isolation.dot (Graphviz) | Image: diagrams/png/22_vendor_isolation.png

الشرح: مسار Request واحد على `POST /Vendor/Products/Edit/17`: التحقق من الـ Login، ثم Policy
ثم تحديد `VendorProfileId` من `ICurrentVendorService`، ثم Query بشرط `Id = 17 AND ApprovedVendor`
`VendorProfileId = currentVendorId`. لو الصف مش موجود (سواء المنتج مش موجود أو بتاع Vendor تاني) الرد دائماً
404 بنفس الشكل، علشان المهاجم ميعرفش إن المنتج موجود.

5. تصميم قاعدة البيانات (Database Design)

ال ERD الكامل في الفصل 4 (Diagram 6A و 6B و 6C). الفصل ده بيشرح كل جدول: ليه موجود، أعمدته، قيوده، وإزاي بيتصرف عند الحذف.

5.1 إزاي جداول ASP.NET Core Identity بتتوافق مع تصميمنا

لما نستخدم **Identity** مع EF Core بيتعمل تلقائياً جداول: **AspNetUsers** و **AspNetRoles** و **AspNetUserRoles** و **AspNetUserClaims** و **AspNetUserLogins** و **AspNetUserTokens** و **AspNetRoleClaims**. إحنا مش بنعيد اختراع المستخدمين: بنعمل Class اسمه **ApplicationUser : IdentityUser** فيه أعمدة إضافية (**FullName**, **IsActive**, **CreatedAt**). وبنربط باقي جداولنا بـ **AspNetUsers.Id** (من نوع **nvarchar(450)**) كـ Foreign Key. ال **AspNetUsers** بتتخزن في **AspNetRoles** (Customer, Vendor, SuperAdmin) والربط في **AspNetUserRoles**. كل جداول المشروع بتستخدم **int IDENTITY** كـ Primary Key ما عدا جداول **Identity** (string GUID).

5.2 قرار نمذجة ال Multi-Vendor Order

المعيار	Approach A: Order + OrderItems (كل Item فيه VendorId)	Approach B: Order + VendorOrders + OrderItems
عدد الجداول	أقل (جدول Order واحد)	أكثر (VendorOrders زيادة)
صعوبة التنفيذ للمبتدئين	سهلة	متوسطة إلى صعبة
Vendor-specific visibility	WHERE OrderItems.VendorProfileId = @vendor	WHERE VendorOrders.VendorProfileId = @vendor أسهل في الاستعلام
Order Status	حالة واحدة للـ Order وحالة Fulfillment لكل Item	حالة لكل VendorOrder + حالة للـ Parent. وده بيحتاج قواعد تجميع
Inventory	واحد في الحالتين (على Products)	نفس الشيء
Commission	CommissionRecord لكل OrderItem	ممكّن لكل VendorOrder أو Item
Shipping per vendor	محتاج حقول على الـ Item	طبيعي (Tracking per VendorOrder)
Future extensibility	ممكّن ترقيته لـ B لاحقاً	مناسب لـ Stripe Connect Transfers وشحن لكل Vendor

التوصية [Recommendation]: Approach A

للـ MVP هنستخدم Approach A لأنها أبسط، وكل متطلباتنا بتتغطي: الـ Vendor يشوف **OrderItems** بتاعته بس، والمخزون على **Products**، والعمولة لكل Item، والـ Status للـ Order الكامل بيتحدد من **Paid** للدفع (الدفع واحد في الحالتين) مع **FulfillmentStatus** على كل Item للشحن. الـ **VendorOrder** اتقيم ورفض للـ MVP لأنه بيضيف جدول وقواعد تجميع حالات (إيه حالة الـ Order لو شحن و Vendor لـ؟) من غير فائدة فورية. لو احتجنا لاحقاً Shipping لكل Vendor أو Stripe Connect Transfers هنضيف **VendorOrder** ونملأه من الـ **OrderItems** بـ Migration (Data Migration)، لأن **OrderItem.VendorProfileId** موجود بالفعل.

5.3 قاموس البيانات (Data Dictionary)

ملاحظات عامة على الأنواع

الاستخدام	نوع SQL Server	#C	السبب
المبالغ المالية	decimal(18,2)	decimal	دقة ثابتة، مغيث float أو double للفلوس أبداً (أخطاء تقريب).
النسب المئوية	decimal(5,2)	decimal	يغطي 0.00 إلى 100.00.

الاستخدام	نوع SQL Server	#C	السبب
التواريخ	datetime2	DateTime (UTC)	دقة أعلى ونطاق أوسع، وبخزن UTC دائماً.
النصوص	nvarchar(n)	string	Unicode للعربي والإنجليزي، وبحدد الطول.
ال Enums	nvarchar(30)	enum <HasConversion<string>()	مقروءة في ال SQL ومبتكسرش لو غيرنا ترتيب ال Enum.
Concurrency	rowversion	[]byte	بيتغير تلقائياً مع كل Update.

ApplicationUser (جدولAspNetUsers)

الغرض: هوية المستخدم. الأعمدة الإضافية بتتضاف على جدول Identity.

العمود	النوع	Null	قيود وملاحظات
Id	nvarchar(450)	لا	PK
Email / UserName	nvarchar(256)	لا	Unique (من Identity)
FullName	nvarchar(100)	لا	Required, MaxLength(100)
IsActive	bit	لا	Default 1, للحظر بدل الحذف
CreatedAt	datetime2	لا	Default SYSUTCDATETIME ()

العلاقات: 1 إلى 1..0 مع 1. VendorProfile إلى كثير مع Order وCartItem وReview. Delete: مفيش حذف مستخدمين في ال MVP (Deactivate بدلاً منه) لحماية التاريخ المالي.

VendorProfile

الغرض: بيانات المتجر وحالة الموافقة. فصل جدول عن المستخدم لأن مش كل مستخدم Vendor.

العمود	النوع	Null	قيود وملاحظات
Id	int	لا	PK, IDENTITY
UserId	nvarchar(450)	لا	FK إلى Unique AspNetUsers (1 إلى 1)
StoreName	nvarchar(100)	لا	Unique, Required
Description	nvarchar(1000)	نعم	
LogoPath	nvarchar(260)	نعم	مسار نسبي
Status	nvarchar(20)	لا	Pending/Approved/Rejected/Suspended, Default Pending
RejectionReason	nvarchar(500)	نعم	إجباري عند Rejected (Validation في ال Service)
ReviewedByUserId	nvarchar(450)	نعم	FK إلى AspNetUsers (ال Admin)
ReviewedAt	datetime2	نعم	
CreatedAt	datetime2	لا	

Delete: Restrict من Index(Status). Unique(StoreName). Unique(UserId). Indexes: User. لفائمة Pending.

Category

الغرض: تصنيف المنتجات (Flat بدون Parent/Child في ال MVP لتبسيط ال UI).

العمود	النوع	Null	قيود
Id	int	لا	PK
Name	nvarchar(80)	لا	Unique
Slug	nvarchar(100)	لا	Unique, لا URLs
IsActive	bit	لا	Default 1

Delete: Restrict (لو فيه Products)؛ بنعمل Deactivate. ال Admin فقط يدير ال Categories.

Product

الغرض: المنتج اللي بيعه Vendor واحد.

العمود	النوع	Null	قيود وملاحظات
Id	int	لا	PK
VendorProfileId	int	لا	FK, أساس العزل
CategoryId	int	لا	FK
Name	nvarchar(150)	لا	Required
Description	nvarchar(4000)	نعم	
Price	decimal(18,2)	لا	CHECK (Price > 0)
StockQuantity	int	لا	CHECK (StockQuantity >= 0)
IsActive	bit	لا	Default 1
RowVersion	rowversion	لا	Concurrency Token
CreatedAt / UpdatedAt	datetime2	لا / نعم	

Indexes: (VendorProfileId, IsActive) لصفحة المنتجات عند ال. Vendor. (CategoryId, IsActive, Price) للفلتر، Name للبحث. **Delete: Restrict** من Vendor ومن Category. قاعدة: لا حذف لمنتج له OrderItems. (BR-09).

ProductImage

العمود	النوع	Null	قيود
Id	int	لا	PK
ProductId	int	لا	FK, Cascade (الصور ملهاش معنى من غير المنتج)
ImagePath	nvarchar(260)	لا	اسم ملف يولده السيرفر (GUID)
IsPrimary	bit	لا	صورة رئيسية واحدة (Validation في ال Service)
SortOrder	int	لا	Default 0

CartItem

الغرض: محتويات ال Cart لكل Customer. مفيش عمود سعر (BR-03).

العمود	النوع	Null	قيود
Id	int	لا	PK

العمود	النوع	Null	قيود
CustomerId	nvarchar(450)	لا	FK, Cascade
ProductId	int	لا	FK, Cascade
Quantity	int	لا	CHECK (Quantity > 0)
AddedAt	datetime2	لا	

Unique Index على (CustomerId, ProductId): يمنع تكرار الصف ويجبر الكود يزود الكمية.

Order

الغرض: طلب العميل الواحد (حتى لو من كذا Vendor).

العمود	النوع	Null	قيود وملاحظات
Id	int	لا	PK
OrderNumber	nvarchar(30)	لا	Unique. مثل MH-20261007-483920
CustomerId	nvarchar(450)	لا	FK, Restrict
Status	nvarchar(20)	لا	PendingPayment/Paid/Completed/Cancelled
TotalAmount	decimal(18,2)	لا	مجموع LineTotal (محفوظ للمطابقة مع Stripe)
Currency	char(3)	لا	Default USD
ShippingFullName / Phone / Address / City / Country	nvarchar	لا	Snapshot وقت الطلب
CreatedAt	datetime2	لا	
PaidAt	datetime2	نعم	

Indexes: Unique(OrderNumber). (CustomerId, CreatedAt DESC). Index(Status)

OrderItem

الغرض: صف في الطلب، تابع لـ Vendor واحد. ده قلب ال Marketplace.

العمود	النوع	Null	قيود وملاحظات
Id	int	لا	PK
OrderId	int	لا	FK, Cascade
ProductId	int	لا	FK, Restrict
VendorProfileId	int	لا	FK, Restrict, Snapshot وقت الشراء
ProductNameSnapshot	nvarchar(150)	لا	
UnitPrice	decimal(18,2)	لا	سعر وقت الشراء
Quantity	int	لا	CHECK (Quantity > 0)
LineTotal	decimal(18,2)	لا	CHECK (LineTotal = UnitPrice * Quantity)
FulfillmentStatus	nvarchar(20)	لا	Pending/Shipped/Delivered/Cancelled
ShippedAt / DeliveredAt	datetime2	نعم	

.Vendor. (OrderId). (ProductId) لشاشة ال **Indexes:** (VendorProfileId, OrderId)

Payment

الغرض: محاولات الدفع لل Order (1 إلى كثير لأن العميل ممكن يعيد المحاولة بعد انتهاء Session).

العمود	النوع	Null	قيود
Id	int	لا	PK
OrderId	int	لا	FK, Restrict
Provider	nvarchar(20)	لا	Stripe
StripeCheckoutSessionId	nvarchar(255)	نعم حتى الإنشاء	Unique (Filtered IS NOT NULL)
StripePaymentIntentId	nvarchar(255)	نعم	Index
Amount / Currency	decimal(18,2) / char(3)	لا	
Status	nvarchar(20)	لا	Pending/Succeeded/Failed/Expired
FailureMessage	nvarchar(500)	نعم	
CreatedAt / CompletedAt	datetime2	لا / نعم	

مهم: مغيث أي بيانات كارت هنا. بنخزن Stripe IDs فقط.

StripeWebhookEvent

الغرض: ضمان ال Idempotency. كل Event بيوصل بيتسجل هنا مرة واحدة.

العمود	النوع	قيود
Id	int	PK
StripeEventId	nvarchar(255)	Unique (مثل ..._evt)
EventType	nvarchar(100)	
ProcessedAt	datetime2	

دي Entity إضافية بسبب واضح: من غيرها مغيث طريقة آمنة لمنع تنفيذ نفس ال Event مرتين.

CommissionSetting

الغرض: تاريخ نسب العمولة. ال Admin مبيعدلش الصف القديم، بيضيف صف جديد.

العمود	النوع	قيود
Id	int	PK
RatePercent	decimal(5,2)	CHECK (RatePercent BETWEEN 0 AND 100)
EffectiveFrom	datetime2	
CreatedByUserId	nvarchar(450)	FK (Admin)
IsActive	bit	Filtered Unique Index على 1 = IsActive: صف نشط واحد فقط

CommissionRecord

الغرض: سجل مالي لا يتغير: كل OrderItem مدفوع له سجل واحد.

العمود	النوع	قيود وملاحظات
Id	int	PK
OrderItemId	int	FK، Unique (1 إلى 1)
VendorProfileId	int	FK
CommissionSettingId	int	FK
GrossAmount	decimal(18,2)	OrderItem.LineTotal =
RatePercentSnapshot	decimal(5,2)	النسبة وقت الدفع
CommissionAmount	decimal(18,2)	ROUND(Gross × Rate / 100, 2)
VendorNetAmount	decimal(18,2)	Gross - CommissionAmount. CHECK (VendorNetAmount = GrossAmount - CommissionAmount)
Status	nvarchar(20)	Recorded/Reversed (Refunds للمستقبل مع)
CreatedAt	datetime2	

Review (Should Have)

العمود	النوع	قيود
Id	int	PK
ProductId	int	FK. Cascade
CustomerId	nvarchar(450)	FK. Restrict
Rating	int	CHECK (Rating BETWEEN 1 AND 5)
Comment	nvarchar(1000)	نعم
CreatedAt	datetime2	

Unique على (ProductId, CustomerId). القاعدة BR-12 (لازم ال Customer يكون استلم المنتج) بتتفرض في ال .Service

ليه SQL Server بيمنع Multiple Cascade Paths؟

لو جدول له أكثر من طريق Cascade لنفس الصف (مثلاً OrderItem من Order ومن VendorProfile)، SQL Server بيرفض ال Migration. عشان كده بنخلي Cascade فقط في العلاقات اللي الابن مالوش معنى بدون الأب (، ProductImage، CartItem، OrderItem→Order، Review→Product) و Restrict في الباقي.

5.4 تحليل ال Normalization

- **1NF**: كل عمود قيمة واحدة (مفيش قوائم في عمود). الصور في جدول ProductImages مش في عمود.
 - **2NF**: كل الجداول بـ PK مفردة (Id) فمفيش Partial Dependency.
 - **3NF**: مفيش Transitive Dependency: اسم ال Category في Categories وليس في Products.
- Denormalization مقصودة (Snapshots)** بتخدم دقة التاريخ المالي:

العمود	ليه مكرر؟
OrderItem.UnitPrice, ProductNameSnapshot	سعر واسم المنتج ممكن يتغيروا بعدين، والطلب القديم لازم يفضل زي ما اتدفع.
OrderItem.VendorProfileId	مشتق من Product، بس بنثبتته علشان استعلامات ال Vendor سريعة وعلشان ال Isolation مايعتمدش على Join.

العمود	ليه مكرر؟
Order.TotalAmount	يُحسب من الـ Items، بس بنخزنه للمطابقة مع Stripe.
CommissionRecord.RatePercentSnapshot, GrossAmount	تغيير النسبة أو السعر ميغيرش سجلات الماضي.
*Order.Shipping	العنوان وقت الطلب، حتى لو الـ Customer غير بياناته.

5.5 إعداد EF Core (Fluent API)

بنحط كل Entity Configuration في Class لوحده تحت **Data/Configurations** علشان **ApplicationDbContext** يفضل صغير، وكمان علشان الأعضاء ميتعارضوش في نفس الملف.

```
public class ProductConfiguration : IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> b)
    {
        b.ToTable("Products", t =>
        {
            t.HasCheckConstraint("CK_Products_Price", "[Price] > 0");
            t.HasCheckConstraint("CK_Products_Stock", "[StockQuantity] >= 0");
        });
        b.Property(p => p.Name).HasMaxLength(150).IsRequired();
        b.Property(p => p.Price).HasColumnType("decimal(18,2)");
        b.Property(p => p.RowVersion).IsRowVersion();
        b.HasOne(p => p.Vendor).WithMany(v => v.Products)
            .HasForeignKey(p => p.VendorProfileId).OnDelete(DeleteBehavior.Restrict);
        b.HasOne(p => p.Category).WithMany(c => c.Products)
            .HasForeignKey(p => p.CategoryId).OnDelete(DeleteBehavior.Restrict);
        b.HasIndex(p => new { p.VendorProfileId, p.IsActive });
        b.HasIndex(p => new { p.CategoryId, p.IsActive, p.Price });
    }
}

public class OrderItemConfiguration : IEntityTypeConfiguration<OrderItem>
{
    public void Configure(EntityTypeBuilder<OrderItem> b)
    {
        b.ToTable("OrderItems", t =>
        {
            t.HasCheckConstraint("CK_OrderItems_LineTotal", "[LineTotal] = [UnitPrice] * [Quantity]");
            b.Property(i => i.UnitPrice).HasColumnType("decimal(18,2)");
            b.Property(i => i.LineTotal).HasColumnType("decimal(18,2)");
            b.Property(i => i.FulfillmentStatus).HasConversion<string>().HasMaxLength(20);
            b.HasOne(i => i.Order).WithMany(o => o.Items).HasForeignKey(i => i.OrderId)
                .OnDelete(DeleteBehavior.Cascade);
            b.HasOne(i => i.Vendor).WithMany().HasForeignKey(i => i.VendorProfileId)
                .OnDelete(DeleteBehavior.Restrict);
            b.HasOne(i => i.Product).WithMany().HasForeignKey(i => i.ProductId)
                .OnDelete(DeleteBehavior.Restrict);
            b.HasIndex(i => new { i.VendorProfileId, i.OrderId });
        });
    }
}

public class StripeWebhookEventConfiguration : IEntityTypeConfiguration<StripeWebhookEvent>
{
    public void Configure(EntityTypeBuilder<StripeWebhookEvent> b)
    {
        b.Property(e => e.StripeEventId).HasMaxLength(255).IsRequired();
        b.HasIndex(e => e.StripeEventId).IsUnique();
    }
}
```

وفي **ApplicationDbContext**:

```
protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder); // IMPORTANT: configures Identity tables
    builder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext).Assembly);
}
```

5.6 استراتيجية الـ Migrations

1. **Code-First**: نغَيِّر الـ Entity والـ Configuration. وبعدين `dotnet ef migrations add AddVendorProfile`. ثم `dotnet ef database update`.
2. اسم الـ Migration يوصف الـ Feature (`AddOrdersAndOrderItems`)، ومتتسماش `Update1`.
3. ممنوع تعديل Migration اتعمله **Merge** في `main`. لو فيه غلط، نعمل Migration جديدة.
4. قواعد التنسيق بين الأعضاء في الفصل 11 (عضو واحد بيعمل Migration لكل Pull Request، ونعمل Rebase قبل إنشائها).
5. على Production: نطبّق الـ Migrations بـ `dotnet ef migrations script --idempotent` أو `Script` أو خطوة منفصلة في الـ Deployment. مش تلقائياً عند بدء التطبيق ([Team Decision])، التلقائي مقبول للـ Demo فقط).

5.7 استراتيجية الـ Seed Data

البيانات	البيئة	الطريقة
Roles (Customer, Vendor, SuperAdmin)	كل البيئات	<code>DbSeeder</code> يتأكد من وجودها (Idempotent)
Super Admin (Email/Password من Configuration)	كل البيئات	من <code>user-secrets</code> أو Environment Variables
<code>CommissionSetting</code> الافتراضي (%10)	كل البيئات	يتزرع لو مفيش صف نشط
Categories أساسية	كل البيئات	<code>HasData</code> أو الـ <code>DbSeeder</code>
Demo Vendors + Products + Customers	Development / فقط Demo	<code>DbSeeder</code> بشرط <code>env.IsDevelopment ()</code> أو Flag واضح

5.8 قرارات التصميم المهمة (ملخص)

1. **Approach A** للـ Orders (شرح 5.2).
2. **Reserve Stock** عند إنشاء الـ Order بدل الخصم بعد الدفع، علشان نمنع Overselling ونبسّط الـ Webhook.
3. **Snapshots** للأسعار والعناوين والعمولة.
4. جدول `StripeWebhookEvent` للـ Idempotency.
5. `CommissionSetting Append-Only` مع `Filtered Unique Index`.
6. **Enums** كـ `string` للقراءة.
7. لا حذف فيزيائي للـ Users والـ Products المرتبطة بـ Orders.
8. **Cart** في الـ Database مش Session، علشان يفضل بعد إعادة تشغيل السيرفر ويشغل على كذا جهاز.

6. معمارية النظام وهيكل المشروع (System Architecture)

6.1 مسؤوليات كل مكون

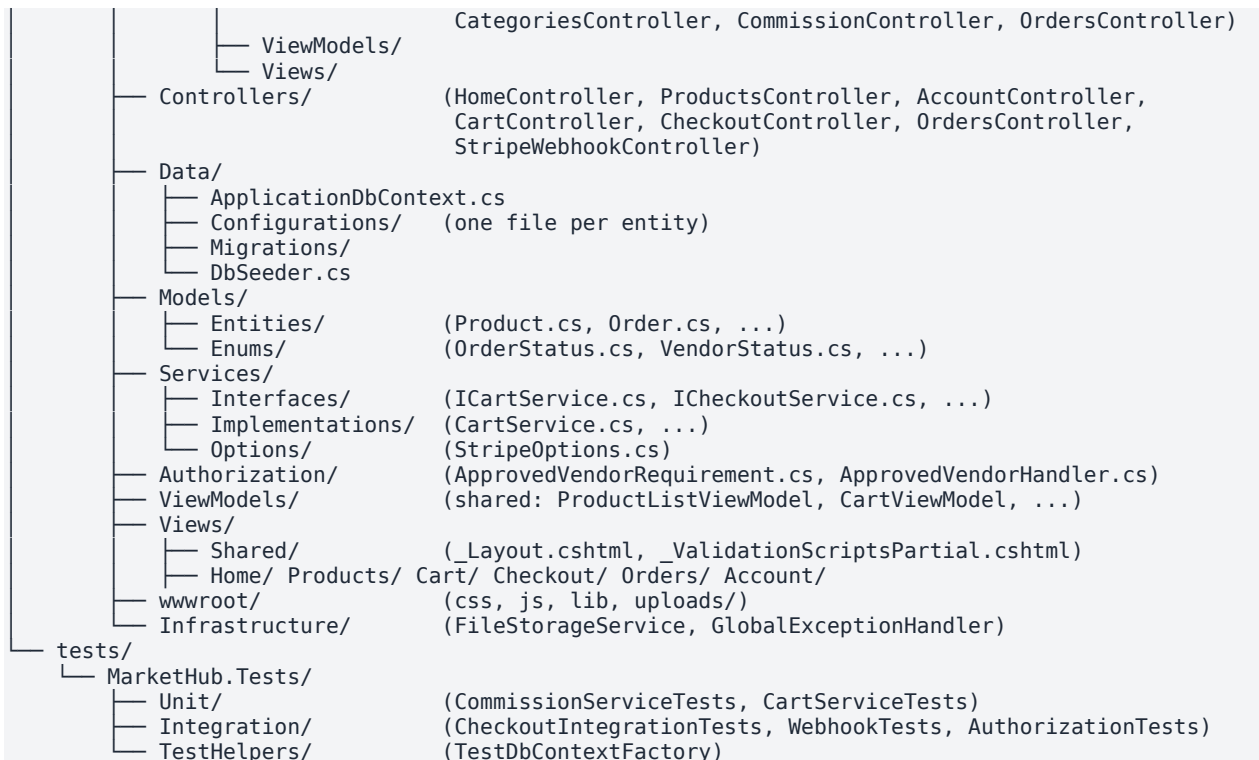
المكون	المسؤولية	مثال في مشروعنا
Controller	يستقبل ال Request، يتحقق من ModelState، ينادي Service، ويرجع View أو Redirect. مفيهوش Business Logic.	CartController.Add
Entity (Model)	Class بيتمثل جدول في ال Database.	Product, Order
ViewModel	شكل البيانات اللي ال View محتاجها أو الفورم ببيعتها، ومعاها Validation Attributes. بيمنع Overposting.	ProductEditViewModel
View (Razor)	عرض HTML فقط.	Views/Products/Details.cshtml
Service	كل ال Business Rules (التحقق من المخزون، حساب العمولة، ال Transactions).	CheckoutService
Repository	مش هنستخدمه. DbContext كفاية (انظر 1.3).	n/a
ApplicationDbContext	بوابة ال EF Core لل Database وبيتمثل Unit of Work.	Data/ApplicationDbContext.cs
ASP.NET Core Identity	Users, Roles, Password Hashing, Cookies.	UserManager<ApplicationUser>
Middleware	مكونات في ال Pipeline بتعالج كل Request (HTTPS, Auth, Exception Handling).	UseAuthentication()
Dependency Injection	ال Framework بيحقن ال Services في ال Constructors بدل ما نعمل .new.	AddScoped<ICartService>, ICartService
Configuration	appsettings.json + user-secrets + Environment Variables.	Stripe:SecretKey
Validation	Data Annotations + Server-Side Checks في ال Services.	[Range(0.01, 100000)]
Logging	<ILogger> لتسجيل الأحداث المهمة.	Webhook رفض
Error Handling	UseExceptionHandler + صفحات Error مخصصة.	Home/Error/

6.2 هيكل الفولدرات المقترح

```

MarketHub/
├── MarketHub.sln
├── global.json
├── README.md
├── .gitignore
├── .editorconfig
├── docs/ (this documentation package + diagrams)
├── .github/
│   ├── workflows/ci.yml
│   └── pull_request_template.md
├── src/
│   └── MarketHub.Web/
│       ├── Program.cs
│       ├── appsettings.json
│       ├── appsettings.Development.json
│       └── Areas/
│           ├── Vendor/
│           │   ├── Controllers/ (VendorBaseController, DashboardController, ProductsController, OrdersController, StoreController)
│           │   ├── ViewModels/
│           │   └── Views/
│           └── Admin/
│               └── Controllers/ (DashboardController, VendorApplicationsController,

```



الهيكل ده مطابق للـ 7 Diagrams و الـ 8: الـ Controllers و الـ Data (Data Access). **Services** (Application). **Areas** (Presentation).

6.3 ليه Areas/Admin و Areas/Vendor ؟ (تقييم)

المزايا	العيوب
فصل واضح للـ Views لكل Role.	محتاجين نكتب [Area("Vendor")] و الـ asp-area في الـ Links.
نقدر نطبق [Authorize] على الـ Base Controller للـ Area كلها.	Routing أعقد شوية (نحتاج MapAreaControllerRoute).
أقل تعارض في Git: كل Role في فولدر مستقل.	Shared Layout لكل Area محتاج ViewStart_ خاص.

القرار [Recommendation]: نستخدم Areas، لأن الأمان (Authorize على مستوى Area) والتنظيم أهم من البساطة اللحظية. الـ Customer Area مش محتاجة Area لأنها هي الـ Storefront الرئيسية.

6.4 دورة حياة الـ Request كاملة

(Diagram 21): نستخدم مثال POST /Vendor/Products/Create:

- الـ Browser يبعث Request فيه الـ Cookie وفورم الـ Product و الـ AntiForgery Token.
- Middleware Pipeline: UseHttpsRedirection ثم UseStaticFiles ثم UseRouting ثم UseAuthentication (يقرأ الـ Cookie ويعرف المستخدم) ثم UseAuthorization (يطبق Policy ApprovedVendor).
- Routing يوصل الـ Request لـ Areas/Vendor/ProductsController.Create.
- Model Binding + Validation: البيانات بتتحول لـ ProductCreateViewModel، و ModelState بيتملئ بالأخطاء لو فيه.
- الـ Controller يتأكد من ModelState.IsValid، وينادي IProductService.CreateAsync.
- الـ Service يطبق Business Rules ويستخدم ApplicationDbContext (EF Core) اللي بيوّد SQL بـ Parameters ويرسله لـ SQL Server.

7. ال **Controller** يرجع **RedirectToAction("Index")** (نمط Post-Redirect-Get) علشان منكرش الإرسال بال (Refresh).
8. ال **View** (في ال GET التالي) بيتحوّل ل HTML ويرجع لل Browser.

6.5 أمثلة كود

Entity

```
public class Product
{
    public int Id { get; set; }
    public int VendorProfileId { get; set; }
    public VendorProfile Vendor { get; set; } = null!;
    public int CategoryId { get; set; }
    public Category Category { get; set; } = null!;
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
    public bool IsActive { get; set; } = true;
    public byte[] RowVersion { get; set; } = Array.Empty<byte>();
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public ICollection<ProductImage> Images { get; set; } = new List<ProductImage>();
}
```

ViewModel (من غير VendorProfileId)

```
public class ProductCreateViewModel
{
    [Required, StringLength(150)]
    public string Name { get; set; } = string.Empty;

    [StringLength(4000)]
    public string? Description { get; set; }

    [Range(0.01, 1_000_000)]
    public decimal Price { get; set; }

    [Range(0, 100_000)]
    public int StockQuantity { get; set; }

    [Required]
    public int CategoryId { get; set; }

    public List<IFormFile> Images { get; set; } = new();
}
```

Implementation , Service Interface

```
public interface ICartService
{
    Task<Result> AddAsync(string customerId, int productId, int quantity);
    Task<CartItem> GetCartItem(string customerId);
}

public class CartService : ICartService
{
    private readonly ApplicationDbContext _db;
    public CartService(ApplicationDbContext db) => _db = db;

    public async Task<Result> AddAsync(string customerId, int productId, int quantity)
    {
        if (quantity < 1) return Result.Fail("Quantity must be at least 1.");

        // Price and stock are always read from the database, never from the browser.
        var product = await _db.Products
            .Include(p => p.Vendor)
            .FirstOrDefaultAsync(p => p.Id == productId
                && p.IsActive && p.Vendor.Status == VendorStatus.Approved);
        if (product is null) return Result.Fail("Product is not available.");

        var item = await _db.CartItems
```

```

        .FirstOrDefaultAsync(c => c.CustomerId == customerId && c.ProductId == productId);
        var newQty = (item?.Quantity ?? 0) + quantity;
        if (newQty > product.StockQuantity) return Result.Fail("Not enough stock.");

        if (item is null)
            _db.CartItems.Add(new CartItem { CustomerId = customerId, ProductId = productId,
Quantity = newQty });
        else
            item.Quantity = newQty;

        await _db.SaveChangesAsync();
        return Result.Ok();
    }
}

```

Controller

```

[Authorize(Roles = Roles.Customer)]
public class CartController : Controller
{
    private readonly ICartService _cart;
    public CartController(ICartService cart) => _cart = cart;

    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Add(int productId, int quantity = 1)
    {
        var userId = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
        var result = await _cart.AddAsync(userId, productId, quantity);
        TempData[result.Success ? "Success" : "Error"] = result.Message;
        return RedirectToAction(nameof(Index));
    }
}

```

Program.cs (الهيكل العام)

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<ApplicationDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(/* options */)
    .AddEntityFrameworkStores<ApplicationDbContext>().AddDefaultTokenProviders();

builder.Services.Configure<StripeOptions>(builder.Configuration.GetSection("Stripe"));
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<ICartService, CartService>();
builder.Services.AddScoped<ICheckoutService, CheckoutService>();
builder.Services.AddScoped<IPaymentService, StripePaymentService>();
builder.Services.AddScoped<IStripeWebhookService, StripeWebhookService>();
builder.Services.AddScoped<ICommissionService, CommissionService>();
builder.Services.AddScoped<ICurrentVendorService, CurrentVendorService>();
builder.Services.AddHttpContextAccessor();
builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(name: "areas", pattern:
    "{area:exists}/{controller=Dashboard}/{action=Index}/{id?}");
app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");

await DbSeeder.SeedAsync(app.Services);
app.Run();

```

[Team Decision]: ترتيب ال Middleware مهم جداً (Authentication قبل Authorization). راجعوا Microsoft Learn: ASP.NET Core Middleware للترتيب الموصى به في النسخة المستخدمة، وجربوا ال Template الرسمي بتاع MVC كمرجع.

ملفات الإعدادات (Configuration)

```
{
  "ConnectionStrings": { "DefaultConnection": "" },
  "Stripe": { "SecretKey": "", "WebhookSecret": "", "PublishableKey": "" },
  "Seed": { "AdminEmail": "", "AdminPassword": "" }
}
```

القيم الحقيقية ستتخط في `dotnet user-secrets` محلياً، وفي Environment Variables على الاستضافة (مثلاً `Stripe__SecretKey`). الملف المرفوع على GitHub فيه مفاتيح فاضية بس.

6.6 Error Handling و Logging و Validation

- **Validation** طبقتين: Data Annotations على ال ViewModel (شكل البيانات) + Business Validation في ال Service (المخزون، الملكية). ال Client-Side Validation تحسين لل UX فقط، ومش بديل.
- **Logging: ILogger<T>** لتسجيل: رفض توقيع Webhook، فشل Transaction، محاولات الوصول المرفوضة. ممنوع نسجل Secrets أو بيانات كارت أو Passwords.
- **Error Handling: UseExceptionHandler** في Production، وصفحة **Error** بتعرض رسالة عامة ومعها Request Id. ال Services بترجع **Result** للأخطاء المتوقعة (مخزون ناقص) وبترمي Exceptions للأخطاء غير المتوقعة فقط.

7. مواصفات الـ Features (Feature Specifications)

كل Feature هنا ستتبنى كـ **Vertical Slice**: Database + Service + Controller + View + Tests مع بعض، وبمشاركة كل الفريق (الفصل 10). الجداول التالية هي الـ Checklist اللي بنراجع عليها قبل ما نقول إن الـ Feature خلصت.

A. Public Storefront (Home, Listing, Details, Categories, Search, 7.1 Filter, Sort, Pagination)

البند	التفاصيل
Purpose (الغرض)	الـ Guest والـ Customer يتصفحوا منتجات نشطة من متاجر Approved بس، ويلاقوا اللي عايزينه بسرعة.
User Stories	US-01, US-05.
Dependencies	Categories + Products + ProductImages + VendorProfile (Seed Data).
Database Changes	لا جداول جديدة. Indexes: (CategoryId, IsActive, Price) و Name.
Backend Tasks	IPublicService.SearchAsync(ProductQuery) يبني Query بـ IQueryable (شرط + Active Approved، بعدما Search/Category/Price/Sort)، ويرجع <PagedResult> بـ Skip/Take. استخدموا AsNoTracking () للقراءة.
Frontend Tasks	Home بـ Featured + Categories، Listing بـ Grid (Bootstrap Cards) وفورم فلتر (GET)، Details بـ Pagination Component، صور و زر Add to Cart.
Authorization Rules	Public ([AllowAnonymous]). المنتجات غير النشطة أو الخاصة بـ Vendor غير معتمد ترجع 404.
Validation	page ≥ 1، pageSize في نطاق مسموح (12-48)، قيم Sort من قائمة ثابتة (Whitelist).
Error Scenarios	Category غير موجودة، صفحة خارج النطاق، Search فاضي.
Testing Requirements	Unit: فلتر وترتيب ومنتجات Inactive مخفية. Manual: Pagination على موبايل.
Definition of Done	كل المنتجات المعروضة Active + Approved، الصفحات سريعة، Responsive.

B. Authentication (Registration, Login, Logout, Roles) 7.2

البند	التفاصيل
Purpose (الغرض)	تسجيل المستخدمين ودخولهم، وتحديد الـ Roles.
User Stories	US-02, US-03.
Dependencies	ASP.NET Core Identity + DbSeeder والـ Super Admin Roles.
Database Changes	Identity Migration + أعمدة FullName, IsActive, CreatedAt على ApplicationUser.
Backend Tasks	(Register, Login, Logout) باستخدام AccountController باستخدام SignInManager و UserManager، إضافة Role Customer تلقائياً، Lockout، منع دخول IsActive = false.
Frontend Tasks	فورم Register/Login بـ Validation Summary، تغيير شكل الـ Navbar حسب الـ Role.
Authorization Rules	[Authorize] على صفحات الـ Customer والـ [AllowAnonymous] Areas، على Account.
Validation	Email Format وتفرّد، AntiForgery، Password Policy على كل returnUrl، POST، يتفحص بـ Url.IsLocalUrl.
Error Scenarios	Email مكرر، Password ضعيف، حساب مغلق، Lockout.
Testing Requirements	Integration: دخول بـ Role صح/غلط. Manual: Open Redirect.
Definition of Done	المستخدم يسجل ويدخل ويخرج، والـ Roles محمية على السيرفر.

C. Vendor Management (Application, Approval, Dashboard, Store 7.3 Profile, Order Items, Summaries)

البند	التفاصيل
Purpose (الغرض)	فتح متجر جديد بموافقة ال Admin وإدارته.
User Stories	US-03, US-04, US-13, US-16, US-17.
Dependencies	Identity, VendorProfile, Orders/OrderItems/CommissionRecords (لشاشات ال Dashboard).
Database Changes	VendorProfiles + Unique على UserId و StoreName.
Backend Tasks	VendorService (Apply, Approve, Reject, Suspend), ICurrentVendorService, Policy Aggregates (SUM(LineTotal), SUM(CommissionAmount)) ب ApprovedVendor, Dashboard مفلتر بال Vendor.
Frontend Tasks	Vendor Application, صفحة حالة الطلب (Pending/Rejected + السبب), Vendor Dashboard (Cards + جدول آخر الطلبات), تعديل Store Profile.
Authorization Rules	Vendor Area: Policy ApprovedVendor. Admin Area: SuperAdminOnly.
Validation	StoreName فريد, سبب الرفض إجباري, الانتقالات: Pending→Approved/Rejected, Approved→Suspended.
Error Scenarios	تقديم مرتين, Approve لطلب اترجع, Vendor Suspended بيجاول الدخول.
Testing Requirements	Vendor A/B Authorization Tests (غير معتمد), Unit ل State Transitions.
Definition of Done	الدورة الكاملة (Apply → Approve → Dashboard) شغالة, والعزل مختبر.

D. Product Management (Create, Edit, Images, Categories, 7.4 Price/Stock Validation, Active/Inactive)

البند	التفاصيل
Purpose (الغرض)	ال Vendor يدير كتالوج منتجاته.
User Stories	US-14, US-15.
Dependencies	Vendor Approval + Categories.
Database Changes	Products, ProductImages + Check Constraints على Price و StockQuantity.
Backend Tasks	ProductService (Create/Update/Deactivate/Adjust Stock) بتفلتر دائماً ب VendorProfileId. FileStorageService يحفظ الصور بأسماء GUID, ويتأكد من الامتداد (jpg/png/webp), والحجم, وال Content Type.
Frontend Tasks	Index/Create/Edit ب ViewModels, رفع صور متعددة مع معاينة, زر Activate/Deactivate.
Authorization Rules	ApprovedVendor + ملكية المنتج في كل Query.
Validation	Category, Price > 0, Stock ≥ 0, (مثلاً 2MB), عدد الصور (≥ 5).
Error Scenarios	منتج Vendor تاني (404), ملف غير صورة, Overposting, تعارض Concurrency على RowVersion.
Testing Requirements	Unit ل Validation, Integration ل Authorization, Create/Edit, (A يعدل منتج B).
Definition of Done	CRUD كامل ومحمي, الصور آمنة, المنتجات المرتبطة بطلبات بتتعطل مش بتتحذف.

E. Shopping Cart (Add, Remove, Update, Stock Validation, Server- 7.5 Side Price, Unavailable Items)

البند	التفاصيل
Purpose (الغرض)	تجميع المشتريات من كذا Vendor في مكان واحد.
User Stories	US-06, US-07.
Dependencies	Products + Authentication.
Database Changes	CartItems + Unique (CustomerId, ProductId).
Backend Tasks	CartService (Add/Update/Remove/Get/Clear), GetCartAsync يعيد حساب الأسعار من Products ويعلم العناصر غير المتاحة (IsAvailable=false) مع رسالة.
Frontend Tasks	صفحة Cart بجدول، تعديل كمية (Input + زر)، عداد في الـ Navbar، تحذير للعناصر غير المتاحة.
Authorization Rules	[Authorize(Roles=Customer)] والـ Cart الخاص بالمستخدم الحالي فقط (الـ CustomerId من الـ Claims).
Validation	Quantity من 1 للمخزون، productId موجود.
Error Scenarios	منتج اتعطل بعد الإضافة، المخزون قَلَّ، سعر اغير، منتج اتحذف.
Testing Requirements	Unit: إضافة، تكرار، تخطي المخزون، تلاعب بالسعر (مفیش سعر في الـ Request).
Definition of Done	الحسابات من الـ Database دائماً، والحالات الشاذة متعاملة.

F. Orders and Checkout (Validation, Order Creation, OrderItems, 7.6 Statuses, History, Vendor Views, Cancellation)

البند	التفاصيل
Purpose (الغرض)	تحويل الـ Cart لطلب مع حجز المخزون.
User Stories	US-08, US-10, US-11, US-16.
Dependencies	Cart + Products + Payment (الفصل 8).
Database Changes	Orders, OrderItems, Payments + Check Constraint على LineTotal.
Backend Tasks	CheckoutService.CreateOrderAndSessionAsync داخل Transaction: التحقق، إنشاء Order/Items بـ ExecuteUpdateAsync، Snapshots، شرطی لخصم المخزون (WHERE StockQuantity >= qty)، ومسح الـ Cart بعد نجاح الدفع. OrderService لتاريخ العميل ولشاشة الـ Vendor ولتحديث الـ Fulfillment.
Frontend Tasks	صفحة Shipping، مراجعة الطلب، تاريخ الطلبات، تفاصيل طلب مجمعة بالـ Vendor، شاشة Vendor Orders، بآزرار Shipped/Delivered.
Authorization Rules	العميل يشوف طلباته فقط (CustomerId == currentUser)، الـ Vendor يشوف OrderItems بتاعته فقط.
Validation	عنوان الشحن إجباري، الانتقالات المسموحة (BR-10)، الإلغاء فقط لـ PendingPayment.
Error Scenarios	مخزون ناقص أثناء الحجز (Rollback)، Order ID لشخص ثاني (404)، تحويل حالة غير مسموح.
Testing Requirements	Integration للـ Transaction (فشل جزئي)، Order Status Transitions، سياق شراء آخر قطعة.
Definition of Done	مفیش Overselling، الطلب مقسم بالـ Vendors، الحالات مضبوطة.

G. Payment Processing (Stripe Test Mode, Webhooks, Idempotency, 7.7 Failed Payments)

البند	التفاصيل
Purpose (الغرض)	استقبال المدفوعات بأمان وتأكيد الطلب.
User Stories	US-09.
Dependencies	Orders + Stripe Account (Test Mode) + Stripe CLI.
Database Changes	Payments, StripeWebhookEvent.
Backend Tasks	StripePaymentService (إنشاء Checkout Session)، StripeWebhookController (قراءة ال raw body)، StripeWebhookService (Signature + Idempotency + Transaction)، معالجة الأحداث الأربعة (completed, async succeeded, async failed, expired).
Frontend Tasks	صفحتنا Success و Cancel (بيعرضوا الحالة فقط)، زرار Retry Payment لـ PendingPayment Order لسه صالح.
Authorization Rules	ال Webhook Endpoint عام ([AllowAnonymous]) ومحميه بالتوقيع فقط، وبدون AntiForgery ([IgnoreAntiforgeryToken]).
Validation	التوقيع إجباري، payment_status == paid، مطابقة amount_total و currency مع ال Order.
Error Scenarios	توقيع خاطئ (400)، Event مكرر (200)، Event متأخر بعد انتهاء ال Stripe Session، بطيء.
Testing Requirements	Stripe CLI (stripe listen, stripe Duplicate Webhook، Failed Payment، Signature .trigger).
Definition of Done	الدفع ينجح فقط من ال Webhook، التكرار مبيكررش آثار، المفاتيح خارج ال Repo.

H. Commission System (Settings, Calculation, Net Amount, Records, 7.8 Snapshots, Reporting)

البند	التفاصيل
Purpose (الغرض)	حساب عمولة المنصة وتتبعها.
User Stories	US-17, US-19.
Dependencies	Payment Webhook (لإنشاء السجلات).
Database Changes	CommissionSettings, CommissionRecords + Filtered Unique على الصف النشط.
Backend Tasks	CommissionService.CreateForOrderAsync داخل Transaction ال Webhook: لكل Item يحسب Net = Gross - Commission = Round(LineTotal × Rate / 100, 2, AwayFromZero) و Commission. SetRateAsync لل Admin: يطفي الصف القديم ويضيف جديد في Transaction. استعلامات تقارير مجمعة.
Frontend Tasks	فورم تغيير النسبة مع عرض السجل التاريخي، جدول عمولات لل Vendor، ملخص لل Admin.
Authorization Rules	تغيير النسبة: SuperAdminOnly. عرض العمولات: ال Vendor يشوف بتاعته فقط.
Validation	النسبة 0-100، منع تغييرها بأثر رجعي، Unique على OrderItemId.
Error Scenarios	عدم وجود Setting نشط (يفشل الدفع ويسجل Log ويظهر Alert لل Admin)، تقريب خاطئ.
Testing Requirements	Unit: حسابات بأرقام محددة، تغيير النسبة لا يغير القديم.
Definition of Done	الحسابات صحيحة ومسجلة ك Snapshot والتقارير مطابقة للمجاميع.

I. Super Admin Dashboard (Approvals, Categories, Orders, 7.9 Commission, Analytics, Moderation)

البند	التفاصيل
Purpose (الغرض)	إدارة المنصة بالكامل.
User Stories	US-04, US-18, US-19, US-20.
Dependencies	كل ال Features السابقة.
Database Changes	لا جداول جديدة (Aggregates).
Backend Tasks	AdminDashboardService (عدد Vendors/Orders /ت)، CRUD لـ Categories، قائمة Orders بفلتر و Deactivate، Pagination، لمنتج مخالف.
Frontend Tasks	Dashboard بـ Cards، جداول بـ Pagination، فورم Categories و Commission.
Authorization Rules	SuperAdminOnly على ال Area كلها.
Validation	اسم Category فريد، منع تعطيل Category فيها منتجات نشطة بدون تأكيد.
Error Scenarios	محاولة وصول (403) Customer/Vendor، تعديل Category مكرر.
Testing Requirements	Authorization على كل Integration، Action، للأرقام.
Definition of Done	الأرقام صحيحة، كل الشاشات محمية.

8. معمارية Stripe والنظام المالي (Stripe and Financial Architecture)

8.1 أربع حاجات لازم نفرّق بينهم

#	الموضوع	مين بيعمله؟	في الـ MVP؟
1	استقبال الدفع من العميل	Stripe (Checkout) بيستقبل ويخصم من الكارت	نعم (Test Mode)
2	تسجيل العمولة	تطبيقنا في جدول CommissionRecords	نعم
3	حساب صافي الـ Vendor (VendorNetAmount)	تطبيقنا (حساب محاسبي)	نعم
4	تحويل الفلوس فعلياً للـ Vendors	Stripe Connect (Transfers/Payouts)	لا: مرحلة مستقبلية منفصلة

تنبيه مهم: Stripe Checkout العادي مبيوزعش الفلوس تلقائياً على أكثر من Vendor. الفلوس كلها بتروح لحساب Stripe بتاع المنصة. الأرقام اللي بنحسبها (العمولة وصافي الـ Vendor) هي سجلات محاسبية في الـ Database بتوضح المستحق لكل Vendor، لكن مفيش تحويل حقيقي في الـ MVP.

8.2 له Stripe Checkout (Hosted) بدل بناء فورم كارت؟

- بيانات الكارت بتتكتب على صفحة Stripe، مبتعديش على سيرفرنا، فبنقل عبء الـ PCI Compliance.
- نقدر نتجاهل UI الكارت ونركز على الـ Business Logic.
- Stripe بيدير الـ 3D Secure وطرق الدفع المتأخرة (ولهذا بنتعامل مع **async_payment_succeeded**).
- خيار **PaymentIntent** مع Stripe Elements أقوى في التخصيص لكنه أعقد في الـ Frontend، وده مش مناسب للـ MVP [Recommendation].

8.3 الـ Workflow الكامل (11 خطوة)

مطابق للـ 14 Diagrams و 15 و 17:

1. **Customer** يضغط Checkout ويدخل بيانات الشحن.
2. **السيرفر يتحقق** من الأسعار والمخزون: يعيد تحميل كل منتج من SQL Server ويتأكد إنه **Active** وإن الـ Vendor **Approved** وإن المخزون كافي.
3. **السيرفر ينشئ Order** بحالة **Payment** + **OrderItems** (Snapshots) + **PendingPayment** بحالة **Pending**. ويحجز المخزون، كل ده في **Transaction** واحدة.
4. **السيرفر ينادي Stripe** لإنشاء Checkout Session فيها **line_items** من الـ Order (مش من الـ Cart) و **metadata.orderId** و **expires_at**.
5. العميل يكمل الدفع في صفحة Stripe بكارت تجريبي.
6. **Stripe** بيعت (**checkout.session.completed**) **Webhook**.
7. التطبيق يتحقق من التوقيع بالـ raw body وسر الـ Endpoint.
8. معالجة **Idempotent**: تسجيل **StripeEventId** في نفس الـ Transaction (Unique).
9. تحديث آمن: **Payment = Succeeded** و **Order = Paid** (بعد التأكد إن المبلغ والعملة مطابقين).
10. **المخزون والعمولة**: المخزون اتحجز بالفعل في الخطوة 3 فمفيش خصم هنا؛ بننشئ **CommissionRecords** داخل نفس الـ Transaction. وفي حالة الفشل/الانتهاء بنرجع المخزون.
11. العميل والـ Vendor يشوفوا الطلب: العميل في **Orders/**، والـ Vendor في **Vendor/Orders/** (Items بتاعته فقط).

استراتيجية المخزون والـ [Recommendation] Transaction

الخيار	الميزة	العيوب
Order Reserve (A) عند إنشاء الـ Order (المختار)	مفیش Overselling. والـ Webhook بسيط	لازم نرجع المخزون لو الدفع ما تمش
(B) خصم بعد الدفع	مفیش حجز لفلوس لسه ما اتدفعش	ممکن اتنين يدفعوا لآخر قطعة والتاني يتدفع له ويتعذر تسليمه

المخزون المحجوز بيرجع في 3 حالات: **checkout.session.expired**, **async_payment_failed**, أو إلغاء العميل. ولو Webhook الـ expired اتأخر أو ضاع، **Background Job** بسيط (**BackgroundService**) كل 10 دقائق يلغي الـ Orders اللي **PendingPayment** من أكثر من (مدة الـ Session + هامش) ويرجع مخزونها، كشبكة أمان [Recommendation].

حالة إلغاء العميل (UC-11): لو العميل ألغى Order لسه **PendingPayment** وفيه Checkout Session مفتوحة، لازم ننهي الـ Session من ناحية **Stripe** أولاً (Expire Session API) وبعدين نلغي الـ Order ونرجع المخزون، وإلا ممكن العميل يدفع بعد الإلغاء. لو الـ Session اتدفعت فعلاً وقت الإلغاء، الـ Webhook هو اللي يحسم الموقف. [Team Decision]: تأكدوا من اسم الـ Method في Stripe.net وسلوكها من الـ [API Reference].

8.4 الكود الأساسي

[Team Decision]: أسماء Classes والـ Properties في **Stripe.net** بتتغير بين الإصدارات. الأمثلة دي للتوضيح، راجعوا الـ **API Reference** الرسمي للنسخة المثبتة قبل الاعتماد عليها.

حجز المخزون بشكل آمن (Conditional UPDATE)

```
foreach (var line in orderLines)
{
    var rows = await _db.Products
        .Where(p => p.Id == line.ProductId && p.IsActive && p.StockQuantity >= line.Quantity)
        .ExecuteUpdateAsync(s => s.SetProperty(p => p.StockQuantity, p => p.StockQuantity -
line.Quantity));

    if (rows == 0)
    {
        await tx.RollbackAsync();
        return Result.Fail($"Insufficient stock for {line.Name}.");
    }
}
```

الـ **UPDATE ... WHERE StockQuantity >= qty** عملية واحدة ذرية (Atomic) في SQL Server، فلو اتنين اشتروا آخر قطعة في نفس اللحظة، واحد بس هيحصل عنده **rows = 1**.

إنشاء Checkout Session

```
var options = new SessionCreateOptions
{
    Mode = "payment",
    ClientReferenceId = order.Id.ToString(),
    SuccessUrl = $"{baseUrl}/Checkout/Success?orderId={order.Id}",
    CancelUrl = $"{baseUrl}/Checkout/Cancel?orderId={order.Id}",
    ExpiresAt = DateTime.UtcNow.AddMinutes(30),
    Metadata = new Dictionary<string, string> { ["orderId"] = order.Id.ToString() },
    LineItems = order.Items.Select(i => new SessionLineItemOptions
    {
        Quantity = i.Quantity,
        PriceData = new SessionLineItemPriceDataOptions
        {
            Currency = order.Currency.ToLowerInvariant(),
            UnitAmount = (long)Math.Round(i.UnitPrice * 100m), // Stripe uses the smallest
currency unit
            ProductData = new SessionLineItemPriceDataProductDataOptions { Name =
i.ProductNameSnapshot }
        }
    }).ToList()
```

```
};
var session = await new SessionService().CreateAsync(options);
payment.StripeCheckoutSessionId = session.Id;
await _db.SaveChangesAsync();
return session.Url;
```

ال `line_items` مبنية من `OrderItems` التي اتحسبت من ال `Database`، مش من ال `Cart` ولا من ال `Browser`.

Webhook Controller

```
[AllowAnonymous]
[IgnoreAntiforgeryToken]
[Route("stripe/webhook")]
public class StripeWebhookController : Controller
{
    private readonly IStripeWebhookService _service;
    private readonly ILogger<StripeWebhookController> _logger;
    public StripeWebhookController(IStripeWebhookService service,
    ILogger<StripeWebhookController> logger)
        => (_service, _logger) = (service, logger);

    [HttpPost]
    public async Task<IActionResult> Post()
    {
        // The raw body is REQUIRED: signature verification fails if the payload is parsed and
        re-serialized.
        var json = await new StreamReader(Request.Body).ReadToEndAsync();
        var signature = Request.Headers["Stripe-Signature"].ToString();
        try
        {
            await _service.HandleAsync(json, signature);
            return Ok();
        }
        catch (StripeException ex) // includes signature verification failures
        {
            _logger.LogWarning(ex, "Stripe webhook rejected.");
            return BadRequest();
        }
    }
}
```

Webhook Service (Signature + Idempotency + Transaction)

```
public async Task HandleAsync(string json, string signature)
{
    var stripeEvent = EventUtility.ConstructEvent(json, signature, _options.WebhookSecret); //
    throws if invalid

    await using var tx = await _db.Database.BeginTransactionAsync();

    // Idempotency: the UNIQUE index on StripeEventId rejects a second insert.
    _db.StripeWebhookEvents.Add(new StripeWebhookEvent { StripeEventId = stripeEvent.Id,
    EventType = stripeEvent.Type });
    try { await _db.SaveChangesAsync(); }
    catch (DbUpdateException) // duplicate event (already processed)
    {
        _logger.LogInformation("Duplicate Stripe event {EventId} ignored.", stripeEvent.Id);
        return; // respond 200 with no side effects
    }

    var session = stripeEvent.Data.Object as Session;
    switch (stripeEvent.Type)
    {
        case "checkout.session.completed" when session?.PaymentStatus == "paid":
        case "checkout.session.async_payment_succeeded":
            await MarkPaidAsync(session!); // Payment, Order, CommissionRecords
            break;
        case "checkout.session.expired":
        case "checkout.session.async_payment_failed":
            await CancelAndReleaseStockAsync(session!);
            break;
    }
    await _db.SaveChangesAsync();
    await tx.CommitAsync();
}
```

الـ **catch (DbUpdateException)** هنا واسع للتوضيح. في الكود الحقيقي افحصوا إن السبب هو **Unique Constraint Violation** (SQL Server Error 2601/2627) علشان منبلعش أخطاء ثانية.

داخل **MarkPaidAsync**: نحمل الـ **Payment** بـ **StripeCheckoutSessionId**، ولو الـ **Order Paid** بالفعل نخرج، ونتأكد إن **session.AmountTotal** يساوي **Payment.Amount × 100** وإن العملة مطابقة، ثم نحدّث الحالات وننادي **CommissionService.CreateForOrderAsync**.

مثال حساب العمولة

بفرض النسبة 10%، والـ **Order** فيه **Item** من **Vendor X** (قيمة 200.00) و**Item** من **Vendor Y** (قيمة 55.55):

Vendor Net	Commission	Rate	Gross	Item
180.00	20.00	10.00%	200.00	Vendor X
49.99	5.56 (5.555 تتقرب لأعلى بـ AwayFromZero)	10.00%	55.55	Vendor Y
229.99	25.56		255.55	الإجمالي

8.5 حالات الفشل ومخاطر الاتساق (Consistency Risks)

الحالة	الخطر	المعالجة
العميل دفع وقفل المتصفح قبل الـ Redirect	الـ Order يفضل PendingPayment	الـ Webhook هو المصدر الحقيقي، فالـ Order بيتحدث برضه
الـ Redirect رجع Success لكن الدفع لسه متأكدش	لو اعتمدنا عليه نبيع من غير تأكيد	Success تعرض "قيد التأكيد" فقط
Webhook اتبعث مرتين	عمولات مكررة، مخزون مكرر	Unique StripeWebhookEvent + Order.Status فحص
Webhook متأخر أو الـ Events وصلت بترتيب مختلف	تحديث على حالة قديمة	نفحص الحالة الحالية قبل أي انتقال
السيرفر وقع بين تحديث Order و Payment	حالة نص-نص	كل حاجة داخل Transaction واحدة؛ لو فشل Stripe هيعيد الإرسال
فشل الاتصال بـ Stripe بعد Commit الـ Order	Order معلق ومخزون محجوز	نحاول إلغائه فوراً، وإلا الـ Background Job ينظفه
مبلغ الـ Session ≠ مبلغ الـ Order	تلاعب أو Bug	نرفض التأكيد ونسجل Alert
فشل الكارت	العميل يعيد المحاولة	Checkout بيسمح بإعادة المحاولة داخل نفس الـ Session

8.6 قواعد الأمان في الدفع

- ممنوع نخزن رقم كارت أو **CVC** في أي مكان (**Database, Logs**).
- ممنوع نرفع **SecretKey** أو **WebhookSecret** على **GitHub**. نستخدم **dotnet user-secrets** محلياً و **Environment Variables** على الاستضافة. لو مفتاح اتسرّب، نغيّره (Roll) من **Stripe Dashboard** فوراً.
- الـ **Webhook Endpoint** لازم يكون **HTTPS** في **Production**.
- نستخدم **Test Mode** فقط في الـ **MVP**. كروت الاختبار موجودة في صفحة **Stripe Testing** الرسمية.

8.7 تجربة الـ Webhooks محلياً

```
stripe login
stripe listen --forward-to https://localhost:7001/stripe/webhook
# The CLI prints a whsec_... secret: put it in user-secrets as Stripe:WebhookSecret
stripe trigger checkout.session.completed
```

ملحوظة: ال `whsec` الذي يبطل من `stripe listen` مختلف عن سر ال Endpoint المسجل في ال Dashboard. استخدموا كل واحد في بيئته.

Stripe Connect 8.8 كمرحلة مستقبلية

إيه هو **Stripe Connect**؟ منتج من Stripe يسمح للمنصات (زينا) إنها تستقبل مدفوعات وتوزعها على أطراف ثانية (Vendors) بشكل مرخص ومتوافق مع القوانين.

الموضوع	الشرح
Connected Accounts	كل Vendor ليه حساب Stripe مرتبط بحساب المنصة.
Vendor Onboarding	ال Vendor بيعمل Onboarding (هوية، حساب بنكي) عن طريق صفحات Stripe المستضافة، والمنصة بتتابع حالة الحساب (هل يقدر يستقبل Payouts).
Platform و Transfers Fees	المنصة ممكن تحول جزء من الفلوس لكل Connected Account وتخصم Application Fee (عمولتنا). التفاصيل بتختلف حسب نوع ال Charge (Destination / Separate Charges and Transfers).
Country eligibility	Connect متاح في دول معينة، وحساب المنصة وحسابات ال Vendors لازم يكونوا في دول مدعومة.
Compliance	KYC، شروط الخدمة، مسؤولية Refunds وال Disputes، وربما التزامات ضريبية وقانونية.
Testing إضافي	سيناريوهات Onboarding ناقص، Payouts معلقة، Refunds، و Webhooks خاصة ب Connect.
أثر على التصميم	منحتاج StripeAccountId في VendorProfile، وربما VendorOrder و Payout. وربط CommissionRecord ب Transfer.

تحذير: قبل ما الفريق يخطط لدفعات حقيقية لازم يتحقق من وثائق **Stripe الحالية (Connect)** ومن الدول المدعومة لحساب المنصة ولك Vendors، لأن الإتاحة والشروط بتتغير. وفر وقت لكتابة Use Case منفصل ومراجعة قانونية قبل الإنتاج.

9. خارطة الطريق التفصيلية (Development Roadmap)

9.1 ترتيب التنفيذ ولماذا نبني بشكل تدريجي (Incremental)

الخطأ الشائع في المشاريع الطلابية إن الفريق يبني كل ال Database أولاً، وبعدئذ كل ال Backend، وبعدئذ كل ال Frontend، وفي الآخر يحاول يوصلهم. النتيجة: التكامل بيتأجل لآخر المشروع وتظهر مشاكل كبيرة متأخر، ومحدث بيشفو حاجة شغالة لأسابيع.

إحنا هنبني **Vertical Slices**: كل Sprint بيطلع Feature صغيرة شغالة من أول ال Database لحد ال View (مثلاً: Vendor يضيف منتج ويظهر في ال Storefront). المزايا:

- ال Feature بتتجرب فعلياً في نفس ال Sprint، فالأخطاء بتظهر بدري.
- كل عضو بيتعلم كل الطبقات بدل ما يتخصص في طبقة واحدة.
- بنقدر نعرض Demo في آخر كل Sprint.
- لو الوقت خلص، عندنا نسخة شغالة (MVP) مش 70% من كل حاجة.

الترتيب اتحدد بال Stripe + Orders ← Cart ← Products ← Vendor Approval ← Identity Dependencies: Commission ← Dashboards ← Testing/Security ← Deployment

9.2 ربط ال Phases بال Sprints

النتائج الرئيسية	Phases	Sprint
Requirements, Diagrams, ERD, Repo جاهز	1-7	Sprint 0
مشروع شغال، Database, Identity	8-10	Sprint 1
Vendor Approval	11	Sprint 2
Storefront + Vendor Products	12-13	Sprint 3
Shopping Cart	14	Sprint 4
Checkout + Orders	15	Sprint 5
Stripe + Commission	16-17	Sprint 6
Customer/Vendor/Admin Dashboards	18-20	Sprint 7
Testing + Security + Integration	21-23	Sprint 8
Deployment + Docs + Presentation	24-26	Sprint 9

ملاحظة: ال Phase 17 (Commission) بيتنفذ مع Phase 16 (Stripe) في Sprint 6 لأن العمليات بتنشئ داخل Transaction ال Webhook، فالأثنين متشابكين.

9.3 تفاصيل كل Phase

Phase 1: Requirements Gathering (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نجمع ونفهم احتياجات المستخدمين والمشروع.
Why it matters (ليه مهم؟)	من غير فهم مشترك هنبني حاجات مختلفة.
Prerequisites	ال Project Brief (الوثيقة الأصلية).
Detailed Tasks	جلسة ورشة (Workshop) لكتابة قائمة ال Actors والمشاكل وال Features، وتحديد ال MVP.

البند	التفاصيل
Expected Deliverables	قائمة Requirements أولية + أسئلة مفتوحة.
Team Collaboration	كل الفريق يحضر ويكتب: كل عضو يعرض فهمه للمشروع في دقيقتين.
Testing Criteria	كل عضو يقدر بشرح المشروع بنفس المعنى.
Common Mistakes	الدخول في الحلول التقنية قبل فهم المشكلة.
Definition of Done	اتفاق الفريق على قائمة ال Requirements وال MVP.
Gate (شرط الانتقال)	لا نبدأ التحليل إلا بعد موافقة الخمسة على القائمة.

Phase 2: Business Analysis (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نكتب Scope و FR و NFR و Business Rules و Risks.
Why it matters (ليه مهم؟)	ال Business Rules هي اللي بتحدد منطق ال Services بعدين.
Prerequisites	Phase 1.
Detailed Tasks	نراجع الفصل 2 ونعدله بما يناسب الفريق، ونحدد Out-of-Scope.
Expected Deliverables	وثيقة BA معتمدة (الفصل 2).
Team Collaboration	كل عضو يراجع قسم ويعرضه، ونصوت على ال MVP.
Testing Criteria	كل Rule قابلة للاختبار ولها رقم (BR-xx).
Common Mistakes	كتابة Rules غامضة مثل 'يجب أن يكون سريعاً'.
Definition of Done	FR/NFR/BR واضحة وبأرقام.
Gate (شرط الانتقال)	مفيش Feature في ال Backlog بدون Requirement.

Phase 3: User Stories and Acceptance Criteria (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نحول ال Requirements لـ Stories صغيرة قابلة للاختبار.
Why it matters (ليه مهم؟)	ال Acceptance Criteria هي أساس الاختبار وتعريف 'خلصنا'.
Prerequisites	Phase 2.
Detailed Tasks	نكتب Stories بصيغة As a/I want/So that ونضيف Acceptance Criteria ونحطها كـ GitHub Issues.
Expected Deliverables	Backlog في GitHub Project Board.
Team Collaboration	كل عضو يكتب 4 Stories ويراجعها عضو تاني.
Testing Criteria	كل Story لها 3 Acceptance Criteria على الأقل، ومفيش Story أكبر من Sprint.
Common Mistakes	Stories ضخمة (Epics) من غير تقسيم.
Definition of Done	Backlog مرتب بالأولوية ومرتببط بال Milestones.
Gate (شرط الانتقال)	كل Story لها Acceptance Criteria مراجعة.

Phase 4: Use Case Modeling (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نرسم ال Actors وال Use Cases ونحدد العلاقات.

البند	التفاصيل
Why it matters (ليه مهم؟)	بتكشف حالات منسية (مثل Vendor Suspended).
Prerequisites	Phase 3.
Detailed Tasks	نكتب UC-01...31، ونرسم Diagrams 2-5 من ال Source.
Expected Deliverables	Diagrams 2-5 + جدول Use Cases.
Team Collaboration	Whiteboard جماعي، وواحد يرسم والباقي يراجع (بيتبدلوا).
Testing Criteria	كل Use Case مربوط ب Story.
Common Mistakes	رسم Use Cases كلها كأنها Features تقنية.
Definition of Done	ال Diagrams معتمدة ومتسقة مع ال Stories.
Gate (شرط الانتقال)	كل UC له Actor واضح و Pre/Post Conditions.

Phase 5: ERD and Database Design (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نصمم الجداول والعلاقات والقيود.
Why it matters (ليه مهم؟)	تغيير ال Schema بعد البرمجة مكلف جداً.
Prerequisites	Phases 2-4.
Detailed Tasks	نراجع الفصل 5 سطر بسطر، نتأكد من ال Cardinality وال Delete Behavior، ونعدل Diagrams 6A-6C.
Expected Deliverables	ERD + Data Dictionary معتمدين.
Team Collaboration	Design Session مشتركة؛ كل عضو يشرح جدولين.
Testing Criteria	كل علاقة لها FK وسلوك حذف، ولا فيه Multiple Cascade Paths.
Common Mistakes	استخدام float للفلوس أو نسيان ال Snapshots.
Definition of Done	ERD متوافق مع Class Diagram وال Stories.
Gate (شرط الانتقال)	كل Entity لها Config مخطط وكل عضو شرح جدول.

Phase 6: Architecture Design (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نحدد الطبقات وهيكل الفولدرات وال Dependencies.
Why it matters (ليه مهم؟)	بيمنع الفوضى لما 5 أشخاص يكتبوا Code.
Prerequisites	Phase 5.
Detailed Tasks	نراجع الفصل 6، نقرر Areas و Services، ونكتب CONTRIBUTING.md فيه قواعد التسمية.
Expected Deliverables	Diagrams 7-10 + هيكل فولدرات + Coding Conventions.
Team Collaboration	كل عضو يشرح طبقة ويجاوب أسئلة الفريق.
Testing Criteria	ال Diagrams متسقة مع الفولدرات.
Common Mistakes	Over-engineering (Repositories و Microservices).
Definition of Done	اتفاق على الهيكل وال Conventions.
Gate (شرط الانتقال)	كل عضو يقدر يقول 'ال Code ده يروح فين؟'.

Phase 7: GitHub Setup (Sprint 0)

البند	التفاصيل
Objective (الهدف)	نجهز ال Repository وقواعد العمل.
Why it matters (ليه مهم؟)	الأساس لأي تعاون ناجح.
Prerequisites	.Phase 6
Detailed Tasks	إنشاء Repo, Branch Protection على main, Issue/PR Templates, Labels, Milestones, Project Board, .gitignore, CI بسيط.
Expected Deliverables	Repository جاهز + CI شغال.
Team Collaboration	كل عضو يعمل Clone ويفتح PR تجريبي ويراجع PR زميله.
Testing Criteria	PR تجريبي اتعمله Merge بعد Review و CI أخضر.
Common Mistakes	Push مباشر على main أو رفع Secrets.
Definition of Done	main محمي وال CI شغال.
Gate (شرط الانتقال)	الخمس عملا PR واحد على الأقل.

Phase 8: ASP.NET Core MVC Setup (Sprint 1)

البند	التفاصيل
Objective (الهدف)	ننشئ ال Solution وال Projects وال Layout الأساسي.
Why it matters (ليه مهم؟)	كل باقي ال Features بتبني عليه.
Prerequisites	.NET 10 SDK + Phase 7
Detailed Tasks	إنشاء MarketHub.Web, DI, Areas, Bootstrap Layout, global.json, MarketHub.Tests, Skeleton, صفحة Home.
Expected Deliverables	مشروع بيشغل محلياً.
Team Collaboration	Pair Programming لأول Setup; باقي الأعضاء يكرروا على أجهزتهم.
Testing Criteria	dotnet build و dotnet test ينجحوا عند الخمسة.
Common Mistakes	كل واحد يستخدم SDK مختلف.
Definition of Done	المشروع يعمل عند الخمسة بنفس الخطوات.
Gate (شرط الانتقال)	README فيه خطوات التشغيل ومجربة.

Phase 9: SQL Server and EF Core Configuration (Sprint 1)

البند	التفاصيل
Objective (الهدف)	نربط ال DbContext بـ SQL Server ونعمل أول Migration.
Why it matters (ليه مهم؟)	كل البيانات هتتدي من هنا.
Prerequisites	.Phase 8
Detailed Tasks	ال Entities الأساسية, ApplicationDbContext, Configurations, أول Migration, DbSeeder, Connection String في user-secrets.
Expected Deliverables	Database بتتعمل بأمر واحد.
Team Collaboration	جلسة مشتركة لقواعد ال Migrations (الفصل 11).
Testing Criteria	dotnet ef database update ينجح عند الجميع.

البند	التفاصيل
Common Mistakes	Connection String مرفوع على GitHub.
Definition of Done	Schema مطابق للـ ERD.
Gate (شرط الانتقال)	Seed بيشتغل Idempotent.

Phase 10: Identity and Authorization (Sprint 1)

البند	التفاصيل
Objective (الهدف)	Register/Login/Logout والـ Roles والـ Policies.
Why it matters (ليه مهم؟)	الأمان لازم يتبني من البداية مش يتحط في الآخر.
Prerequisites	Phase 9.
Detailed Tasks	Admin, Policies, VendorBaseController , للـ ApplicationUser , Identity, Roles, Seeder Navbar حسب الـ Role.
Expected Deliverables	Auth كامل مع Tests للملاحيات.
Team Collaboration	كل عضو يكتب Authorization Test واحد.
Testing Criteria	403/401 صحيحين، والـ Role Claims شغالة.
Common Mistakes	الاعتماد على إخفاء الأزرار.
Definition of Done	كل Route محمي على السيرفر.
Gate (شرط الانتقال)	Test Cases 1 و 14 (الفصل 12) نجحوا.

Phase 11: Vendor Approval (Sprint 2)

البند	التفاصيل
Objective (الهدف)	Vendor Application ومراجعة الـ Admin.
Why it matters (ليه مهم؟)	أول Flow Multi-Role كامل.
Prerequisites	Phase 10.
Detailed Tasks	VendorProfile , فورم التقديم، قائمة SecurityStamp , Pending, Approve/Reject.
Expected Deliverables	Dashboard → Approve → Flow Apply فاضي.
Team Collaboration	Pair: واحد Vendor وواحد Admin أثناء الاختبار.
Testing Criteria	الانتقالات الصحيحة فقط.
Common Mistakes	إضافة Role عند التقديم بدل الموافقة.
Definition of Done	Test Case 4 (Unapproved vendor) نجح.
Gate (شرط الانتقال)	Vendor مرفوض ما يقدرش يوصل لـ Vendor .

Phase 12: Categories and Products (Sprint 3)

البند	التفاصيل
Objective (الهدف)	Seed Categories و Storefront للقراءة.
Why it matters (ليه مهم؟)	محتاجين كتالوج قبل أي حاجة تانية.
Prerequisites	Phase 11.

البند	التفاصيل
Detailed Tasks	Category CRUD لا Admin، Product Listing/Details/Search/Filter/Sort/Pagination للعامة.
Expected Deliverables	Seed Data بـعرض Storefront.
Team Collaboration	كل عضو يطور جزء (Search / Filter / Sort / Pagination) في Pair مع زميل.
Testing Criteria	Unit Tests لا Search والـ Pagination.
Common Mistakes	جلب كل المنتجات بدون Pagination.
Definition of Done	منتجات Inactive أو Vendor غير معتمد مخفية.
Gate (شرط الانتقال)	Performance مقبول مع 1000 منتج Seed.

Phase 13: Vendor Product Management (Sprint 3)

البند	التفاصيل
Objective (الهدف)	CRUD للمنتجات بصور وعزل كامل.
Why it matters (ليه مهم؟)	قلب الـ Marketplace وأول اختبار للـ Isolation.
Prerequisites	Phases 10-12.
Detailed Tasks	ProductService, FileStorageService, Create/Edit/Deactivate, Stock Adjust, Concurrency.
Expected Deliverables	Vendor يدير منتجاته.
Team Collaboration	Pair Programming + Cross Review، وكل عضو يحاول يكسر العزل.
Testing Criteria	Test Cases 2 و 5 و 13 نجحوا.
Common Mistakes	VendorProfileId في الـ ViewModel.
Definition of Done	Vendor A مستحيل يوصل لمنتج Vendor B.
Gate (شرط الانتقال)	رفع ملف خطر مرفوض.

Phase 14: Shopping Cart (Sprint 4)

البند	التفاصيل
Objective (الهدف)	إضافة/حذف/تعديل مع حسابات من الـ DB.
Why it matters (ليه مهم؟)	أول Flow فيه Business Rules تجارية.
Prerequisites	Phases 9, 12.
Detailed Tasks	CartService, CartController, Views, Navbar Counter، التعامل مع منتج غير متاح.
Expected Deliverables	Cart كامل.
Team Collaboration	Pair: واحد يكتب Test والثاني Code (TDD Ping-Pong).
Testing Criteria	Test Cases 6-8 نجحوا.
Common Mistakes	تخزين السعر في الـ Cart.
Definition of Done	السعر من الـ DB دائماً.
Gate (شرط الانتقال)	Cart بيعيد الحساب بعد تغيير السعر.

Phase 15: Order Management (Sprint 5)

البند	التفاصيل
Objective (الهدف)	Checkout وإنشاء Order وحجز المخزون وشاشات الطلبات.
Why it matters (ليه مهم؟)	المنطق المالي والاتساق.
Prerequisites	Phase 14.
Detailed Tasks	Transaction, Conditional UPDATE, Order History, Vendor Orders, CheckoutService, Fulfillment, Cancel.
Expected Deliverables	Orders تتعمل PendingPayment.
Team Collaboration	Design Session لـ Transaction, و Pair على الحجز, وكل الفريق يختبر سباق الشراء.
Testing Criteria	Test Cases 9-12 و 15 نجحوا.
Common Mistakes	خضم المخزون بـ Read-then-Write بدل Conditional UPDATE.
Definition of Done	مفیش Overselling.
Gate (شرط الانتقال)	الطلب يظهر مقسم بال Vendor.

Phase 16: Stripe Integration (Sprint 6)

البند	التفاصيل
Objective (الهدف)	Checkout Session و Webhook آمن.
Why it matters (ليه مهم؟)	أعقد وأخطر Feature.
Prerequisites	Phase 15 + Stripe Test Account.
Detailed Tasks	StripePaymentService, StripeWebhookController/Service, Idempotency, Retry, Payment, Background Cleanup.
Expected Deliverables	الدفع التجريبي شغال end-to-end.
Team Collaboration	كل الفريق يقرأ Stripe Docs المعتمدة ويشارك في Design Session, و Pair على الـ Webhook.
Testing Criteria	Test Cases 9-11 + stripe trigger.
Common Mistakes	Redirect = Paid, أو قراءة الـ Body بعد Model Binding.
Definition of Done	Order يتحول Paid من الـ Webhook فقط.
Gate (شرط الانتقال)	Webhook مكرر مبيغترش شيء.

Phase 17: Commission Calculation (Sprint 6)

البند	التفاصيل
Objective (الهدف)	حساب وتسجيل العمولة وإعداد النسبة.
Why it matters (ليه مهم؟)	القيمة التجارية للمنصة.
Prerequisites	Phase 16 (نفس الـ Sprint).
Detailed Tasks	CommissionSetting, CommissionService, دمج في Transaction الـ Webhook, تقارير.
Expected Deliverables	CommissionRecords صحيحة.
Team Collaboration	Pair على الـ Calculation مع جدول أمثلة محسوبة يدوياً.
Testing Criteria	Test Cases 11-12 نجحوا.
Common Mistakes	تغيير النسبة بأثر رجعي.

البند	التفاصيل
Definition of Done	كل Item مدفوع له سجل واحد.
Gate (شرط الانتقال)	التقريب والمجاميع مطابقة للجدول اليدوي.

Phase 18: Customer Dashboard (Sprint 7)

البند	التفاصيل
Objective (الهدف)	Profile و Order History.
Why it matters (ليه مهم؟)	تجربة العميل بعد الشراء.
Prerequisites	Phases 15-16.
Detailed Tasks	قائمة الطلبات، تفاصيل مقسمة بال (Vendor, Cancel, Review (Should).
Expected Deliverables	Customer Area.
Team Collaboration	Page لكل Cross-review.
Testing Criteria	العميل يشوف طلباته فقط.
Common Mistakes	عرض Orders بدون فلتر بال Customer.
Definition of Done	IDOR على Orders مرفوض.
Gate (شرط الانتقال)	Responsive.

Phase 19: Vendor Dashboard (Sprint 7)

البند	التفاصيل
Objective (الهدف)	ملخص المبيعات والعمولات وال Order Items.
Why it matters (ليه مهم؟)	ال Vendor محتاج يتابع فلوسه.
Prerequisites	Phases 15-17.
Detailed Tasks	Aggregates, Charts بسيطة (اختياري)، فلتر تاريخ.
Expected Deliverables	Dashboard كامل.
Team Collaboration	Pair: واحد Query والثاني View.
Testing Criteria	Vendor يشوف أرقامه فقط.
Common Mistakes	Aggregates من غير فلتر Vendor.
Definition of Done	الأرقام = مجاميع ال Database.
Gate (شرط الانتقال)	Test Case 3 نجاح.

Phase 20: Super Admin Dashboard (Sprint 7)

البند	التفاصيل
Objective (الهدف)	إحصائيات ومراقبة ال Orders و Commission و Moderation.
Why it matters (ليه مهم؟)	الرقابة على المنصة.
Prerequisites	Phases 11, 16-17.
Detailed Tasks	AdminDashboardService, Orders Monitoring, Commission Settings, Moderation.
Expected Deliverables	Admin Area.

البند	التفاصيل
Team Collaboration	Cross-review.
Testing Criteria	غير ال 403 = Admin.
Common Mistakes	Dashboard بدون Pagination.
Definition of Done	كل Action محمي.
Gate (شرط الانتقال)	الأرقام صحيحة.

Phase 21: Testing (Sprint 8)

البند	التفاصيل
Objective (الهدف)	اختبار شامل: Unit/Integration/Manual/Regression.
Why it matters (ليه مهم؟)	Features كثير هتتكسر بالتغييرات.
Prerequisites	كل ال Features.
Detailed Tasks	تنفيذ الفصل 12 كله، Test Matrix, Bug Triage, Regression Run.
Expected Deliverables	Test Report.
Team Collaboration	كل عضو ينفذ سيناريوهات عضو تاني (Fresh Eyes).
Testing Criteria	تغطية الحالات ال 15 الأساسية.
Common Mistakes	اختبار ال Happy Path فقط.
Definition of Done	لا Bugs Critical مفتوحة.
Gate (شرط الانتقال)	Test Report معتمد.

Phase 22: Security Review (Sprint 8)

البند	التفاصيل
Objective (الهدف)	مراجعة الثغرات حسب الفصل 13.
Why it matters (ليه مهم؟)	الحماية قبل العرض.
Prerequisites	Phase 21.
Detailed Tasks	Checklist OWASP-style، مراجعة Secrets، File Upload، CSRF، Headers، Logging.
Expected Deliverables	Security Checklist موقعة.
Team Collaboration	Peer Review: كل عضو يراجع مجال.
Testing Criteria	كل بند Pass أو له Ticket.
Common Mistakes	اعتبار HTTPS محلي كأنه Production.
Definition of Done	لا Secrets ولا IDOR.
Gate (شرط الانتقال)	Checklist مكتملة.

Phase 23: Integration (Sprint 8)

البند	التفاصيل
Objective (الهدف)	دمج نهائي وحل التعارضات وتنظيف.
Why it matters (ليه مهم؟)	نضمن إن المشروع بيشتغل ككل.

البند	التفاصيل
Prerequisites	Phases 21-22.
Detailed Tasks	Warnings, Final Migration Check حل ,تنظيف Code, Rebase/Merge, Full Regression
Expected Deliverables	Release Candidate على <code>main</code> .
Team Collaboration	Pair Debugging جماعي.
Testing Criteria	Build + Tests خضر وال Demo Flow كامل.
Common Mistakes	Merge في آخر لحظة بدون اختبار.
Definition of Done	RC موسوم بـ Tag.
Gate (شرط الانتقال)	<code>v1.0.0-rc</code> Tag

Phase 24: Deployment (Sprint 9)

البند	التفاصيل
Objective (الهدف)	نشر نسخة Demo.
Why it matters (ليه مهم؟)	العرض الحي.
Prerequisites	Phase 23 + قرار الاستضافة.
Detailed Tasks	اختيار ال Host, SQL Server, مستضاف, Env Variables, HTTPS, Webhook Endpoint, Migrations, Script.
Expected Deliverables	Demo Online.
Team Collaboration	كل عضو ينفذ خطوة (Rotation) Deployment.
Testing Criteria	Smoke Test على Production.
Common Mistakes	استخدام Secrets مرفوعة أو Test Keys بدون Webhook.
Definition of Done	Checkout تجريبي ناجح Online.
Gate (شرط الانتقال)	Rollback Plan مكتوب.

Phase 25: Documentation (Sprint 9)

البند	التفاصيل
Objective (الهدف)	README و Screenshots وتحديث ال Diagrams.
Why it matters (ليه مهم؟)	استمرارية المشروع.
Prerequisites	Phase 24.
Detailed Tasks	README, دليل التشغيل, Screenshots, Demo Accounts, تحديث ال Diagrams, ملاحظات التعلم.
Expected Deliverables	Docs مكتملة.
Team Collaboration	كل عضو يكتب قسم ويراجع قسم.
Testing Criteria	شخص جديد يشغل المشروع بال README فقط.
Common Mistakes	Docs قديمة مش متسقة مع ال Code.
Definition of Done	ال Docs مطابقة للنسخة الأخيرة.
Gate (شرط الانتقال)	جُرِّبَت على جهاز نظيف.

Phase 26: Final Presentation (Sprint 9)

البند	التفاصيل
Objective (الهدف)	إعداد العرض وال Demo.
Why it matters (ليه مهم؟)	نقدّم الشغل بشكل مقنع.
Prerequisites	Phase 25.
Detailed Tasks	نكتب السيناريو (الفصل 15)، نتدرب 3 مرات، نجهز Backup (فيديو).
Expected Deliverables	عرض + Demo.
Team Collaboration	كل عضو يقدم جزء ويجاوب أسئلة الأجزاء الثانية.
Testing Criteria	Dry Run ناجح بالوقت المحدد.
Common Mistakes	Demo حي بدون Plan B.
Definition of Done	كل عضو يقدر يشرح أي جزء.
Gate (شرط الانتقال)	Dry Run مسجل.

10. خطة التعاون للفريق (Five-Member Collaboration Plan)

10.1 الفلسفة: خمسة أشخاص، مشروع واحد مفهوم عند الكل

هدفنا مش بس نخلص المشروع، هدفنا إن كل واحد فينا يقدر يبني ويصلح ويشرح أي جزء في النظام. عشان كده مفيش 'مسؤول Backend' ثابت ولا 'مسؤول Frontend' ثابت. لو كل واحد اتخصص في طبقة، هتطلع النتيجة خمس موديولات منفصلة، وكل واحد مش فاهم باقي المشروع.

الممارسات اللي هنلتزم بيها

- **Shared Planning**: كل Sprint يبدأ بجلسة واحدة للخمسة، وكل واحد بيقول فهمه للـ Stories.
- **Small Cross-Functional Tasks**: كل Task صغيرة (يوم إلى يومين) ويتمس طبقة واحدة، ولكن بتتجمع في Vertical Slice واحد.
- **Pair Programming**: جلستين Pair في كل Sprint بتتبدل الثنائيات (Driver/Navigator بيتبدلوا كل 20-30 دقيقة).
- **Rotating Leadership**: القيادة (مش الملكية) بتدور كل Sprint بين 5 أدوار.
- **Cross-Member Code Review**: كل PR بيتراجع من عضو غير الكاتب.
- **Collective Testing**: كل عضو بيحرب Feature كتبها عضو تاني (Fresh Eyes).
- **Knowledge Sharing**: 30 دقيقة في آخر كل Sprint: كل عضو يشرح جزء كتبه زميله.
- **Retrospective, Sprint Review** في آخر كل Sprint.

10.2 الأدوار الدوارة (Rotating Leadership Roles)

الدور	المسؤولية خلال الـ Sprint
Facilitator	يدير الـ Planning والـ Review والـ Retro ويتابع الـ Board
Backend Lead	يقود قرارات الـ Database والـ Services في الـ Sprint
Frontend Lead	يقود قرارات الـ Views والـ Layout والـ UX
QA Lead	يقود خطة الاختبار واختبارات الـ Authorization والـ Regression
Integration and Docs Lead	يقود ترتيب الـ Merges والـ Migrations وتحديث الـ Docs والـ Diagrams

الـ Lead مش المالك. الـ Backend Lead مثلاً بيدبر النقاش ويقرر لو الفريق اختلف، لكن الـ Code الـ Backend بيكتبه ويراجعه أكثر من واحد، وأي حد ممكن يسأل ويعذل. القاعدة: لو عضو لوحده بي فهم جزء، ده Risk لازم نعالجه في نفس الـ Sprint.

10.3 جدول المسؤوليات الدوارة (Responsibility Matrix)

الجدول بيبيّن دور كل عضو في كل Sprint (Fac = Facilitator, BE = Backend Lead, FE = Frontend Lead, QA = QA, INT = Integration and Docs Lead). كل عضو بياخد كل دور مرتين على مدار الـ 10 Sprints.

العضو	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9
Member A	Fac	BE	FE	QA	INT	Fac	BE	FE	QA	INT
Member B	BE	FE	QA	INT	Fac	BE	FE	QA	INT	Fac
Member C	FE	QA	INT	Fac	BE	FE	QA	INT	Fac	BE
Member D	QA	INT	Fac	BE	FE	QA	INT	Fac	BE	FE
Member E	INT	Fac	BE	FE	QA	INT	Fac	BE	FE	QA

جدول الـ Pair Programming (Session 1 / Session 2)

في كل Sprint ينقسم الفريق مرتين: الـ Rover هو العضو الخامس اللي بينضم لثنائي ويشارك كـ Navigator تالت.

Sprint	Session 1 (النصف الأول)	Session 2 (النصف الثاني)
S0	A+B , C+D (Rover: E)	B+C , D+E (Rover: A)
S1	B+C , D+E (Rover: A)	C+D , E+A (Rover: B)
S2	C+D , E+A (Rover: B)	D+E , A+B (Rover: C)
S3	D+E , A+B (Rover: C)	E+A , B+C (Rover: D)
S4	E+A , B+C (Rover: D)	A+B , C+D (Rover: E)
S5	A+B , C+D (Rover: E)	B+C , D+E (Rover: A)
S6	B+C , D+E (Rover: A)	C+D , E+A (Rover: B)
S7	C+D , E+A (Rover: B)	D+E , A+B (Rover: C)
S8	D+E , A+B (Rover: C)	E+A , B+C (Rover: D)
S9	E+A , B+C (Rover: D)	A+B , C+D (Rover: E)

جدول الـ Code Review

كل صف بيوضح مين بيراجع PRs مين. الـ Reviewer الأول مطلوب لكل PR، والثاني مطلوب لـ PRs اللي بتمس الـ Database أو الـ Authorization.

Sprint	PR بتاع A	PR بتاع B	PR بتاع C	PR بتاع D	PR بتاع E
S0	B + C	C + D	D + E	E + A	A + B
S1	C + D	D + E	E + A	A + B	B + C
S2	D + E	E + A	A + B	B + C	C + D
S3	E + B	A + C	B + D	C + E	D + A
S4	B + C	C + D	D + E	E + A	A + B
S5	C + D	D + E	E + A	A + B	B + C
S6	D + E	E + A	A + B	B + C	C + D
S7	E + B	A + C	B + D	C + E	D + A
S8	B + C	C + D	D + E	E + A	A + B
S9	C + D	D + E	E + A	A + B	B + C

10.4 خطة كل Sprint بالتفصيل

المدة المقترحة 2 أسبوع لكل [Team Decision] Sprint. الـ Tasks الخمسة صغيرة ومتكاملة في نفس الـ Vertical Slice. توزيع الـ Tasks بيدور كل Sprint.

Sprint 0: Discovery and Design (Phases 1-7)

البند	التفاصيل
Sprint Goal 1.	وصول الفريق لفهم مشترك: Requirements معتمدة، ERD، Diagrams، و Repository جاهز.

البند	التفاصيل
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=Fac, B=BE, C=FE, D=QA, E=INT).
3. Shared Requirements and Design	Workshop للـ Requirements، كتابة User Stories، مراجعة الـ ERD سطر بسطر، رسم الـ Wireframes.
4. Small Tasks لكل عضو	Member A: تحديث الـ ERD والـ Data Dictionary Member B: كتابة Stories وـ Acceptance Criteria لـ Cart/Orders Member C: رسم Wireframes للـ Storefront وصفحات Vendor Member D: إعداد GitHub (Protection, Templates, CI) Member E: تجهيز مجلد docs وتحديث الـ Diagrams
5. Pair Programming	.Session 1: A+B , C+D (Rover E). Session 2: B+C , D+E (Rover A)
6. Code Review	B و A , PR E ← A و E , PR D ← E و D , PR C ← D و C , PR B ← C و PR A ← B
7. Integration Tasks	دمج الـ Docs والـ Diagrams عبر PR واحد مراجع من اثنين. (المسؤول: E ≤ INT Lead).
8. Testing Tasks	مراجعة قابلية اختبار كل Acceptance Criteria. الكل يختبر Feature زميله يدوياً، و (D) QA Lead يجمع النتائج.
9. Documentation Tasks	Docs v1 + CONTRIBUTING.md. (E يتأكد من التحديث).
10. Sprint Review	Demo: عرض الـ Diagrams والـ Backlog. بتقديم عضو مختلف لكل جزء (A يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 1: Foundation (Phases 8-10)

البند	التفاصيل
1. Sprint Goal	المشروع شغال عند الخمسة، Database و Identity جاهزين.
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=BE, B=FE, C=QA, D=INT, E=Fac).
3. Shared Requirements and Design	جلسة على هيكل الفولدرات وقواعد الـ Migrations وسياسة الـ Roles.
4. Small Tasks لكل عضو	Member A: Layout وـ Navbar حسب الـ Role Member B: CI مشروع الـ TestDbContext + Tests Member C: Authorization Tests + خطوات الـ README Member D: Entities والـ Configurations وأول Migration Member E: إعداد Identity والـ Seeder
5. Pair Programming	.Session 1: B+C , D+E (Rover A). Session 2: C+D , E+A (Rover B)
6. Code Review	C و B , PR E ← B و A , PR D ← A و E , PR C ← E و D , PR B ← D و PR A ← C
7. Integration Tasks	الـ Merge الـ Migration الأولى أولاً ثم باقي الفروع تعمل Rebase. (المسؤول: D ≤ INT Lead).
8. Testing Tasks	Test Cases 1 و 14. الكل يختبر Feature زميله يدوياً، و (C) QA Lead يجمع النتائج.
9. Documentation Tasks	README: خطوات التشغيل. (D يتأكد من التحديث).
10. Sprint Review	Demo: تسجيل دخول بـ 3 Roles. بتقديم عضو مختلف لكل جزء (E يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 2: Vendor Onboarding (Phases 11)

البند	التفاصيل
1. Sprint Goal	Vendor يقدم طلب والـ Admin يوافق أو يرفض.

البند	التفاصيل
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=FE, B=QA, C=INT, D=Fac, E=BE).
3. Shared Requirements and Design	مراجعة 11 Diagram وقواعد الانتقال بين الحالات.
4. Small Tasks لكل عضو	Member A: Tests: Vendor Migration, Member B: VendorProfile: Entity Member C: VendorService: Apply/Approve/Reject Pending فورم التقديم وقائمة Member D: VendorBaseController, Member E: Policy ApprovedVendor
5. Pair Programming	.Session 1: C+D , E+A (Rover B). Session 2: D+E , A+B (Rover C)
6. Code Review	D و C , PR E ← C و B , PR D ← B و A , PR C ← A و E , PR B ← E و PR A ← D
7. Integration Tasks	ترتيب: Entity ثم Service ثم Views. (المسؤول: C ≤ INT Lead).
8. Testing Tasks	Test Case 4. الكل يختبر Feature زميله يدوياً، و (B) QA Lead يجمع النتائج.
9. Documentation Tasks	تحديث 11 Diagram لو اتغير. (C يتأكد من التحديث).
10. Sprint Review	Demo: Apply ثم Approve ثم دخول Vendor Area. بتقديم عضو مختلف لكل جزء (D يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 3: Catalog (Phases 12-13)

البند	التفاصيل
1. Sprint Goal	Storefront للعامة وإدارة منتجات Vendor مع عزل كامل.
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=QA, B=INT, C=Fac, D=BE, E=FE).
3. Shared Requirements and Design	مراجعة ERD للـ Product وقواعد الصور (BR-09) و 12 Diagram و 22.
4. Small Tasks لكل عضو	Member A: ProductService: Search/Paging Vendor CRUD بالملكية Storefront (Listing/Details) Member B: Views FileStorageService, Member C: Vendor Product Views ERD تحديث + Search, Member D: Tests: IDOR Seed, Migration, Member E: Category/Product/ProductImage: Entities
5. Pair Programming	.Session 1: D+E , A+B (Rover C). Session 2: E+A , B+C (Rover D)
6. Code Review	A و E , PR E ← D و D , PR D ← C و C , PR C ← B و B , PR B ← A و PR A ← E
7. Integration Tasks	Merge الـ Migration أولاً، ثم الـ Services، ثم الـ Views. (المسؤول: B ≤ INT Lead).
8. Testing Tasks	Test Cases 2 و 5 و 13. الكل يختبر Feature زميله يدوياً، و (A) QA Lead يجمع النتائج.
9. Documentation Tasks	تحديث 6B Diagram. (B يتأكد من التحديث).
10. Sprint Review	Demo: Vendor يضيف منتج ويظهر في الـ Storefront. بتقديم عضو مختلف لكل جزء (C يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 4: Shopping Cart (Phases 14)

البند	التفاصيل
1. Sprint Goal	Cart كامل بحسابات من الـ Database.

البند	التفاصيل
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة ال Backlog، تقسيم ال Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=INT, B=Fac, C=BE, D=FE, E=QA).
3. Shared Requirements and Design	مراجعة Diagram 13 وقواعد BR-03 و BR-04.
4. Small Tasks لكل عضو	Member A: CartController و AntiForgery و رسائل الأخطاء Member B: Tests مخزون/كمية/تلاعب بالسعر Member C: CartItem: Entity و Unique Index و Migration Member D: CartService: Add/Update/Remove/Get Member E: Cart Views وعداد ال Navbar
5. Pair Programming	.Session 1: E+A , B+C (Rover D). Session 2: A+B , C+D (Rover E)
6. Code Review	B و A , PR E ← A و E , PR D ← E و D , PR C ← D و C , PR B ← C و PR A ← B
7. Integration Tasks	ربط عداد ال Navbar بال Layout المشترك بالتنسيق مع Frontend Lead (المسؤول: A ≤ INT Lead).
8. Testing Tasks	Test Cases 6-8. الكل يختبر Feature زميله يدوياً، و (E) QA Lead يجمع النتائج.
9. Documentation Tasks	.README: Seed Demo Accounts (A يتأكد من التحديث).
10. Sprint Review	Demo: Cart من 3 Vendors. بتقديم عضو مختلف لكل جزء (B يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 5: Orders and Checkout (Phases 15)

البند	التفاصيل
1. Sprint Goal	Checkout بـ Transaction وحجز المخزون وشاشات الطلبات.
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة ال Backlog، تقسيم ال Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=Fac, B=BE, C=FE, D=QA, E=INT).
3. Shared Requirements and Design	Design Session لـ Transaction ورسم Diagram 14 على السبورة قبل الكود.
4. Small Tasks لكل عضو	Member A: Order/OrderItem/Payment: Entities و Constraints و Migration Member B: CheckoutService: Transaction Conditional UPDATE و Member C: Checkout Views و Order History Member D: Vendor Orders و OrderService.UpdateFulfillment Member E: Tests و سباق آخر قطعة و State Transitions
5. Pair Programming	.Session 1: A+B , C+D (Rover E). Session 2: B+C , D+E (Rover A)
6. Code Review	C و B , PR E ← B و A , PR D ← A و E , PR C ← E و D , PR B ← D و PR A ← C
7. Integration Tasks	Migration واحدة بس لكل الجداول المالية، يكتبها Integration Lead بعد مراجعة ال Entities. (المسؤول: E ≤ INT Lead).
8. Testing Tasks	Test Cases 9-12 و 15. الكل يختبر Feature زميله يدوياً، و (D) QA Lead يجمع النتائج.
9. Documentation Tasks	.Diagram 20A/20B (E يتأكد من التحديث).
10. Sprint Review	Demo: Order مقسوم بين Vendorين. بتقديم عضو مختلف لكل جزء (A يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 6: Payments and Commission (Phases 16-17)

البند	التفاصيل
1. Sprint Goal	دفع Stripe Test Mode و Webhook Idempotent وعمولات صحيحة.

البند	التفاصيل
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=BE, B=FE, C=QA, D=INT, E=Fac).
3. Shared Requirements and Design	الفريق كله يقرأ صفحات Stripe المعتمدة (Webhooks و Checkout Fulfillment) ويرسم Diagram 17 بإيده.
4. Small Tasks لكل عضو	CommissionService, Member A: CommissionSetting/Record Admin Commission Form, Member B: Success/Cancel/Retry Stripe CLI, Member C: Tests: Duplicate/Signature/Commission Member D: StripePaymentService: Create Session Idempotency, Member E: Webhook Controller/Service: Signature
5. Pair Programming	.Session 1: B+C , D+E (Rover A). Session 2: C+D , E+A (Rover B)
6. Code Review	D و C , PR E ← C و B , PR D ← B و A , PR C ← A و E , PR B ← E و PR A ← D
7. Integration Tasks	الـ Webhook والـ Commission يتدمجوا في Branch مشترك قبل الـ Merge (Pair). (المسؤول: INT ≤ D). (Lead)
8. Testing Tasks	Test Cases 9-11. الكل يختبر Feature زميله يدوياً، و (C) QA Lead يجمع النتائج.
9. Documentation Tasks	Stripe CLI Guide. (D يتأكد من التحديث).
10. Sprint Review	Demo: دفع تجريبي كامل حتى ظهور Commission. بتقديم عضو مختلف لكل جزء (E يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 7: Dashboards (Phases 18-20)

البند	التفاصيل
1. Sprint Goal	Admin Dashboards و Vendor و Customer
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=FE, B=QA, C=INT, D=Fac, E=BE).
3. Shared Requirements and Design	مراجعة الأرقام المطلوبة في كل Dashboard مع الـ Stories.
4. Small Tasks لكل عضو	Member A: Tests الـ Authorization و الـ Database مقابل الأرقام Vendor لـ Member B: Aggregates Admin لـ Member C: Aggregates Orders Monitoring Customer لـ Member D: تفاصيل الطلب و الـ Cancel Charts و Member E: Views
5. Pair Programming	.Session 1: C+D , E+A (Rover B). Session 2: D+E , A+B (Rover C)
6. Code Review	A و E , PR E ← D و D , PR D ← C و C , PR C ← B و B , PR B ← A و PR A ← E
7. Integration Tasks	توحيد الـ Layout بين الـ Areas. (المسؤول: C ≤ INT). (Lead)
8. Testing Tasks	Test Case 3. الكل يختبر Feature زميله يدوياً، و (B) QA Lead يجمع النتائج.
9. Documentation Tasks	Screenshots أولية. (C يتأكد من التحديث).
10. Sprint Review	Demo: لوحة Vendor و Admin بأرقام حقيقية. بتقديم عضو مختلف لكل جزء (D يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 8: Hardening (Phases 21-23)

البند	التفاصيل
1. Sprint Goal	اختبار شامل وأمان وإصلاح الأخطاء.

البند	التفاصيل
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=QA, B=INT, C=Fac, D=BE, E=FE).
3. Shared Requirements and Design	مراجعة Checklist الأمان و Test Matrix كاملة.
4. Small Tasks لكل عضو	Member A: إصلاح Bugs من Test Report Member B: مراجعة Indexes والـ Queries Member C: Regression Automation في الـ CI Member D: Test Report وتحديث Docs Member E: Security Review (Secrets/CSRF/Uploads)
5. Pair Programming	.Session 1: D+E , A+B (Rover C). Session 2: E+A , B+C (Rover D)
6. Code Review	B و A , PR E ← A و E , PR D ← E و D , PR C ← D و C , PR B ← C و PR A ← B
7. Integration Tasks	Freeze للـ Features الجديدة؛ Bug Fixes فقط. (المسؤول: B ≤ INT Lead).
8. Testing Tasks	كل الحالات الـ 15. الكل يختبر Feature زميله يدوياً، و (A) QA Lead يجمع النتائج.
9. Documentation Tasks	Test Report. (B يتأكد من التحديث).
10. Sprint Review	Demo: Regression كامل. بتقديم عضو مختلف لكل جزء (C يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

Sprint 9: Release (Phases 24-26)

البند	التفاصيل
1. Sprint Goal	نشر Demo و Webhook حي وتجهيز العرض.
2. Shared Planning Session	اليوم الأول، ساعتين، الخمسة: مراجعة الـ Backlog، تقسيم الـ Stories لـ Tasks صغيرة، وتحديد Definition of Done، وتوزيع الأدوار (A=INT, B=Fac, C=BE, D=FE, E=QA).
3. Shared Requirements and Design	مراجعة Deployment Plan وسيناريو الـ Demo.
4. Small Tasks لكل عضو	Member A: العرض التقديمي وسكربت الـ Demo Member B: تحديث نهائي للـ Dry Runs و Diagrams Member C: Deployment Config و Migrations Script Member D: Smoke Tests على Production و Webhook Endpoint Member E: README و Screenshots و Demo Accounts
5. Pair Programming	.Session 1: E+A , B+C (Rover D). Session 2: A+B , C+D (Rover E)
6. Code Review	C و B , PR E ← B و A , PR D ← A و E , PR C ← E و D , PR B ← D و PR A ← C
7. Integration Tasks	Tag بعد v1.0.0 Smoke Test. (المسؤول: A ≤ INT Lead).
8. Testing Tasks	Smoke + Dry Run. الكل يختبر Feature زميله يدوياً، و (E) QA Lead يجمع النتائج.
9. Documentation Tasks	Docs النهائية. (A يتأكد من التحديث).
10. Sprint Review	Demo: العرض النهائي. بتقديم عضو مختلف لكل جزء (B يدير الجلسة).
11. Retrospective	45 دقيقة: ما الذي نجح؟ ما الذي لم ينجح؟ Action Item واحد على الأقل بمسؤول وموعد.

10.5 مثال عملي: الفريق كله على Feature واحدة (Vendor Product Management, Sprint 3)

اليوم	النشاط	مين؟
1	Planning: مراجعة US-14 وDiagrams 12 و22 وERD الخاص بـ Product. كل عضو يشرح جزء بصوته.	الخمسة
2	Design Session: نكتب على السبورة شكل ProductService وحالات الخطأ وقواعد الملكية.	الخمسة
2-3	Entities وConfigurations وMigration (Pair).	عضوين + Rover
3-5	ProductService (Create/Update/Deactivate) بالملكية (Pair بالتبادل Driver/Navigator).	عضوين
3-5	Views الـ Vendor (Index/Create/Edit) وFileStorageService.	عضوين
5	Integration: Merge الـ Migration، ثم الـ Service، ثم الـ Views. مع Rebase للفروع.	INT Lead + الباقي
6	Collective Testing: كل عضو يحاول يكسر الملاحظات (Vendor A ← منتج Vendor B) ويكتب Test.	الخمسة
7	Code Review متبادل (حسب الجدول)، وتصحيح الملاحظات.	كل عضو بيراجع PR
8	Docs: تحديث Diagram 6B وـ README.	INT Lead + Reviewer
9	Sprint Review: Demo من عضو، والباقي يجاوب أسئلة. Knowledge Share لمدة 30 دقيقة.	الخمسة

10.6 تنسيق التعديلات على الملفات المشتركة

الملف المشترك	المشكلة	القاعدة
ApplicationDbContext	كل Feature بتضيف DbSet. فيحمل Conflict على نفس السطور.	نضيف الـ DbSet في سطر جديد بترتيب أبجدي، ونخط الـ Fluent Config في ملفات Configurations / منفصلة (ApplyConfigurationsFromAssembly).
Program.cs	كل Feature بتسجل Services.	نقسم التسجيل لـ Extension Methods (AddCartServices() ...) في Infrastructure/ServiceRegistration.cs، وكل Feature تعدّل ملف خاص بيها قدر الإمكان.
Shared ViewModels	تعديلات متضاربة على نفس الـ Class.	لا نعدّل ViewModel مشترك إلا بعد إعلام الفريق، والتغيير يتعمل في PR صغير مستقل.
Shared Layouts (_Layout.cshtml)	تغييرات في الـ Navbar من أكثر من عضو.	Frontend Lead في الـ Sprint هو بوابة التغيير، ونستخدم Partial Views (_NavbarCart.cshtml) لتقليل التضارب.
Entity Configuration	تعديل نفس الـ Configuration.	ملف لكل Entity، وأي تغيير في Entity موجودة يمر على PR من الـ Backend Lead.
DI Configuration	ترتيب التسجيل.	الترتيب مش مهم لـ Scoped Services العادية، لكن لازم نتجنب تسجيل نفس الـ Interface مرتين.
Migrations	Snapshot واحد، لو اتدّين عملوا Migration هيتعارضوا.	القواعد في الفصل 11.

10.7 ازاي نقل الـ Merge Conflicts ونضمن إن الكل فاهم؟

1. Branches قصيرة العمر (1-3 أيام) وRebase + git fetch على main كل يوم.
2. Small PRs (أقل من 400 سطر قدر الإمكان).
3. ملفات صغيرة (Configuration لكل Controller، Entity لكل مجال).
4. Merge Order يحدده INT Lead في الـ Integration: Migrations ثم Services ثم Controllers ثم Views.

5. **Code Review** كوسيلة تعلم: ال Reviewer لازم يكتب في ال PR سطرين بيشرح فيهم ماذا فهم من ال Change. وإن ما فهمش يسأل.
6. **'Explain it back'**: في ال Sprint Review أي عضو ممكن يتسأل عن أي Feature، وليس فقط اللي كتبها.
7. **Rotating Tasks**: مغيث حد ياخذ نفس نوع ال Task في Sprintين متتاليين.
8. **Bus Factor ≥ 3** : أي جزء في النظام لازم يكون على الأقل 3 أشخاص يعرفوه.

11. سير العمل على Git و GitHub (Git and GitHub Workflow)

11.1 إعداد الـ Repository

1. واحد من الفريق (يتبدل) ينشئ Repository خاص باسم **MarketHub** ويضيف الأعضاء الخمسة كـ Collaborators.
2. نضيف **gitignore** لـ **dotnet new gitignore** و.NET. ونتأكد إنه بيتجاهل: **bin/**, **obj/**, **.vs/**, **global.json**, **CONTRIBUTING.md**, **README.md**, **editorconfig**, **github/pull_request_template.md**, **Issue Templates (Bug / Story)** و **github/pull_request_template.md**.
3. نضيف **global.json** (لتثبيت الـ SDK) و **editorconfig**.
4. نضيف **github/pull_request_template.md** و **Issue Templates (Bug / Story)**.

Branch Protection على main (من Settings → Branches)

- Require a pull request before merging (Approvals: 1) و 2 لا PRs اللي فيها Migration أو Payment.
- Require status checks to pass (الـ Build و الـ Tests).
- Require branches to be up to date before merging.
- Do not allow bypassing the above settings (حتى لا Admins).
- ممنوع Force Push على **main**.

11.2 تسمية الـ Branches و الـ Commits

النوع	الصيغة	مثال
Feature	<feature/<issue#>-<short-name>	feature/42-vendor-product-create
Bug Fix	<fix/<issue#>-<short-name>	fix/57-cart-negative-qty
Docs	<docs/<short-name>	docs/update-erd
Chore	<chore/<short-name>	chore/ci-workflow

Commit Message (Conventional Commits): **type(scope): short description** مثل **feat(cart):** **validate stock before adding item** و **fix(webhook): ignore duplicate stripe events**.
test(vendor): add idor tests. الأنواع: **feat**, **fix**, **docs**, **test**, **refactor**, **chore**. الرسالة بصيغة الأمر وأقل من 72 حرف في السطر الأول.

11.3 مثال كامل: Feature صغيرة من الأول للآخر

السيناريو: Member C مضيف زرار "Deactivate" للمنتج (Issue رقم 42). و Member D هو الـ Reviewer.

```
# 1) Start from an up-to-date main
git switch main
git pull origin main

# 2) Create the feature branch
git switch -c feature/42-product-deactivate

# 3) Work and commit in small steps
git add src/MarketHub.Web/Services/Implementations/ProductService.cs
git commit -m "feat(product): add DeactivateAsync with vendor ownership check"
git add tests/MarketHub.Tests/Unit/ProductServiceTests.cs
git commit -m "test(product): deactivate other vendor product returns not found"

# 4) Sync with main before pushing (keeps history linear)
git fetch origin
git rebase origin/main

# 5) Push and open a Pull Request
```

```
git push -u origin feature/42-product-deactivate
gh pr create --base main --title "feat(product): deactivate product (#42)" \
  --body "Closes #42. Adds DeactivateAsync + tests. Reviewer: @member-d"
```

Review: Member D (3) يفتح ال PR، يقرأ ال Diff، ويكتب ملاحظة: *ال Controller بيرجع Redirect حتى لو ال Service رجعت Fail. ممكن نعرض رسالة الخطأ؟* ويضغط **Request changes**.

```
# 6) Fix the review comments on the same branch
git add src/MarketHub.Web/Areas/Vendor/Controllers/ProductsController.cs
git commit -m "fix(product): show error message when deactivate fails"
git push
```

```
# 7) After approval + green CI: Squash and merge on GitHub, then clean up
git switch main
git pull origin main
git branch -d feature/42-product-deactivate
git push origin --delete feature/42-product-deactivate
```

ال Reviewer بعد ما يوافق (Approve) يكتب سطرين: *فهمت إن ال Service بتبحث بـ **VendorProfileId** فبترجع NotFound لمنتجات الغير*، وده دليل الفهم (القاعدة 5 في 10.7).

قالب ال Pull Request

```
## What
Short description. Closes #<issue>

## Why
Link to the user story / business rule (BR-xx).

## How to test
1. Steps ...

## Checklist
- [ ] Build and tests pass locally
- [ ] Server-side validation and authorization covered
- [ ] No secrets or personal data committed
- [ ] Migration included? (needs 2 reviewers)
- [ ] Docs / diagrams updated if the design changed
```

11.4 حل ال Merge Conflicts

```
git fetch origin
git rebase origin/main
# Git stops at a conflict: open the files, look for <<<<<<, =====, >>>>>>
# Keep the correct combination of both changes, then:
git add <resolved-file>
git rebase --continue
dotnet build && dotnet test      # ALWAYS re-run after resolving
git push --force-with-lease      # only on YOUR feature branch, never on main
```

قواعد: (1) نحل ال Conflict وإحنا فاهمين الاتنين تغييرات (ما نختارش "Accept mine" بشكل أعمى). (2) لو ال Conflict في ملف ال Migrations: انظر 11.7. (3) لو الموضوع معقد: Pair مع كاتب التغيير التاني.

11.5 GitHub Issues و Project Board و Milestones

- **Issue** لكل **Story-slice**: عنوان واضح، Acceptance Criteria، كـ (backend, frontend)، Checklist. Labels (db, test, docs, bug, payment) و Milestone = Sprint (مثلاً 3 Sprint).
- **Project Board** بأعمدة: **Done** ← **In Review** ← **In Progress** ← **Sprint Backlog** ← **Backlog**. حد أقصى 2 Issues In Progress للعضو (WIP Limit).
- **Milestones**: واحد لكل Sprint ومعا تاريخ Demo.
- ال PR بيحتوي **#42 Closes** فال Issue بيتنقل تلقائياً بعد ال Merge.

11.6 Environment Configuration و إدارة الـ Secrets

الـ **README** لازم يحتوي: وصف المشروع، المتطلبات (NET 10 SDK, SQL Server, Stripe CLI)، خطوات التشغيل، إعداد الـ Secrets، تشغيل الـ Demo Accounts, Tests, هيكل الفولدرات، و لينك للـ Docs.

```
# One-time setup per developer (secrets live outside the repo)
cd src/MarketHub.Web
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=(localdb)\\
MSSQLLocalDB;Database=MarketHub;Trusted_Connection=True;TrustServerCertificate=True"
dotnet user-secrets set "Stripe:SecretKey" "sk_test_..."
dotnet user-secrets set "Stripe:WebhookSecret" "whsec_..."
dotnet user-secrets set "Seed:AdminEmail" "admin@markethub.test"
dotnet user-secrets set "Seed:AdminPassword" "<choose a strong password>"
```

- الملف **appsettings.json** فيه مفاتيح فاضية فقط.
- نفعّل **Secret Scanning** و **Push Protection** في GitHub لو متاحة للـ Repository.
- لو حد رفع Secret بالغلط: نغيّر المفتاح فوراً (Roll)، مش بس نمسحه من الـ Commit لأن التاريخ بيغفل.

11.7 تنسيق الـ EF Core Migrations بين الأعضاء

المشكلة: كل Migration بتعدل ملف **ApplicationDbContextModelSnapshot.cs**. لو عضوين عملوا Migration من نفس نقطة البداية، الـ Snapshot هيتعارض، والـ Migrations ممكن تتناقض.

القواعد:

1. **Migration** واحدة في الـ PR، وبيعملها عضو واحد (غالباً INT Lead للـ Sprint) بعد ما الـ Entities تتراجع.
2. قبل ما تنشئ Migration: **git fetch && git rebase origin/main** وتأكد إنك على آخر نسخة، وتطبق الـ Migrations الموجودة (**dotnet ef database update**).
3. بعد إنشاء الـ Migration، افتح ملف الـ Migration وراجع الـ **Up** و **Down**. وتأكد إنها بتعمل بس اللي إنت عايزه.
4. اعلن في قناة الفريق "أنا شغال على Migration" لحد ما تتعمل Merge.
5. لو حصل **Conflict** في الـ Snapshot: احذف الـ Migration بتاعتك (**dotnet ef migrations remove**) واعمل Rebase، ثم أعد إنشائها. متحلش الـ Snapshot يدويًا.
6. ممنوع تعديل Migration اتعمل لها Merge.
7. الـ CI بيتأكد إن **dotnet ef migrations has-pending-model-changes** (أو ما يعادله في النسخة المستخدمة) مبرجعش تغييرات ناسية [Team Decision]: تأكدوا من اسم الأمر في الإصدار الحالي من **dotnet ef**.

11.8 مثال CI بسيط (GitHub Actions)

```
name: ci
on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          global-json-file: global.json
      - run: dotnet restore
      - run: dotnet build --no-restore -c Release
      - run: dotnet test --no-build -c Release
```

[Team Decision]: تأكدوا من إصدارات الـ Actions (**checkout**, **setup-dotnet**) من الـ Marketplace الرسمي لأنها بتتحدث.

12. استراتيجيات الاختبار وضمان الجودة (Testing and QA)

12.1 أنواع الاختبارات ومكانها في المشروع

النوع	بيختبر إيه؟	الأداة	مثال
Unit Testing	Logic معزول من غير Database حقيقية	xUnit + (Moq أو NSubstitute اختياري)	CommissionService يحسب 10% صح
Integration Testing	Database مع Services فعلية	xUnit + SQL Server LocalDB أو SQLite In-Testcontainers [Team Decision] Memory بحذر	CheckoutService ينشئ Order ويحجز المخزون
MVC Functional Testing	Request كامل عبر ال Pipeline	WebApplicationFactory<Program> + HttpClient	GET /Vendor/Products بدون Login يرجع Redirect
Manual Testing	تجربة المستخدم وال Flow	Test Scripts في Issue Templates	Checkout بكارت تجريبي
Authorization Testing	الملاحيات (أهم نوع عندنا)	Functional Tests بـ Users مختلفين	Vendor A ضد منتج Vendor B
Database Testing	القيود وال Indexes وال Transactions	Integration على SQL Server حقيقي	CHECK (Price > 0) يرفض -5
Payment Testing	Stripe في Test Mode وال Webhooks	Stripe CLI (stripe trigger, stripe Handler لل listen) + Unit Tests	Webhook مكرر
Regression Testing	إن القديم لسه شغال	تشغيل كل ال Tests في كل PR (CI) + Checklist بدوي قبل ال Release	

تنبيه: SQLite أو InMemory Provider مبيتصرفوش زي SQL Server في بعض الحاجات (Transactions, rowversion, CHECK, ExecuteUpdate). اختبارات ال Checkout وال Concurrency لازم تشتغل على SQL Server حقيقي (LocalDB أو Container).

12.2 هرم الاختبارات في مشروعنا

أغلب الاختبارات Unit/Integration سريعة على ال Services، وعدد قليل من ال Functional Tests لل Authorization، وقليل جداً من Manual Tests لل End-to-End مع Stripe. كل Feature Pull Request لازم يضيف Tests لل Service Logic وال Authorization.

12.3 Test Cases الأساسية (ال 15 مجال المطلوبين)

الأرقام TC-01 إلى TC-15 مطابقة لترتيب المجالات. بنفذ كل واحد ك Automated Test لما يكون ممكن، وإلا ك Manual Script.

ID	السيناريو	النتيجة المتوقعة	طريقة التنفيذ
TC-01a	Unauthorized Access: Guest يفتح /Cart	Redirect لـ (302) Login، مفيش بيانات	Functional: GET /Cart بدون Cookie
TC-01b	Guest يفتح /Vendor/Products/ و /Admin	Redirect/403	نفس الطريقة لكل Route محمي (Theory بـ InlineData)
TC-02	Vendor A يعذل منتج Vendor B: POST /Vendor/Products/Edit/{idB}	404، والمنتج لم يتغير في ال DB	Functional بـ Vendor A Login، وبعدها Assert على ال DB
TC-02b	Overposting: إرسال VendorProfileId=B في ال Form	يتجاهل الحقل، والملكية تفضل A	Functional بإرسال حقل زيادة

ID	السيناريو	النتيجة المتوقعة	طريقة التنفيذ
TC-03	Vendor A يشوف OrderItems بتاعة B: GET /Vendor/Orders/Details/{itemB}	404، وقائمة الطلبات فيها A items فقط	Seed Order فيه Items ل Vendor، ثم Assert على المحتوى
TC-04a	Vendor غير معتمد (Pending/Rejected/Suspended) / */Vendor	403 أو Redirect، مع رسالة الحالة	Seed Vendor بكل حالة، Theory
TC-04b	Vendor اتعمله Suspend وهو Logged-In	الطلب التالي مرفوض فوراً	Suspend ثم طلب بنفس ال Cookie
TC-05a	Invalid Product ID: GET /Products/Details/99999	404	Functional
TC-05b	POST /Cart/Add ب productId غير موجود/ Inactive	رسالة خطأ ولا Insert	Unit على CartService
TC-06	Insufficient Stock: المخزون 2 والعميل يطلب 3	رفض ب "Not enough stock"	Unit + Integration
TC-06b	آخر قطعة، اتنين Checkout في نفس اللحظة	واحد بس ينجح، والثاني يُرفض، والمخزون = 0 مش سالب	Integration ب Task.WhenAll على SQL Server
TC-07	Invalid Quantities: 0، -1، 1000000 نص	رفض Validation	Unit ب [Theory]
TC-08	Client-side Price Manipulation: إرسال Request في ال price=0.01	السعر المستخدم هو سعر ال DB، وال Order بالسعر الصحيح	Functional: إضافة price Field وAssert على OrderItem.UnitPrice
TC-09	Failed Payment: كارت مرفوض	ال Order يفضل PendingPayment ولا Commission، ولو ال Session انتهت: Cancelled والمخزون يرجع	Manual ب كارت الرفض من Stripe Testing + stripe trigger
TC-10	Duplicate Webhooks: نفس _evt مرتين	المرة الأولى تعمل Paid، الثانية تُتجاهل: Commissions ما اتكررتش	Unit/Integration: نادى HandleAsync مرتين بنفس ال Event
TC-10b	توقيع غلط أو Body متعطل	400 ولا تغيير	Functional بإرسال Signature خاطئ
TC-10c	مبلغ ال Session ≠ مبلغ ال Order	رفض + Log، ولا Paid	Unit ب Event مصطنع
TC-11	Incorrect Commission Calculation	النتائج مطابقة لجدول الأمثلة (255.55 ← 229.99 / 25.56)	Unit ب قيم محسوبة يدوياً، مع حالات التقريب (0.005)
TC-11b	تغيير النسبة بعد الدفع	السجلات القديمة بنفس النسبة القديمة	Integration
TC-12	Multi-Vendor Order: Cart Vendors فيه منتجات من 3	Order واحد، +3 Items، كل Item ب VendorProfileId المحج، لكل Item Commission	Integration
TC-13a	DB Constraints: Price = -1 DB مباشرة في ال	SQLException (CHECK)	Integration ب SaveChanges مباشرة
TC-13b	Unique: StoreName/Email/StripeEventId مكرر	DbUpdateException	Integration
TC-13c	FK: OrderItem لم منتج غير موجود	رفض	Integration
TC-14a	Invalid Role Permissions: Customer */Admin/ و */Vendor/ يفتح	403	Functional ب Customer
TC-14b	Vendor يفتح */Admin/	403	Functional
TC-14c	Customer يطلب / Orders/Details/{orderOf0therCustomer}	404	Functional

ID	السيناريو	النتيجة المتوقعة	طريقة التنفيذ
TC-15	Order Status Transitions المسموح PendingPayment→Paid/Cancelled, Paid→Cancelled, الممنوع, Paid→Completed Paid قبل Completed→Paid, Shipped	الممنوع يرجع Fail ولا يتغير شيء	Unit على State Machine Theory ب (Diagram 20)
TC-15b	Fulfillment: Delivered قبل Shipped	رفض	Unit

12.4 مثال Authorization Test (xUnit + WebApplicationFactory)

```
public class VendorIsolationTests : IClassFixture<MarketHubFactory>
{
    private readonly MarketHubFactory _factory;
    public VendorIsolationTests(MarketHubFactory factory) => _factory = factory;

    [Fact]
    public async Task VendorA_CannotEdit_VendorB_Product()
    {
        var (vendorA, productOfB) = await _factory.SeedTwoVendorsAsync();
        var client = _factory.CreateClientLoggedInAs(vendorA);

        var response = await client.PostFormAsync(
            $"/Vendor/Products/Edit/{productOfB.Id}",
            new { Name = "Hacked", Price = 1 });

        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
        var fromDb = await _factory.GetProductAsync(productOfB.Id);
        Assert.NotEqual("Hacked", fromDb.Name);
    }
}
```

MarketHubFactory و **PostFormAsync** و **Helpers** و **CreateClientLoggedInAs** نكتبهم مرة في **TestHelpers** (ببتعاملوا مع AntiForgery Token وتسجيل الدخول). الفريق يتفق على Test Authentication Handler بسيط لل Tests [Team Decision].

12.5 مثال Unit Test للعمولة

```
[Theory]
[InlineData(200.00, 10.00, 20.00, 180.00)]
[InlineData(55.55, 10.00, 5.56, 49.99)] // 5.555 rounds away from zero to 5.56
[InlineData(100.00, 0.00, 0.00, 100.00)]
public void Calculate_ReturnsExpectedAmounts(decimal gross, decimal rate, decimal commission, decimal net)
{
    var result = CommissionCalculator.Calculate(gross, rate);
    Assert.Equal(commission, result.CommissionAmount);
    Assert.Equal(net, result.VendorNetAmount);
}
```

12.6 الـ Regression و الـ Collective Testing

- **Fresh Eyes**: في آخر كل Sprint، كل عضو يجرب Feature عضو ثاني بـ Test Script مكتوب (من الـ Acceptance Criteria).
- **Bug Report Template**: خطوات، المتوقع، الفعلي، Screenshot، البيئة.
- **Regression Checklist** قبل أي Release: Register/Login. Vendor Apply/Approve. Product Create. Add to Cart. Checkout بـ كارت تجريبي، Admin Dashboard. Commission. Vendor Orders. Webhook.
- تعريف الجودة للـ **Sprint**: مفيش Bug من نوع Critical (أمان/فلوس/فقد بيانات) مفتوح.

13. الأمان والموثوقية (Security and Reliability)

كل بند هنا سيتحول لـ Checklist بنراجعها في Sprint 8 (Phase 22)، وبعضها سيتطبق من أول Sprint.

#	الخطر	الشرح بالعربي ومثال في MarketHub	الحل
1	Authentication / Authorization	لو Route من غير <code>[Authorize]</code> أي حد يوصله. مثال: <code>Admin/Commission/Edit/</code> مفتوح.	<code>[Authorize]</code> على <code>VendorBaseController</code> وكل <code>Admin Controller</code> ، و <code>Tests (TC-01, TC-14)</code> ، و <code>FallbackPolicy</code> تطلب Login ما لم يكتب <code>[AllowAnonymous]</code> <code>[Recommendation]</code> .
2	Vendor Data Isolation (IDOR)	تغيير <code>id</code> في الـ URL للوصول لبيانات Vendor ثاني.	فلتر بـ <code>VendorProfileId</code> في كل <code>Query + 404</code> موحد (انظر 3.4، TC-02/03).
3	Server-Side Validation	الـ Client Validation بـتتجاوز بسهولة (Postman).	<code>Data Annotations + Business Validation</code> في الـ <code>Services</code> ، ومغيش ثقة في أي قيمة من الـ <code>Browser</code> .
4	Overposting (Mass Assignment)	إرسال <code>Price</code> أو <code>VendorProfileId</code> أو <code>Status</code> مش مقصود إنه يتعدل.	<code>ViewModels</code> مخصصة لكل <code>Action</code> ، ومغيباش الحقول الحساسة؛ ولا <code>[Bind]</code> على الـ <code>Entities</code> .
5	CSRF	موقع خبيث يخلي المتصفح بتاعك بيعت POST (مثلاً <code>Approve Vendor</code>).	<code>[ValidateAntiForgeryToken]</code> (أو <code>AutoValidateAntiforgeryTokenAttribute</code> <code>asp-action</code> Tag، و <code>POST</code> على كل <code>global</code> <code>Helpers</code> بتضيف الـ <code>Token</code> تلقائياً، الـ <code>Webhook</code> استثناء ومحمي بالتوقيع.
6	Secure File Uploads	ملف <code>exe</code> ، أو <code>svg</code> فيه <code>Script</code> بيتخط كمورة.	<code>Whitelist</code> للامتدادات (<code>jpg/png/webp</code>)، فحص الـ <code>Content-Type</code> وحجم الملف، اسم ملف جديد (<code>GUID</code>) من السيرفر، تخزين تحت <code>wwwroot/uploads</code> (أو <code>Storage</code> خارجي) بدون تنفيذ، ومنع <code>Path Traversal</code> (<code>Path.GetFileName</code>).
7	SQL Injection	دمج مدخلات المستخدم في <code>Query</code> كنص.	<code>EF Core/LINQ</code> بتستخدم <code>Parameters</code> ، لو احتجنا <code>SQL</code> خام: <code>FromSqlInterpolated</code> ، وممنوع <code>FromSqlRaw</code> بتجميع <code>Strings</code> .
8	Password Security	تخزين <code>Passwords</code> أو سياسة ضعيفة.	<code>Identity</code> بيعمل <code>Hashing</code> ؛ ضبط الطول +8، <code>Lockout</code> ، ولا نسجل <code>Passwords</code> في <code>Logs</code> .
9	Secrets Management	مفتاح <code>Stripe</code> على <code>GitHub</code> = خسارة مالية.	<code>user-secrets</code> محلياً، <code>Environment Variables</code> على الاستضافة، <code>Roll</code> ، <code>Secret Scanning</code> عند التسريب (11.6).
10	Stripe Webhook Verification	أي حد بيعت <code>POST</code> ويقول "الدفع تم".	التحقق من <code>Stripe-Signature</code> بالـ <code>raw body</code> وسر الـ <code>Endpoint</code> ، ورفض أي حاجة غير موقعة (TC-10b).
11	Idempotency	<code>Stripe</code> بيعيد الإرسال (At-least-once)، فنكرر العمليات.	<code>StripeWebhookEvent</code> <code>Unique</code> + فحص حالة الـ <code>Order</code> (8.4، TC-10).
12	Transaction Management	فشل في نص العملية يسبب بيانات متناقضة.	<code>BeginTransactionAsync</code> حوالين: إنشاء الـ <code>Order</code> + الحجز، ومعالجة الـ <code>Webhook</code> .
13	Inventory Consistency	اتنين يشتروا آخر قطعة.	<code>Conditional UPDATE + RowVersion</code> + TC-06b.
14	Error Handling	<code>Stack Trace</code> ظاهر للمستخدم بيكشف معلومات.	<code>ExceptionHandler</code> + صفحة <code>Error</code> عامة في <code>Production</code> ، و <code>UseDeveloperExceptionPage</code> في <code>Development</code> فقط.
15	Logging	غياب <code>Logs</code> = مغيش تحقيق، و <code>Logs</code> فيها بيانات حساسة = تسريب.	<code>ILogger</code> لـ: رفض 403، <code>Webhook</code> متكرر، فشل <code>Transaction</code> ، ممنوع تسجيل <code>Secrets</code> أو بيانات بطاقات أو <code>Passwords</code> .
16	Open Redirect	<code>returnUrl</code> يوجه لموقع خارجي بعد <code>Login</code> .	<code>Url.IsLocalUrl(returnUrl)</code> قبل <code>Redirect</code> .

#	الخطر	الشرح بالعربي ومثال في MarketHub	الحل
17	Cookie / HTTPS / Headers	سرقة ال Session على HTTP.	HttpOnly Cookies و HTTPS + HSTS، Secure Security Headers (الافتراضي في Identity)، ومراجعة (CSP/X-Content-Type-Options) [Team Decision]

مثال عملي: لماذا نرفض السعر القادم من ال Browser؟

لو فورم ال Checkout فيه `<input type="hidden" name="price" value="500">` وال Controller بيستخدمها، أي مستخدم يفتح Developer Tools ويخليها 1، وبكده اشترى بـ 1. الحل ببساطة إن مفيش Input للسعر أصلاً، وال Service بتجيب `Product.Price` من SQL Server.

مثال عملي: ال Idempotency

Stripe ممكن بيعت `checkout.session.completed` نفسه مرتين (مثلاً لأن ردنا متأخر). من غير الحماية: Commission مرتين ومخزون متحرر مرتين. مع جدول `StripeWebhookEvent` ال Insert الثاني بيفشل بسبب ال Unique، فبنرجع 200 وما بنعملش حاجة.

14. النشر والتسليم النهائي (Deployment and Final Delivery)

14.1 ثلاث بيئات مختلفة لازم نفرّق بينها

الجانِب	Local Development	Demo Test Mode (على Stripe)	Production حقيقي (مستقبلي)
الغرض	تطوير واختبار	عرض للجنة التقييم	عملاء حقيقيين
Stripe Keys	user-..._sk_test secrets	Environment ..._sk_test Variables	..._sk_live (بعد التحقق من الحساب والامتثال)
Webhook	stripe listen (سر مؤقت من الـ CLI) _whsec	Endpoint مسجل في Dashboard بـ HTTPS عام	Endpoint مخصص بسر مختلف
فلوس حقيقية	لا	لا	نعم، ومحتاج مراجعة قانونية ومالية
Database	LocalDB/Express	SQL Server مستضاف (بيانات Demo)	SQL Server مُدار مع Backups
Seed Data	Demo كامل	Demo Accounts محدودة	Roles + Admin فقط
Stripe Connect Payouts	غير موجود	غير موجود	مرحلة مستقبلية منفصلة (8.8)

الـ Demo Online مش **Production**: بياخد فلوس وهمية فقط. لو قررنا نستقبل دفع حقيقي لازم نتحقق من: دولة حساب Stripe، شروط الخدمة، الضرائب، وسياسة الخصوصية.

14.2 خيارات الاستضافة [Team Decision]

مفيش خيار واحد صح، ولا نفترض إن مزود معين متاح أو مجاني. قارنوا الخيارات الحالية من مواقعها الرسمية، وتأكدوا من الأسعار والـ Free Tiers وقت التنفيذ.

الخيار	ملاحظات للمقارنة
Azure App Service + Azure SQL	تكاملي قوي مع .NET و SQL Server، راجعوا الأسعار وبرامج الطلاب (لو متاحة لكم).
استضافة Windows/IIS (Shared/VPS)	مناسبة لو متاح عندكم، محتاجة إعداد يدوي.
VPS Linux + Docker	مرونة، ومحتاج SQL Server Container أو مزود قاعدة بيانات ثاني، وخبرة أعلى.
مزودين PaaS تانيين	تأكدوا من دعم .NET 10 و SQL Server والـ Environment Variables والـ HTTPS.

معايير الاختيار: دعم .NET 10، دعم SQL Server (مُدار أو Container)، HTTPS مجاني، Environment Variables، Logs، السعر.

14.3 خطوات النشر

1. **Production Configuration:** `ASPNETCORE_ENVIRONMENT=Production`، و App Settings من Environment Variables (`ConnectionString__DefaultConnection`، `Stripe__SecretKey`، `Stripe__WebhookSecret`، `Seed__AdminEmail`، `Seed__AdminPassword`)
2. **SQL Server Hosting**: إنشاء Database و User بأقل صلاحيات لازمة، وتفعيل الـ Firewall للـ Host فقط.
3. **Database Migrations**: توليد `Script dotnet ef migrations script --idempotent -o` و `deploy.sql` وتطبيقه، أو أمر منفصل في الـ Deployment [Team Decision]: التطبيق التلقائي عند البدء مقبول للـ Demo فقط.

4. **Application Deployment: dotnet publish -c Release**, ورفع المخرجات (أو نشر عبر GitHub Actions بعد ال Merge).
5. **HTTPS**: تفعيل شهادة (غالباً مجانية من الاستضافة)، و **UseHsts** و **UseHttpsRedirection**.
6. **Stripe Webhook**: تسجيل `https://<your-domain>/stripe/webhook` في Stripe Dashboard (Test Mode)، واختيار الأحداث المطلوبة (`checkout.session.completed`), ونسخ ال `checkout.session.async_payment_succeeded`, `checkout.session.async_payment_failed`, `checkout.session.expired` إلى Environment Variables Signing Secret.
7. **Uploads**: تأكد إن مكان الصور دائم (Persistent). على بعض الاستضافات ال File System مؤقت، فممكّن تحتاجوا Blob/Object Storage [Team Decision].
8. **Logging**: تفعيل ال Logs الاستضافة، ومراجعة الأخطاء بعد أول تشغيل.
9. **Error Handling**: تأكد إن صفحة الخطأ العامة بتظهر ومفّيش Stack Trace.
10. **Smoke Test**: Register → Vendor Apply/Approve → Product → Cart → Checkout → Webhook → Commission → Vendor Orders. بكارت تجريبي →
11. **Rollback Plan**: احتفظوا بآخر Build سليم وبـ Backup لل Database قبل أي Migration.

Demo Accounts 14.4

الدور	Email	ملاحظة
Super Admin	من <code>Seed:AdminEmail</code>	الباسورد في Environment Variables
Vendor 1 / 2 (Approved)	<code>vendor1@demo.test</code> / <code>vendor2@demo.test</code>	باسورد Demo ظاهر في README (حسابات تجريبية فقط)
Vendor (Pending)	<code>pending@demo.test</code>	لعرض Flow الموافقة
Customer	<code>customer@demo.test</code>	

مهم: حسابات ال Demo باسوردها المعلن مبيتحطش في ال Production الحقيقي. ال Seed بتاعها مشروط بـ `Flag/Environment`.

14.5 README النهائي و Screenshots و Demonstration

- **README: (11.6) + Screenshots** (Home, Product Details, Cart, Stripe Checkout Test, Vendor Dashboard, Admin Dashboard, Order History) + لينك Demo + Test Cards (من Stripe Testing Docs) + Diagrams.
- **Final Demonstration**: سكربت مكتوب ومجرب (الفصل 15)، مع Backup Video لو النت أو Stripe اتعطلوا.
- **Checklist** قبل العرض: Database Seeded. Webhook Endpoint شغال (Stripe Dashboard → Webhooks → Recent deliveries →)، حسابات ال Demo تدخل، Stripe Test Mode فعلاً مفّعل.

15. خطة العرض النهائي (Final Presentation)

15.1 هيكل العرض (حوالي 20 إلى 25 دقيقة [Team Decision])

#	الموضوع	المحتوى	Diagram / Screenshot	المقدم
1	The Problem	بائعون صغار بلا متجر، وعملاء يريدون مكاناً واحداً	Home لـ Screenshot	يتبدل
2	The Proposed Solution	منصة Multi-Vendor بثلاث بوابات	Diagram 1 (System Context)	
3	Project Goals	O1-O5 من الفصل 2	جدول الأهداف	
4	User Roles	Guest/Customer/Vendor/Super Admin	Diagram 2 (Use Case)	
5	Main Features	Storefront, Cart, Checkout, Vendor/Admin	Screenshots	
6	System Architecture	الطبقات والفولدرات	Diagrams 7 و 8	
7	Database Design	الجدول الأساسية و Snapshots	Diagrams 6A و 6C	
8	Security and Authorization	Vendor Isolation	Diagram 22 + Demo قصير لـ 404	
9	Payment Flow	Checkout ← Webhook	Diagrams 15 و 17	
10	Commission System	مثال حساب	جدول 8.4	
11	Team Collaboration	Rotation, Pair, Review	Diagrams 18 و 19 + Matrix	
12	Testing Results	عدد الـ Tests ونتائج الـ 15 مجال	لقطه CI أخضر	
13	Live Demo	(انظر 15.2)		
14	Challenges and Solutions	Migrations, Overselling, Webhooks Conflicts		
15	Future Improvements	Stripe Connect, Shipping, Email, Refunds	جدول 2.8	

كل عضو يقدم جزء ويجاوب أسئلة الأجزاء الأخرى (علشان يثبت إن الكل فاهم المشروع).

15.2 سيناريوهات الـ Live Demo المقترحة

1. **Vendor Onboarding:** Guest Admin ← Vendor يوافق ← الـ Vendor يدخل Dashboard.
2. **Product Creation:** Vendor يضيف منتج بصورة ويظهر في الـ Storefront.
3. **Multi-Vendor Cart:** Customer يضيف منتجات من Vendor ين مختلفين.
4. **Checkout و Stripe Test:** دفع بكارث تجريبي، وشاشة "قيد التأكيد"، ثم الـ Order يتحول لـ Paid (من الـ Webhook).
5. **Vendor Views:** كل Vendor يشوف Items بتاعته بس ويعمل Shipped/Delivered.
6. **Commission:** Admin/Vendor يشوف العمولة وصافي المبلغ.
7. **Security Moment:** Vendor A يحاول فتح URL منتج 404 ← Vendor B.
8. **Idempotency (اختياري):** إعادة إرسال Event بـ Stripe CLI/Dashboard ← مفيش تغيير.

15.3 نصائح التحضير

- جرب الـ Demo 3 مرات على نفس البيئة، وجّهز **Backup Video** و Screenshots.
- جّهز حسابات Demo ومتصفحين (Vendor و Customer) جنب بعض.

- اعرض ال Test Card من Stripe Testing Docs قدامك.
- جهّز إجابات لأسئلة متوقعة: ليه Approach A؟ ليه Reserve Stock؟ ازاي منعنوا Duplicate Webhook؟ ليه مفيش Repository؟ إيه الفرق بين Stripe Checkout و Connect؟

16. المراجع والروابط الرسمية (References)

الروابط دي نقطة البداية للتحقق. الوثائق بتتحدث، فالفريق مسؤول عن مراجعة النسخة الحالية قبل التنفيذ. بعض الروابط ثابتة المنشأ (Microsoft Learn و Stripe Docs)، لكن ممكن تتغير مسارات الصفحات.

Microsoft

الموضوع	المراجع
.NET Support Policy	https://dotnet.microsoft.com/platform/support/policy/dotnet-core
.NET Download	https://dotnet.microsoft.com/download
ASP.NET Core MVC	https://learn.microsoft.com/aspnet/core/mvc/overview
Identity on ASP.NET Core	https://learn.microsoft.com/aspnet/core/security/authentication/identity
Authorization (Roles/Policies)	https://learn.microsoft.com/aspnet/core/security/authorization/introduction
Entity Framework Core	https://learn.microsoft.com/ef/core
EF Core Migrations	https://learn.microsoft.com/ef/core/managing-schemas/migrations
ExecuteUpdate / ExecuteDelete	https://learn.microsoft.com/ef/core/saving/execute-insert-update-delete
Concurrency Tokens	https://learn.microsoft.com/ef/core/saving/concurrency
Areas in ASP.NET Core	https://learn.microsoft.com/aspnet/core/mvc/controllers/areas
Anti-Request Forgery	https://learn.microsoft.com/aspnet/core/security/anti-request-forgery
Safe storage of app secrets	https://learn.microsoft.com/aspnet/core/security/app-secrets
Integration tests	https://learn.microsoft.com/aspnet/core/test/integration-tests
Host and deploy	https://learn.microsoft.com/aspnet/core/host-and-deploy

Stripe

الموضوع	المراجع
Stripe Docs الرئيسية	https://docs.stripe.com
Checkout: Fulfill orders	https://docs.stripe.com/checkout/fulfillment
Webhooks	https://docs.stripe.com/webhooks
Testing	https://docs.stripe.com/testing
Stripe CLI	https://docs.stripe.com/stripe-cli
Stripe Connect	https://docs.stripe.com/connect
Stripe.net (GitHub)	https://github.com/stripe/stripe-dotnet
API Reference	https://docs.stripe.com/api

الموضوع	المرجع
GitHub Docs (Branch protection, PRs, Actions)	https://docs.github.com
Conventional Commits	https://www.conventionalcommits.org
Mermaid	https://mermaid.js.org
Graphviz	https://graphviz.org
OWASP Top 10	https://owasp.org/www-project-top-ten
Bootstrap	https://getbootstrap.com/docs

ملخص ما تم التحقق منه وما لم يتم

- تم التحقق (وقت إعداد الوثيقة): سياسة دعم (.NET 10 LTS).NET، وسلوك أحداث Stripe Checkout الأساسية (checkout.session.completed, async_payment_succeeded, async_payment_failed), ومتطلب Signature Verification بال raw body. وأمر stripe listen.
- لم يتم التحقق ويجب على الفريق مراجعته: أسماء Classes ودوال Stripe.net بالضبط، حدود expires_at، تفاصيل الأحداث الإضافية مثل checkout.session.expired وقواعد Stripe Connect والدول المدعومة، وأسماء Methods في Identity، وأوامر dotnet ef الأحدث، وأسعار وشروط الاستضافة.